

Pranor Ecosystem — Master Technical Architecture Reference Manual

Reference: PRANOR-MASTER-SPEC-2026-V1.0 | 17 Core Modules | Phases 1-87

Pranor Auth — Identity & Access Management

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/auth`

Default Port: 8098

License: AGPL-3.0 (OSS) / Enterprise License (EE with Adaptive MFA & Federation)

Overview

Pranor Auth is the centralized authentication, authorization, and identity management service for the Pranor ecosystem. It provides OAuth2/OIDC provider functionality, WebAuthn/FIDO2 passkey login, adaptive multi-factor authentication, JWT issuance with automatic key rotation, RBAC/ABAC policy enforcement, session management, and SCIM provisioning.

Pranor Auth can run as:

- A **standalone binary** with local user store and JWT signing
- An **integrated module** within the Pranor ecosystem with mTLS, OTel tracing, tenant isolation, and federated IdP support

Key Features

Feature	Description
OAuth2/OIDC Provider	Full Authorization Code (PKCE), Client Credentials, Refresh Token flows with JWKS endpoint
WebAuthn/FIDO2 Passkeys	Hardware keys, biometric authenticators, cross-device synced passkeys
JWT Issuance & Rotation	RS256/ES256 signed tokens with automatic JWKS key rotation via KMS
Adaptive MFA	TOTP, SMS OTP, Email OTP, Magic Links with risk-based step-up challenges
RBAC/ABAC	Hierarchical roles, granular permissions, tenant-scoped policy enforcement
Session Management	Secure session tokens with rotation, device tracking, and bulk invalidation
Social Login	OAuth2 social provider integration (Google, GitHub, etc.)
Credential Stuffing Detection	Real-time detection of credential stuffing attacks
SCIM Provisioning	SCIM v2 user lifecycle management for enterprise directory sync
SPIFFE/SPIRE Exchange	Workload identity attestation via short-lived x509 SVID certificates

Architecture

```
graph TD

subgraph Clients ["■ Auth Ceremony Clients"]
    PasskeyClient["WebAuthn FIDO2 Passkey"]
    MFAClient["TOTP / SMS / Email OTP"]
    OIDCCClient["OAuth2 / OIDC Client (PKCE)"]
end

subgraph Core ["■ Core Identity Engine"]
    SessionMgr["Session Manager and Rotation Engine"]
    AdaptiveMFA["Adaptive Risk-Based Step-Up MFA"]
    JWTProvider["JWT / OIDC Issuer (RS256 / JWKS)"]
    RBACEngine["Granular RBAC / ABAC Policy Engine"]
    SPIFFEExchange["SPIFFE/SPIRE SVID Token Exchanger"]
end

subgraph IdentityStores ["■ Enterprise Identity Provider Federation"]
    FederatedIdP["IdP Mapper (Okta / Azure AD SAML)"]
    UserStore["User Credential Store"]
end

PasskeyClient --> SessionMgr
MFAClient --> AdaptiveMFA
OIDCCClient --> JWTProvider
SessionMgr --> UserStore
AdaptiveMFA --> UserStore
JWTProvider --> RBACEngine
FederatedIdP --> SPIFFEExchange
```

Workload Identity Exchange & Authentication Sequence Flow

```
sequenceDiagram
    autonumber
    participant App as Client / Service Workload
    participant Gate as Pranor Gate Ingress
    participant Auth as Pranor Auth Engine
    participant IdP as Okta / Azure AD (SAML)
    participant SPIFFE as SPIFFE/SPIRE Issuer

    App->>Auth: POST /api/v1/auth/login (Passkey / OAuth2 PKCE)
    Auth->>IdP: Federated Identity Claim Exchange (SAML 2.0)
    IdP-->>Auth: SAML Assertion (User Roles & Group Claims)
    Auth->>SPIFFE: Issue Short-Lived x509 SVID Certificate
    SPIFFE-->>Auth: Signed SVID Workload Identity
    Auth-->>App: RS256 Signed JWT + SPIFFE SVID Certificate
    App->>Gate: Access API (JWT Header + SVID mTLS)
    Gate->>Auth: Introspect Token & Verify RBAC Claims
    Auth-->>Gate: Token Validated & Permissions Granted
```

Ecosystem Cross-Module Integration

Pranor Auth establishes zero-trust identity across all platform components:

- **Pranor Gate:** Enforces route-level JWT signature checks, SAML attribute mapping, and SPIFFE/SPIRE workload authentication.
- **Pranor Secret:** Uses authenticated user identities to authorize access to encrypted vault keys and environment secret maps.
- **Pranor Notify:** Triggers multi-factor authentication (MFA) Email/SMS one-time passcodes during step-up login ceremonies.
- **Pranor Console:** Managed via Auth RBAC roles, granting workspace administrators granular cluster control plane privileges.

Installation & Deployment

Binary

```
cd pranor/auth
go build -o pranor-auth .
./pranor-auth --port 8098
```

Docker

```
docker run -p 8098:8098 ghcr.io/vyuvaraj/pranor-auth:latest
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Auth integrates automatically with Gate (JWT enforcement), Trace (OTel spans), Secret (key storage), and Console (dashboard visibility).

Configuration

Environment Variables

Variable	Default	Description
-----	-----	-----
<code>`PORT`</code>	<code>`8098`</code>	HTTP listener port
<code>`PRANOR_AUTH_JWT_ALGORITHM`</code>	<code>`RS256`</code>	JWT signing algorithm (<code>`RS256`</code> or <code>`ES256`</code>)
<code>`PRANOR_AUTH_JWT_KEY_PATH`</code>	—	Path to RSA/EC private key for JWT signing
<code>`PRANOR_AUTH_SESSION_SECRET`</code>	—	32-byte secret for session token signing
<code>`PRANOR_AUTH_MFA_TOTP_ISSUER`</code>	<code>`Pranor`</code>	TOTP issuer name shown in authenticator apps
<code>`PRANOR_AUTH_PRANOR_NOTIFY_URL`</code>	—	Pranor Notify URL for email/SMS OTP delivery
<code>`PRANOR_AUTH_OTEL_ENDPOINT`</code>	—	OpenTelemetry collector URL
<code>`PRANOR_AUTH_KMS_ROTATION_INTERVAL`</code>	<code>`24h`</code>	KMS envelope key rotation interval

YAML Config (``auth.yaml``)

```
port: "8098"
jwt_algorithm: "RS256"
jwt_key_path: "/keys/auth-signing.pem"
session_secret: "32-byte-random-secret-here"
mfa_totp_issuer: "Pranor"
notify_url: "http://pranor-notify:8094"
otel_endpoint: "http://pranor-trace:8090"
```

CLI Flags

Flag	Default	Description
-----	-----	-----
<code>`--port`</code>	<code>`8098`</code>	HTTP listen port

API Reference

Base URL: ``http://localhost:8098``
API Version: ``/api/v1/`` (recommended) or ``/api/`` (legacy)

POST `/api/auth/register`

Register a new user.

Request:

```
{
```

```
"username": "alice",
"password": "secure-password-123",
"email": "alice@example.com"
}
```

Response (201):

```
{
  "status": "success",
  "user_id": "usr-abc-123",
  "message": "User registered successfully"
}
```

POST /api/auth/login

Authenticate a user and receive a JWT.

Request:

```
{
  "username": "alice",
  "password": "secure-password-123"
}
```

Response (200):

```
{
  "token": "eyJhbGciOiJSUzI1NiIs... ",
  "expires_at": "2026-08-01T11:00:00Z",
  "user_id": "usr-abc-123"
}
```

POST /api/auth/passkey/register/challenge

Begin WebAuthn passkey registration ceremony.

Request:

```
{
  "user_id": "usr-abc-123"
}
```

Response (200):

```
{
  "challenge": "base64-encoded-challenge",
  "rp": { "name": "Pranor", "id": "pranor.net" },
  "user": { "id": "usr-abc-123", "name": "alice" }
}
```

POST /api/auth/passkey/login/challenge

Begin WebAuthn authentication ceremony.

Request:

```
{
  "username": "alice"
}
```

Response (200):

```
{
  "challenge": "base64-encoded-challenge",
  "allowCredentials": [{ "id": "cred-xyz", "type": "public-key" }]
}
```

POST /api/auth/mfa/setup

Set up MFA for a user (TOTP, SMS, or Email).

Request:

```
{
  "user_id": "usr-abc-123",
  "method": "totp"
}
```

Response (200):

```
{
  "secret": "JBSWY3DPEHPK3PXP",
  "qr_code_url": "otpauth://totp/Pranor:alice?secret=JBSWY3DPEHPK3PXP&issuer;=Pranor"
}
```

POST /api/auth/mfa/step-up

Request adaptive MFA step-up based on risk signals.

Request:

```
{
  "user_id": "usr-abc-123",
  "context": {
    "ip": "203.0.113.42",
    "device_fingerprint": "fp-new-device",
    "action": "high-value-transfer"
  }
}
```

```
}
```

Response (200):

```
{
  "step_up_required": true,
  "risk_score": 78,
  "required_factors": ["totp"],
  "reason": "new_device_detected"
}
```

GET /.well-known/jwks.json

JSON Web Key Set for token verification.

Response (200):

```
{
  "keys": [
    {
      "kty": "RSA",
      "kid": "key-2026-08",
      "use": "sig",
      "alg": "RS256",
      "n": "...",
      "e": "AQAB"
    }
  ]
}
```

POST /api/auth/sessions/revoke

Invalidate all sessions for a user.

Request:

```
{
  "user_id": "usr-abc-123"
}
```

Response (200):

```
{
  "status": "success",
  "revoked_count": 3
}
```

GET /healthz

Liveness probe.

```
{"status":"ok"}
```

Security

Standalone Mode

In standalone mode, Prantor Auth uses a local user store with bcrypt-hashed passwords and issues self-signed JWTs. Configure ``PRANOR_AUTH_SESSION_SECRET`` for session signing.

Ecosystem Mode (Full Auth Stack)

When running within the Prantor ecosystem, the full middleware chain activates:

1. **OTel Tracing** — every request gets a span
2. **Rate Limiting** — per-client request throttling
3. **CORS** — cross-origin request handling
4. **Max Body Size** — 10MB request body limit
5. **JWT Auth** — validates Bearer tokens
6. **Token Revocation** — checks revocation list
7. **Tenant Isolation** — multi-tenant namespace enforcement

mTLS / SPIFFE

Enable mutual TLS for service-to-service authentication with SPIFFE SVID certificates. Auth issues short-lived x509 workload identities for zero-trust inter-service communication.

KMS Key Rotation

Background KMS envelope key rotation runs on a configurable schedule (default: 24h). JWKS endpoints serve both current and previous keys during rollover for zero-downtime rotation.

Observability

Prometheus Metrics

Metric	Type	Description
----- ----- -----		
`prantor_auth_logins_total`	Counter	Total login attempts (labeled by method, status)
`prantor_auth_mfa_challenges_total`	Counter	MFA challenges issued
`prantor_auth_token_issued_total`	Counter	JWTs issued
`prantor_auth_sessions_active`	Gauge	Currently active sessions
`prantor_auth_stuffing_blocks_total`	Counter	Credential stuffing attacks blocked

OpenTelemetry Tracing

Every authentication flow generates OTEL spans:

- ``auth.login`` — full login ceremony
- ``auth.mfa.verify`` — MFA verification step
- ``auth.token.issue`` — JWT generation
- ``auth.passkey.ceremony`` — WebAuthn challenge/response

Logging

Structured JSON logs with fields: ``level``, ``timestamp``, ``trace_id``, ``user_id``, ``action``, ``ip``, ``risk_score``.

Enterprise Edition

Feature OSS EE
----- :--- :--
Local user store & JWT issuance ✓ ✓
TOTP/Email/SMS MFA ✓ ✓
WebAuthn/FIDO2 Passkeys ✓ ✓
Session management & revocation ✓ ✓
RBAC roles & permissions ✓ ✓
Social login (OAuth2 providers) ✓ ✓
SCIM v2 provisioning ✓ ✓
Adaptive Risk-Based MFA Step-Up — ✓
Device Fingerprinting & Trusted Device Registry — ✓
Per-Tenant OIDC Federation (Okta, Azure AD, Google) — ✓
SPIFFE/SPIRE Workload Identity Exchange — ✓
Credential Stuffing Detection Engine — ✓

Operational Runbook

Users cannot log in

1. Check ``/healthz`` endpoint is returning 200
2. Verify JWT signing key is accessible (``PRANOR_AUTH_JWT_KEY_PATH``)
3. Check logs for ``auth.login`` span errors
4. If MFA is failing, verify Prnor Notify connectivity for OTP delivery
5. Check rate limiter isn't blocking legitimate traffic

JWT tokens rejected by downstream services

1. Verify JWKS endpoint (``/.well-known/jwks.json``) is accessible from downstream services

2. Check if key rotation occurred — downstream services may be caching stale keys
3. Ensure clock skew between Auth and consumer services is < 30 seconds
4. Check token hasn't been explicitly revoked via ``/api/auth/sessions/revoke``

High credential stuffing alerts

1. Monitor ``pranor_auth_stuffing_blocks_total`` metric
2. Review blocked IPs in logs
3. Consider enabling adaptive MFA step-up for all logins from flagged IPs
4. Integrate with upstream WAF for IP-level blocking

KMS key rotation failures

1. Check KMS connectivity and credentials
2. Verify rotation interval configuration (``PRANOR_AUTH_KMS_ROTATION_INTERVAL``)
3. Monitor logs for ``kms.rotation`` errors
4. Manual key rotation: ``POST /api/auth/rotate-keys``

Pranor Cache — Distributed Caching Engine

Version: 0.1.0

Module Path: `github.com/vyuvaraj/pranor/cache`

Default Port: 8086

License: AGPL-3.0 (OSS) / Enterprise License (EE with TLS offload & SIMD vector cache)

Overview

Pranor Cache is a distributed, high-performance caching service for the Pranor ecosystem. It exposes a low-latency REST API backed by pluggable engines (in-memory or Redis) with native support for OpenTelemetry context propagation, read-through/write-behind database synchronization, key pattern invalidation, bloom filter guards, multi-region replication, and a Redis wire protocol adapter.

Pranor Cache can run as:

- A **standalone binary** with zero external dependencies (in-memory engine)
- An **integrated module** within the Pranor ecosystem with mTLS, OTel tracing, and multi-region sync

Key Features

Feature	Description
Pluggable Engines	Swap transparently between thread-safe in-memory storage and Redis/Valkey clusters
TTL Eviction	Automatic background time-based pruning of expired cache keys
Key Pattern Invalidation	Delete matching keys via wildcards and prefix matching
Read-Through Cache	Misses auto-load from backend database and populate the cache
Write-Behind Cache	Writes asynchronously update the backend database for eventual consistency
Multi-Region Replication	Forward mutations to peer cache nodes for global consistency
Bloom Filter Guard	Probabilistic filter prevents unnecessary backend lookups on non-existent keys
Redis Wire Protocol	RESP-compatible adapter allows existing Redis clients to connect directly
SIMD Vector Similarity	AVX-512 accelerated cosine-distance vector cache for LLM embedding lookups
Multi-Tenant Pools	Isolated memory pools per tenant to prevent noisy-neighbor issues
OTel Instrumentation	Hit/miss/latency metrics exported via OpenTelemetry tracing context

Architecture



```
RedisProto["Redis Wire Protocol Adapter"]
end

subgraph Core ["■ Core Cache Engine"]
MemGrid["Thread-Safe In-Memory Data Grid"]
SIMDVector["SIMD AVX-512 Vector Similarity Cache"]
BloomFilter["Probabilistic Bloom Filter Guard"]
MultiTenantPool["Multi-Tenant Isolation Memory Pool"]
end

subgraph Persistence ["■ Pluggable Backends and DB Sync"]
RedisCluster["Redis / Valkey Cluster"]
ReadThrough["Read-Through and Write-Behind DB Sync"]
ActiveMirror["Active-Active Multi-Cluster Sync"]
end

API --> MemGrid
RedisProto --> MemGrid
MemGrid --> SIMDVector
SIMDVector --> BloomFilter
BloomFilter --> MultiTenantPool
MultiTenantPool --> RedisCluster
MultiTenantPool --> ReadThrough
MultiTenantPool --> ActiveMirror
```

Read-Through & SIMD Vector Cache Sequence Flow

```
sequenceDiagram
    autonumber
    participant App as Microservice / LLM Client
    participant Cache as Prnor Cache Engine
    participant SIMD as SIMD AVX-512 Vector Engine
    participant DB as Backend Database / S3 Store

    App->>Cache: GET /api/cache/prompt-embedding (Cosine Distance < 0.05)
    Cache->>SIMD: Search In-Memory Vector Cache via SIMD AVX-512
    alt Cache Hit (Vector Distance Match)
        SIMD-->>Cache: Cached LLM Response Payload
        Cache-->>App: 200 OK (Instant Cache Hit <50µs)
    else Cache Miss
        SIMD-->>Cache: Cache Miss / Entry Expired
        Cache->>DB: Read-Through Fetch from Backend Storage
        DB-->>Cache: Fresh Payload Data
        Cache->>Cache: Asynchronously Populate Cache Entry & Update Bloom Filter
        Cache-->>App: 200 OK (Read-Through Response)
    end
end
```

Ecosystem Cross-Module Integration

Prnor Cache provides sub-millisecond data acceleration across all platform components:

- **Prnor Gate:** Accelerates semantic prompt caching and API response caching for high-frequency ingress routes.
- **Prnor Vault:** Caches HNSW vector graph nodes and S3 object metadata in memory for sub-5ms query performance.
- **Prnor Auth:** Stores active user session tokens, OAuth2 authorization grants, and rate-limiting counters.
- **Prnor Trace:** Exports cache hit/miss ratio metrics, memory pool allocations, and latency exemplars via OpenTelemetry.

Installation & Deployment

Binary

```
cd prnor/cache
go build -o prnor-cache .
./prnor-cache --port 8086
```

Docker

```
docker run -p 8086:8086 ghcr.io/vyuvraj/prnor-cache:latest
```

With Redis Backend

```
./pranor-cache --port 8086 --backend redis --redis-url redis://localhost:6379
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Cache integrates automatically with Auth (JWT/mTLS), Trace (OTel spans), and Console (dashboard visibility).

Configuration

Environment Variables

Variable	Default	Description
-----	-----	-----
<code>`PORT`</code>	<code>`8086`</code>	HTTP Server port
<code>`REDIS_URL`</code>	—	Redis cluster URL. Uses in-memory engine if unset
<code>`PRANOR_CACHE_BACKEND_DB`</code>	—	Backend database URL for read-through & write-behind sync
<code>`PRANOR_CACHE_PEERS`</code>	—	Comma-separated peer URLs for multi-region replication
<code>`PRANOR_CACHE_TLS_CERT`</code>	—	Path to TLS certificate for HTTPS
<code>`PRANOR_CACHE_TLS_KEY`</code>	—	Path to TLS private key
<code>`PRANOR_OTLP_ENDPOINT`</code>	—	OpenTelemetry collector URL

YAML Config (`cache.yaml`)

```
port: "8086"

backend: "memory" # "memory" or "redis"

redis_url: "redis://localhost:6379"

backend_db: "" # read-through DB endpoint

peers: [] # peer cache nodes for replication

tls_cert: ""

tls_key: ""
```

CLI Flags

Flag	Default	Description
-----	-----	-----
<code>`--port`</code>	<code>`8086`</code>	HTTP listen port
<code>`--backend`</code>	<code>`memory`</code>	Cache backend: <code>`memory`</code> or <code>`redis`</code>
<code>`--redis-url`</code>	<code>`redis://localhost:6379`</code>	Redis connection URL
<code>`--version`</code>	—	Print version and exit

API Reference

Base URL: `http://localhost:8086`

POST /api/cache

Set a cache entry.

Request:

```
{
  "key": "user:101",
  "value": { "name": "Alice", "role": "admin" },
  "ttl": "5m"
}
```

Response (200):

```
{
  "status": "success",
  "key": "user:101"
}
```

GET /api/cache/{key}

Get a cache entry.

Response (200):

```
{
  "key": "user:101",
  "value": { "name": "Alice", "role": "admin" }
}
```

Response (404):

```
{
  "status": "not_found",
  "key": "user:101"
}
```

DELETE /api/cache/{key}

Delete a specific cache entry.

Response (200):

```
{
  "status": "deleted",
  "key": "user:101"
}
```

DELETE /api/cache?pattern={pattern}

Invalidate keys by pattern. If no pattern is provided, clears the entire cache.

Response (200):

```
{
  "status": "success",
  "invalidated": 42
}
```

GET /health

Health probe showing cache readiness and connection status.

Response (200):

```
{"status": "UP", "service": "pranor-cache", "version": "0.1.0", "backend": "memory"}
```

Security

Standalone Mode

In standalone mode, Pranor Cache runs without authentication. Suitable for development and testing.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem (detected automatically), the full middleware chain activates:

1. **OTel Tracing** — every request gets a span
2. **Rate Limiting** — per-client request throttling
3. **CORS** — cross-origin request handling
4. **Max Body Size** — 10MB request body limit
5. **JWT Auth** — validates Bearer tokens against Pranor Auth
6. **Tenant Isolation** — multi-tenant namespace enforcement

TLS

Enable HTTPS with TLS certificates:

```
tls_cert: "/certs/cache.crt"
tls_key: "/certs/cache.key"
```

TLS offload is an Enterprise feature that uses optimized kernel-bypass SSL termination.

Observability

Prometheus Metrics

Metric	Type	Description
----- ----- -----		
`pranor_cache_hits_total`	Counter	Cache hit count
`pranor_cache_misses_total`	Counter	Cache miss count
`pranor_cache_keys_active`	Gauge	Currently stored keys
`pranor_cache_evictions_total`	Counter	Keys evicted by TTL
`pranor_cache_read_through_total`	Counter	Read-through backend fetches
`pranor_cache_replication_lag_ms`	Histogram	Peer replication latency

OpenTelemetry Tracing

- Every cache operation generates OTel spans:
- `cache.get` — read operation with hit/miss attribute
 - `cache.set` — write operation with TTL
 - `cache.delete` — deletion/invalidation
 - `cache.read\_through` — backend fetch on miss

Logging

Structured JSON logs with fields: `level`, `timestamp`, `trace\_id`, `operation`, `key`, `hit`, `latency\_us`.

Enterprise Edition

Feature	OSS	EE
----- --- ---		
In-memory cache engine	✓	✓
Redis/Valkey backend	✓	✓
TTL eviction	✓	✓
Key pattern invalidation	✓	✓
Read-through / Write-behind	✓	✓
Multi-region peer replication	✓	✓
Bloom filter guard	✓	✓
TLS offload (kernel-bypass SSL)	—	✓
SIMD AVX-512 vector similarity cache	—	✓
Multi-tenant memory pool isolation	—	✓
Redis wire protocol adapter	—	✓
Active-active multi-cluster sync	—	✓

Operational Runbook

High cache miss rate

1. Check `/health` endpoint for backend connectivity
2. Verify TTLs aren't too short for workload patterns
3. Review bloom filter effectiveness — false positive rate should be $< 1\%$
4. If using read-through, check backend DB latency via `pranor\_cache\_read\_through\_total`
5. Consider increasing memory allocation for the in-memory engine

Replication lag between regions

1. Monitor `pranor\_cache\_replication\_lag\_ms` histogram
2. Check network connectivity to peer nodes (`PRANOR\_CACHE\_PEERS`)
3. Verify peer URLs are reachable and responding to health checks
4. Consider reducing write volume if replication can't keep up

Memory pressure / OOM

1. Check `pranor\_cache\_keys\_active` gauge for key count growth
2. Review TTL policies — ensure all entries have finite TTLs
3. Use pattern invalidation to bulk-remove stale namespaces
4. If using multi-tenant pools, check per-tenant quotas

Redis backend connection failures

1. Verify `REDIS\_URL` is correct and Redis is reachable
2. Check Redis cluster health (CLUSTER INFO)
3. Pranor Cache falls back to in-memory in standalone mode
4. Monitor reconnection attempts in structured logs

Pranor Chrono — Distributed Job Scheduler

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/chrono`

Default Port: 8087

License: AGPL-3.0 (OSS) / Enterprise License (EE with smart scheduling & timezone DSL)

Overview

Pranor Chrono is the distributed, fault-tolerant job scheduling service for the Pranor ecosystem. It supports interval and cron scheduling, exactly-once semantics, DAG job chaining, Pranor cron-as-code declarations, persistent S3 job registries, leader election, retry policies with configurable backoff, and full OTel tracing.

Pranor Chrono can run as:

- A **standalone binary** with single-node scheduling (no Redis required)
- An **integrated module** within the Pranor ecosystem with distributed leader election, mTLS, and OTel tracing

Key Features

Feature	Description
Interval & Cron	Run jobs at fixed intervals or standard 5-field cron patterns
Exactly-once semantics	Redis-based leader election ensures only one node fires each job
DAG Job Chaining	Multi-step dependency graphs with topological sort execution
Cron-as-Code	Define jobs in .pnr files with hot-reload on change
Retry Policies	Fixed, linear, or exponential backoff with jitter
Dead Letter Queue	Jobs exhausting retries are moved to DLQ for audit
S3 Persistence	Job registry and audit logs persisted to Pranor Vault S3
Leader Election	Redis-based distributed lease ensures cluster-safe scheduling
OTel Tracing	traceparent headers propagated to all HTTP callbacks
Fan-out / Fan-in	Parallelize independent steps, synchronize at join points

Architecture

```
graph TD
    subgraph API ["Scheduler Control and Cron-as-Code"]
        CronAsCode["Pranor Language .pnr Watcher"]
        JobAPI["REST Scheduler API"]
    end
```

```

subgraph SchedulerCore ["■ Distributed Timer and DAG Engine"]
CronEvaluator["High-Precision Cron Evaluator"]
LeaderLock["Pranor Lock Fencing Token Leader"]
DAGRunner["DAG Topological Fan-Out and Join Engine"]
HTTPDispatcher["HTTP Callback Dispatcher"]
end

subgraph History ["■ Audit Trail and Vault Storage"]
VaultS3["Pranor Vault S3 Job Registry and Audit Logs"]
RetryEngine["Exponential Backoff Retry Engine"]
end

CronAsCode --> CronEvaluator
JobAPI --> CronEvaluator
CronEvaluator --> LeaderLock
LeaderLock --> DAGRunner
DAGRunner --> HTTPDispatcher
HTTPDispatcher --> RetryEngine
RetryEngine --> VaultS3

```

High-Precision Distributed Cron Trigger Sequence Flow

```

sequenceDiagram
    autonumber
    participant Chrono as Pranor Chrono Leader
    participant Lock as Pranor Lock Manager
    participant Service as Target Microservice
    participant Vault as Pranor Vault S3
    participant Trace as Pranor Trace

    Chrono->>Lock: Acquire Job Execution Lease (Key: "cron/cleanup-db")
    Lock-->>Chrono: Granted (Fencing Token = 2088)
    Note over Chrono: Evaluate Cron Expression & Trigger Sub-ms TimeWheel
    Chrono->>Service: POST /tasks/cleanup (Traceparent + Fencing Token)
    Service-->>Chrono: 200 OK (Task Completed in 140ms)
    Chrono->>Vault: Write Audit Execution Log (audit/cleanup-db_20260803.json)
    Chrono->>Trace: Emit OTel Span with Job Execution Metrics
    Chrono->>Lock: Release Job Lease (Token = 2088)

```

Ecosystem Cross-Module Integration

Pranor Chrono manages high-precision job scheduling across all platform services:

- **Pranor Lock:** Uses exclusive fencing token leases to guarantee job callbacks execute on exactly one node during multi-replica deployments.

- **Pranor Vault:** Persists serialized ``jobs.json`` configurations and append-only execution audit logs.
- **Pranor Flow:** Triggers scheduled workflow sagas and periodic maintenance DAGs.
- **Pranor Trace:** Emits OpenTelemetry trace spans with ``traceparent`` context headers for every dispatched cron job.

Installation & Deployment

Binary

```
cd pranor/chrono
go build -o pranor-chrono .
./pranor-chrono --addr :8087
```

Docker

```
docker run -p 8087:8087 ghcr.io/vyuvaraj/pranor-chrono:latest
```

With Redis Leader Election

```
./pranor-chrono --addr :8087 --redis-url redis://localhost:6379
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Chrono integrates automatically with Lock (leader election), Vault (persistence), Trace (OTel spans), and Console (dashboard visibility).

Configuration

Environment Variables

Variable	Default	Description
-----	-----	-----
<code>`PORT`</code>	<code>`8087`</code>	HTTP listener port
<code>`REDIS_URL`</code>	—	Redis URL for distributed leader election
<code>`REDIS_LOCK_KEY`</code>	<code>`pranor-chrono:leader:lock`</code>	Redis key for leader lease lock
<code>`REDIS_LEASE_DURATION`</code>	<code>`15s`</code>	Lease duration for leader election
<code>`PRANOR_CHRONO_PRANOR_VAULT_URL`</code>	—	Pranor Vault URL for job persistence
<code>`PRANOR_CHRONO_PRANOR_VAULT_BUCKET`</code>	<code>`pranor-chrono-jobs`</code>	S3 bucket name for job registry
<code>`PRANOR_CHRONO_OTEL_ENDPOINT`</code>	—	OpenTelemetry collector URL
<code>`PRANOR_CHRONO_PRANOR_FILES_DIR`</code>	—	Directory to watch for <code>` .pnr `</code> job definitions

YAML Config (`chrono.yaml`)

```
port: "8087"
redis_url: "redis://localhost:6379"
redis_lock_key: "pranor-chrono:leader:lock"
redis_lease_duration: "15s"
vault_url: "http://pranor-vault:7070"
vault_bucket: "pranor-chrono-jobs"
otel_endpoint: "http://pranor-trace:8090"
pnr_files_dir: "./jobs"
```

CLI Flags

Flag	Default	Description
----	-----	-----
<code>--addr</code>	<code>:8087</code>	HTTP listening address
<code>--redis-url</code>	—	Redis URL for leader election
<code>--redis-lock-key</code>	<code>pranor-chrono:leader:lock</code>	Redis key for leader lease
<code>--redis-lease-duration</code>	<code>15s</code>	Leader lease duration

API Reference

Base URL: `http://localhost:8087`
API Version: `/api/v1/` (recommended) or `/api/` (legacy)

POST `/api/v1/jobs`

Create a scheduled job.

Request:

```
{
  "name": "health-check",
  "schedule": "30s",
  "callback_url": "http://myapp/health",
  "retry": {
    "max": 3,
    "backoff": "exponential"
  }
}
```

Response (201):

```
{
  "id": "job-abc-123",
  "name": "health-check",
  ...
}
```

```
"schedule": "30s",
"status": "active",
"next_run": "2026-08-01T10:00:30Z"
}
```

GET /api/v1/jobs

List all jobs.

Response (200):

```
{
  "jobs": [
    {
      "id": "job-abc-123",
      "name": "health-check",
      "schedule": "30s",
      "status": "active",
      "last_run": "2026-08-01T10:00:00Z",
      "next_run": "2026-08-01T10:00:30Z"
    }
  ]
}
```

POST /api/v1/jobs/{id}/run

Trigger a job manually (ignores schedule).

Response (200):

```
{
  "status": "triggered",
  "execution_id": "exec-xyz-789"
}
```

POST /api/v1/dag

Define a DAG job chain.

Request:

```
{
  "name": "nightly-pipeline",
  "schedule": "0 2 * * *",
  "steps": [
    { "id": "extract", "callback_url": "http://etl/extract", "depends_on": [] },
  ]
}
```

```
{ "id": "transform", "callback_url": "http://etl/transform", "depends_on": ["extract"] },
{ "id": "load", "callback_url": "http://etl/load", "depends_on": ["transform"] }
]
```

Response (201):

```
{
  "id": "dag-001",
  "name": "nightly-pipeline",
  "status": "active",
  "step_count": 3
}
```

GET /api/v1/jobs/{id}/history

Execution history for a job.

Response (200):

```
{
  "executions": [
    {
      "id": "exec-001",
      "started_at": "2026-08-01T10:00:00Z",
      "duration_ms": 140,
      "status": "success",
      "http_status": 200
    }
  ]
}
```

GET /healthz

Liveness probe.

```
{"status":"UP","service":"pranor-chrono","version":"1.0.0"}
```

Security

Standalone Mode

In standalone mode, Pranor Chrono runs with single-node scheduling and no Redis dependency. No authentication is required.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem, the full middleware chain activates:

1. **OTel Tracing** — every request gets a span
2. **Rate Limiting** — per-client request throttling
3. **CORS** — cross-origin request handling
4. **Max Body Size** — 10MB request body limit
5. **JWT Auth** — validates Bearer tokens against Pranor Auth
6. **Tenant Isolation** — multi-tenant namespace enforcement

Job Callback Security

Job callbacks include:

- ``traceparent`` header for distributed tracing
- Fencing token from Pranor Lock leader lease
- Optional bearer token for authenticated callbacks

Observability

Prometheus Metrics

Metric	Type	Description
----- ----- -----		
<code>`pranor_chrono_jobs_active`</code>	Gauge	Currently registered active jobs
<code>`pranor_chrono_fires_total`</code>	Counter	Total job fires (labeled by job name, status)
<code>`pranor_chrono_execution_duration_ms`</code>	Histogram	Job execution duration
<code>`pranor_chrono_retries_total`</code>	Counter	Total retry attempts
<code>`pranor_chrono_dlq_depth`</code>	Gauge	Dead letter queue depth
<code>`pranor_chrono_leader_elections_total`</code>	Counter	Leader election events

OpenTelemetry Tracing

Every job execution generates OTel spans:

- ``chronoschedule.evaluate`` — cron expression evaluation
- ``chronojob.dispatch`` — HTTP callback dispatch
- ``chronojob.retry`` — retry attempt
- ``chronoleader.acquire`` — leader lease acquisition

Logging

Structured JSON logs with fields: ``level``, ``timestamp``, ``trace_id``, ``job_id``, ``execution_id``, ``status``, ``duration_ms``.

Enterprise Edition

Feature OSS EE
----- :--- :---
Interval & cron scheduling ✓ ✓
DAG job chaining ✓ ✓
Leader election (Redis) ✓ ✓
Retry policies ✓ ✓
Cron-as-Code (.pnr) ✓ ✓
S3 job persistence ✓ ✓
OTel tracing ✓ ✓
Smart scheduling (load-aware distribution) — ✓
Timezone-aware cron DSL — ✓
Multi-cluster job federation — ✓
AI-powered schedule optimization — ✓

Operational Runbook

Jobs not firing

1. Check leader election — only the leader fires jobs. Verify Redis connectivity
2. Review `/api/v1/jobs` to confirm job status is `active`
3. Check `pranor\_chrono\_leader\_elections\_total` metric for frequent re-elections
4. Verify callback URLs are reachable from the Chrono node
5. Check `REDIS\_LEASE\_DURATION` isn't too short causing leader thrashing

DAG steps stuck in pending

1. Check `/api/v1/dag/{id}` for step dependency resolution status
2. Verify upstream step completed successfully (check execution history)
3. Look for circular dependencies in step definitions
4. Check if step callback URL is timing out

High retry rate

1. Monitor `pranor\_chrono\_retries\_total` metric by job name
2. Check callback service health and response times
3. Review backoff strategy — exponential with jitter prevents thundering herds
4. Consider increasing timeout for slow callbacks
5. Jobs exhausting retries move to DLQ — check DLQ depth

Leader election instability

1. Monitor Redis connectivity and latency
2. Check `REDIS\_LEASE\_DURATION` (default 15s) — too short causes flapping

3. Verify clock synchronization between Chrono nodes
4. Check network partitions between nodes and Redis

Pranor Console — Unified Management Dashboard

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/console`

Default Port: 8083

License: AGPL-3.0 (OSS) / Enterprise License (EE with AI Co-Pilot & Chaos Panel)

Overview

Pranor Console is the unified, premium management dashboard and observability console for the Pranor ecosystem. It provides a single pane of glass for managing all Pranor components — Gate, Pulse, Vault, Mesh, Deploy, Trace, Flow, and more — with a glassmorphic, real-time UI designed for power users. It features global search, chaos engineering controls, incident management, eBPF flamegraphs, and WebSocket-driven live telemetry.

- Pranor Console can run as:
- A **standalone binary** serving the web UI with manual service URL configuration
 - An **integrated module** within the Pranor ecosystem with auto-discovery, mTLS, and federated telemetry

Key Features

Feature	Description
Single Pane of Glass	Manage the entire Pranor stack from one premium glassmorphic UI
Global ■ K Search	Fuzzy search across all resources — services, routes, queues, buckets, workflows
API Gateway Management	Live route audits, WASM hot-swap, circuit breaker status board
Queue Inspector	Topic browser, DLQ replay, consumer group lag dashboard
Storage Inspector	Bucket browser, vector index namespaces, branch management
eBPF Flamegraphs	Live CPU/memory profiling from the kernel layer
SLO Burn Rate	Real-time error budget dashboards with fast/slow windows
Chaos Engineering	Design, trigger, and monitor chaos experiments
Service Topology	Interactive dependency map with live traffic flow edges
Incident Manager	Alert rules, triage, severity management, resolution tracking
Environment Provisioner	One-click isolated environments and branch previews
SQL Workbench	Interactive query editor with schema exploration

Architecture

```
graph TD
    subgraph UserInterface ["■ Glassmorphic Web and TUI Interface"]
```

```
SPA["React / WASM Glassmorphic SPA"]
TUI["Terminal TUI Control Plane"]
WSClient["WebSocket Live Telemetry Stream"]
end

subgraph BackendCore ["■ Central Control Plane Backend"]
  SearchEngine["Global Ecosystem ■K Indexer"]
  ChaosControl["Chaos Experiment Orchestrator"]
  IncidentEngine["Incident Triage and Alert Engine"]
  AIAssistant["Autonomous AI Co-Pilot"]
end

subgraph ServiceIntegrations ["■ Platform Services Monitoring Hub"]
  GateSync["Pranor Gate API Sync"]
  PulseSync["Pranor Pulse Queue and DLQ Sync"]
  VaultSync["Pranor Vault Bucket and Vector Sync"]
  TraceSync["Pranor Trace and eBPF Flamegraph Sync"]
end

SPA --> SearchEngine
TUI --> SearchEngine
WSClient --> SearchEngine
SearchEngine --> ChaosControl
SearchEngine --> IncidentEngine
SearchEngine --> AIAssistant
AIAssistant --> GateSync
AIAssistant --> PulseSync
AIAssistant --> VaultSync
AIAssistant --> TraceSync
```

Real-Time WebSocket Telemetry Stream & Global Search Sequence Flow

```
sequenceDiagram
    autonumber
    participant Admin as Admin as Cluster Operator / Web UI
    participant Console as Console as Pranor Console Backend
    participant Gate as Gate as Pranor Gate / Pulse / Vault
    participant Trace as Trace as Pranor Trace Engine
    participant AI as AI as Autonomous AI Co-Pilot

    Admin->>Console: Open Console Dashboard & Trigger 🔍 Search ("vector-index-01")
    Console->>Console: Index & Match Cross-Module Resources in Memory
    Console-->>Admin: Display Instant Search Matches (<5ms)
    Console->>Gate: Subscribe to Live WebSocket Metrics Stream (/ws/feeds)
    Gate-->>Console: Stream Throughput, Latency & Error Telemetry
    Console->>Trace: Query High-Burn SLO Spans & eBPF Flamegraphs
    Trace-->>Console: Correlated Flamegraph + Span Waterfall
    Console->>AI: Analyze Cluster Anomaly & Suggest Auto-Remediation
    AI-->>Admin: Render Remediation Action Card in Glassmorphic Panel
```

Ecosystem Cross-Module Integration

Pranor Console provides single-pane-of-glass management for all platform components:

- **Pranor Gate:** Inspects dynamic HTTP routes, hot-swaps WASM security modules, and monitors AI token costs.
- **Pranor Pulse:** Browses topics, tracks consumer group partition lag, and performs 1-click DLQ message triage.
- **Pranor Vault:** Visualizes HNSW vector graph indexes, browses S3 buckets, and manages CoW bucket branches.
- **Pranor Trace:** Renders interactive eBPF CPU flamegraphs, distributed service dependency maps, and SLO burn rate dashboards.

Installation & Deployment

Binary

```
cd pranor/console
go build -o pranor-console .
./pranor-console --port 8083
```

Docker

```
docker run -p 8083:8083 ghcr.io/vyuvaraj/pranor-console:latest
```

With Service Discovery

```
docker run -p 8083:8083 \
-e PRANOR_CONSOLE_PRANOR_GATE_URL=http://pranor-gate:8080 \
-e PRANOR_CONSOLE_PRANOR_PULSE_URL=http://pranor-pulse:9090 \
-e PRANOR_CONSOLE_PRANOR_VAULT_URL=http://pranor-vault:7070 \
-e PRANOR_CONSOLE_PRANOR_TRACE_URL=http://pranor-trace:8090 \
-e PRANOR_CONSOLE_PRANOR_MESH_URL=http://pranor-mesh:8089 \
ghcr.io/vyuvaraj/pranor-console:latest
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Console auto-discovers all services via Mesh and displays the full topology.

Configuration

Environment Variables

Variable	Default	Description
----- ----- -----		
`PRANOR_CONSOLE_PORT`	`8083`	HTTP port
`PRANOR_CONSOLE_PRANOR_GATE_URL`	—	Pranor Gate backend URL
`PRANOR_CONSOLE_PRANOR_PULSE_URL`	—	Pranor Pulse backend URL
`PRANOR_CONSOLE_PRANOR_VAULT_URL`	—	Pranor Vault backend URL
`PRANOR_CONSOLE_PRANOR_TRACE_URL`	—	Pranor Trace OTLP URL
`PRANOR_CONSOLE_PRANOR_MESH_URL`	—	Pranor Mesh backend URL
`PRANOR_CONSOLE_AUTH_TOKEN`	—	Static admin auth token
`PRANOR_CONSOLE_THEME`	`dark`	Default theme (`dark`, `light`, `glassmorphism`)

YAML Config (`console.yaml`)

```
port: "8083"
gate_url: "http://pranor-gate:8080"
pulse_url: "http://pranor-pulse:9090"
vault_url: "http://pranor-vault:7070"
trace_url: "http://pranor-trace:8090"
mesh_url: "http://pranor-mesh:8089"
auth_token: "admin-secret-token"
theme: "dark"
```

CLI Flags

Flag	Default	Description
----- ----- -----		

```
| `--port` | `8083` | HTTP listen port |
```

API Reference

Base URL: `http://localhost:8083`

GET /api/v1/search?q={query}

Global resource search (■K).

Response (200):

```
{
  "results": [
    { "type": "service", "name": "orders-api", "module": "mesh", "url": "/mesh/services/orders-api" },
    { "type": "route", "name": "/api/orders", "module": "gate", "url": "/gate/routes/api-orders" }
  ],
  "took_ms": 3
}
```

GET /api/v1/topology/graph

Live service topology graph data.

Response (200):

```
{
  "nodes": [
    { "id": "orders-api", "type": "service", "status": "healthy" },
    { "id": "payments-api", "type": "service", "status": "degraded" }
  ],
  "edges": [
    { "from": "orders-api", "to": "payments-api", "rps": 120, "error_rate": 0.02, "p99_ms": 45 }
  ]
}
```

POST /api/v1/chaos/experiments

Create a chaos experiment.

Request:

```
{
  "name": "latency-spike-test",
  "target_service": "payments-api",
  "fault_type": "latency",
  "latency_ms": 500,
}
```



```
"percentage": 25,
"duration": "5m"
}
```

Response (201):

```
{
  "id": "exp-001",
  "status": "active",
  "blast_radius": ["orders-api", "checkout-api"],
  "expires_at": "2026-08-01T10:05:00Z"
}
```

POST /api/v1/incidents

Create an incident.

Request:

```
{
  "title": "High error rate on payments-api",
  "severity": "P2",
  "services": ["payments-api"],
  "description": "Error rate exceeded 5% SLO threshold"
}
```

Response (201):

```
{
  "id": "inc-001",
  "status": "open",
  "created_at": "2026-08-01T10:00:00Z"
}
```

GET /healthz

Liveness probe.

```
{"status": "UP", "service": "pranor-console", "version": "1.0.0"}
```

Security

Standalone Mode

Set `PRANOR\_CONSOLE\_AUTH\_TOKEN` for basic token authentication. Clients authenticate via:

```
Authorization: Bearer admin-secret-token
```

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem, Console integrates with Pranor Auth for RBAC-based access control:

- 1. **JWT Auth** — validates Bearer tokens against Pranor Auth
- 2. **Role-based dashboard access** — admins see all panels; operators see limited views
- 3. **Audit logging** — all management actions logged with user identity
- 4. **mTLS** — service-to-service communication encrypted

CORS

Console serves the SPA from a configurable origin. CORS headers allow the frontend to call backend APIs cross-origin.

Observability

Prometheus Metrics

Metric	Type	Description
----- ----- -----		
`pranor_console_active_sessions`	Gauge	Active WebSocket connections
`pranor_console_search_latency_ms`	Histogram	■K search response time
`pranor_console_chaos_experiments_active`	Gauge	Running chaos experiments
`pranor_console_incidents_open`	Gauge	Open incidents
`pranor_console_ws_messages_total`	Counter	WebSocket messages received

OpenTelemetry Tracing

Console emits spans for:

- `console.search` — global search queries
- `console.chaos.inject` — chaos experiment triggers
- `console.topology.refresh` — topology graph rebuilds

Logging

Structured JSON logs with fields: `level`, `timestamp`, `user\_id`, `action`, `module`, `latency\_ms`.

Enterprise Edition

Feature	OSS	EE
----- :--: :--:		
Unified dashboard UI	✓	✓
Global ■K search	✓	✓
Service topology graph	✓	✓
SLO burn rate dashboards	✓	✓

Incident management	✓	✓
Queue inspector (DLQ replay)	✓	✓
Chaos engineering panel	—	✓
eBPF flamegraph profiling	—	✓
AI Co-Pilot auto-remediation	—	✓
Environment provisioner	—	✓
Custom keyboard shortcuts & themes	—	✓
Multi-cluster federation view	—	✓

Operational Runbook

WebSocket connections dropping

1. Check `pranor\_console\_active\_sessions` gauge for sudden drops
2. Verify network stability between Console and downstream services
3. Check if rate limiting is affecting WebSocket upgrade requests
4. Review nginx/load balancer timeout settings for WebSocket connections
5. Ensure `Connection: Upgrade` headers are not being stripped

Global search returning stale results

1. Console indexes resources on startup and via WebSocket feeds
2. Force re-index by restarting Console or triggering topology refresh
3. Check connectivity to all configured service URLs
4. Verify Mesh is reporting accurate service catalog

Chaos experiment not propagating

1. Verify Pranol Mesh connectivity (`PRANOR\_CONSOLE\_PRANOR\_MESH\_URL`)
2. Check experiment status via `GET /api/v1/chaos/experiments/{id}`
3. Confirm target service is registered in Mesh service catalog
4. Review blast radius preview before re-triggering

Dashboard panels blank or loading

1. Check browser console for WebSocket connection errors
2. Verify backend service URLs are correct and accessible
3. Check auth token validity if using static token auth
4. Review CORS configuration if frontend is served from a different origin

Pranor Deploy — Deployment Orchestrator

Version: 0.1.0

Module Path: `github.com/vyuvaraj/pranor/deploy`

Default Port: 8085

License: AGPL-3.0 (OSS) / Enterprise License (EE with FinOps & DR Chaos Suite)

Overview

Pranor Deploy is the managed deployment platform and process orchestrator for the Pranor ecosystem. It provides PaaS-style service deployment, blue/green and canary strategies, per-branch preview environments, container isolation, ring-buffer log streaming, and deep integration with Pranor Gate for automatic routing registration.

- Pranor Deploy can run as:
- A **standalone binary** deploying processes with dynamic port allocation
 - An **integrated module** within the Pranor ecosystem with Gate route sync, OTel tracing, and container isolation

Key Features

Feature	Description
PaaS Deployment API	Deploy services on demand via REST with automatic route registration
Blue/Green Deployment	Atomic zero-downtime traffic cutover with instant rollback
Canary Deployment	Configurable traffic split with auto-rollback on error threshold
Preview Environments	Per-branch ephemeral environments with unique subdomains
Container Isolation	Docker/OCI container mode with resource limits and network isolation
Process Mode	Lightweight raw process execution for development
Ring-buffer Logs	Capture stdout/stderr with streaming log API
Gate Auto-Registration	Deployed services automatically get Pranor Gate routes
Health Gate	Deployments must pass health checks before traffic cutover
GitOps Webhooks	Trigger deployments from Git push events

Architecture

```
graph TD
    subgraph Trigger ["■ Deployment Control API"]
        GitOps["GitOps [\"GitOps Webhook and Branch Trigger\"]"]
        DeployAPI["DeployAPI [\"REST Deployment API\"]"]
    end
```

```
subgraph Orchestrator ["■ Core Deployment and FinOps Engine"]
  StrategyMgr["Deployment Strategy Manager"]
  FinOps["AI FinOps Cloud Cost Optimizer"]
  ChaosSuite["Automated DR Chaos Simulation Suite"]
  GateReg["Pranor Gate Route Auto-Registrar"]
end

subgraph IsolatedEnvs ["■ Environment Provisioning and Artifacts"]
  ContainerIso["OCI / Docker Container Isolation Engine"]
  PreviewMgr["Ephemeral Preview Environment Provisioner"]
  AirgapHub["Air-Gapped Private Artifact Registry"]
end

GitOps --> StrategyMgr
DeployAPI --> StrategyMgr
StrategyMgr --> FinOps
FinOps --> ChaosSuite
ChaosSuite --> GateReg
GateReg --> ContainerIso
GateReg --> PreviewMgr
ContainerIso -.-> AirgapHub
```

Canary Rollout & AI FinOps Promotion Sequence Flow

```
sequenceDiagram
    autonumber
    participant Developer as Developer / GitOps Pipeline
    participant Deploy as Pranor Deploy Engine
    participant FinOps as AI FinOps Optimizer
    participant Gate as Pranor Gate Ingress
    participant Pods as Canary / Blue-Green Pods

    Developer->>Deploy: POST /api/v1/deployments (Canary 10% Traffic)
    Deploy->>FinOps: Evaluate Node Allocation & Spot Instance Budgets
    FinOps-->>Deploy: Optimal Node Topology Approved
    Deploy->>Pods: Spin Up New Version (Canary Container Pods)
    Deploy->>Gate: Update Weighted Route (10% Canary, 90% Stable)
    Gate-->>Deploy: Traffic Splitting Active (Monitoring Latency/Errors)
    alt Error Rate < 0.01% & Health Check Passed
        Deploy->>Gate: Promote Canary to 100% Traffic (Cutover)
        Gate-->>Deploy: Full Production Cutover Complete
    else Latency Spike / Error Threshold Exceeded
        Deploy->>Gate: Immediate Auto-Rollback to 0% Canary
        Deploy-->>Developer: Deployment Aborted & Rollback Triggered
    end
end
```

Ecosystem Cross-Module Integration

Pranor Deploy automates release rollouts across all platform components:

- **Pranor Gate:** Enforces zero-downtime weighted canary traffic splits, blue/green cutovers, and preview subdomain routing.
- **Pranor Hub:** Pulls signed OCI container images, WebAssembly modules, and Helm charts for air-gapped deployments.
- **Pranor Trace:** Monitors real-time error rate budgets and latency burn rates during progressive canary rollouts.
- **Pranor Console:** Provides interactive multi-cluster deployment dashboards, 1-click rollback controls, and live container logs.

Installation & Deployment

Binary

```
cd pranor/deploy
go build -o pranor-deploy .
./pranor-deploy --port 8085
```

Docker

```
docker run -p 8085:8085 \
-v /var/run/docker.sock:/var/run/docker.sock \
ghcr.io/vyuvaraj/pranor-deploy:latest
```

With Pranor Gate Sync

```
./pranor-deploy --port 8085 --gateway http://pranor-gate:8080 --auth-token secret-token
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Deploy integrates automatically with Gate (route sync), Hub (artifact pull), Trace (OTel spans), and Console (dashboard visibility).

Configuration

Environment Variables

Variable	Default	Description
PRANOR_DEPLOY_PORT	8085	HTTP listener port
PRANOR_DEPLOY_PRANOR_GATE_URL	—	Pranor Gate URL for route registration
PRANOR_DEPLOY_OTEL_ENDPOINT	—	OpenTelemetry collector URL
PRANOR_DEPLOY_CONTAINER_RUNTIME	process	process (raw) or docker (OCI container)
PRANOR_DEPLOY_PREVIEW_DOMAIN	—	Base domain for preview environments
PRANOR_DEPLOY_PREVIEW_TTL	7d	Default preview environment TTL
PRANOR_DEPLOY_WORKDIR	./.deployments	Directory for deployment artifacts

YAML Config (deploy.yaml)

```
port: "8085"
gateway_url: "http://pranor-gate:8080"
auth_token: "secret-token"
container_runtime: "docker"
preview_domain: "preview.pranor.net"
preview_ttl: "7d"
workdir: "./.deployments"
otel_endpoint: "http://pranor-trace:8090"
```

CLI Flags

Flag	Default	Description
--port	8085	HTTP listen port

`--workdir`	`./.deployments`	Deployment working directory
`--gateway`	`http://localhost:8080`	Pranor Gate URL
`--auth-token`	`secret-token`	Auth token for Gateway registration
`--version`	—	Print version and exit

API Reference

Base URL: `http://localhost:8085`

POST /api/v1/deployments

Deploy a service.

Request:

```
{
  "service": "orders-api",
  "image": "ghcr.io/myorg/orders:v2.1.0",
  "strategy": "canary",
  "port": 3000,
  "canary_weight": 10,
  "auto_rollback_error_rate": 0.05
}
```

Response (201):

```
{
  "id": "dep-abc-123",
  "service": "orders-api",
  "strategy": "canary",
  "status": "deploying",
  "canary_weight": 10,
  "url": "http://orders-api:3000"
}
```

POST /api/v1/deployments/{id}/promote

Promote canary to higher traffic weight.

Request:

```
{
  "weight": 50
}
```

Response (200):

```
{
```



```
"status": "promoted",
"canary_weight": 50
}
```

POST /api/v1/deployments/{id}/rollback

Roll back to previous stable version.

Response (200):

```
{
  "status": "rolled_back",
  "restored_version": "v2.0.0"
}
```

POST /api/v1/deployments/{id}/cutover

Blue/Green: cut all traffic to new version.

Response (200):

```
{
  "status": "cutover_complete",
  "active_version": "green"
}
```

GET /api/v1/deployments/{id}/logs

Stream deployment logs from ring buffer.

Response (200):

```
{
  "lines": [
    "[2026-08-01 10:00:01] Server started on :3000",
    "[2026-08-01 10:00:02] Connected to database"
  ]
}
```

POST /api/v1/previews

Create a preview environment.

Request:

```
{
  "branch": "feature/new-checkout",
  "ttl": "7d"
}
```

```
}

Response (201):

{
  "id": "prev-001",
  "url": "https://feature-new-checkout.preview.pranor.net",
  "expires_at": "2026-08-08T10:00:00Z"
}

---

GET /healthz

Liveness probe.

{"status":"UP","service":"pranor-deploy","version":"0.1.0"}

---
```

Security

Standalone Mode

Configure `--auth-token` for Gateway registration authentication. Deploy endpoints are unauthenticated in standalone mode.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem:

- 1. **JWT Auth** — validates Bearer tokens against Pranor Auth
- 2. **RBAC enforcement** — deployment permissions per service/environment
- 3. **Audit trail** — every deploy, promote, rollback logged with operator identity
- 4. **Container isolation** — network namespaces prevent cross-deployment access
- 5. **OTel Tracing** — deployment lifecycle spans

Docker Socket Security

When using Docker runtime, Deploy requires access to the Docker socket. In production, use rootless Docker or configure appropriate socket permissions.

Observability

Prometheus Metrics

Metric	Type	Description
pranor_deploy_active_deployments	Gauge	Currently running deployments
pranor_deploy_rollbacks_total	Counter	Total rollback events
pranor_deploy_canary_promotions_total	Counter	Canary promotions

| `pranor\_deploy\_preview\_environments\_active` | Gauge | Active preview environments |

| `pranor\_deploy\_error\_rate` | Gauge | Current canary error rate |

OpenTelemetry Tracing

Every deployment generates OTel spans:

- `deploy.create` — deployment initialization
- `deploy.health\_check` — health gate validation
- `deploy.cutover` — traffic cutover event
- `deploy.rollback` — rollback trigger

Logging

Structured JSON logs with fields: `level`, `timestamp`, `trace\_id`, `deployment\_id`, `service`, `strategy`, `action`.

Enterprise Edition

Feature OSS EE
----- :--- :--
Direct deployment (process mode) ✓ ✓
Blue/green deployment ✓ ✓
Canary with auto-rollback ✓ ✓
Preview environments ✓ ✓
Docker container isolation ✓ ✓
Gate route auto-registration ✓ ✓
Ring-buffer log streaming ✓ ✓
AI FinOps cost optimizer — ✓
Automated DR chaos simulation — ✓
Air-gapped private artifact registry — ✓
Multi-cluster deployment federation — ✓
GitOps webhook triggers — ✓

Operational Runbook

Deployment stuck in "deploying" state

1. Check `/api/v1/deployments/{id}` for status details
2. Verify container image is pullable (check registry credentials)
3. Check health check endpoint of the deployed service
4. Review deployment logs via `/api/v1/deployments/{id}/logs`
5. If using Docker, check `docker ps` for container state

Canary auto-rollback triggered unexpectedly

1. Check ``pranor_deploy_error_rate`` metric during the canary window
2. Review the ``auto_rollback_error_rate`` threshold configuration
3. Verify Trace/Gate are reporting accurate error rates (not false positives)
4. Check if a downstream dependency caused the errors (not the canary itself)

Preview environments not cleaning up

1. Check ``PRANOR_DEPLOY_PREVIEW_TTL`` configuration
2. List active previews: ``GET /api/v1/previews``
3. Manually delete expired previews: ``DELETE /api/v1/previews/{id}``
4. Verify the cleanup background worker is running (check logs)

Gate route not registering after deploy

1. Verify ``PRANOR_DEPLOY_PRANOR_GATE_URL`` is configured and reachable
2. Check auth token matches between Deploy and Gate
3. Review Deploy logs for route registration errors
4. Manually verify route via Gate's route listing API

Pranor Flow — DAG Workflow & Saga Orchestrator

Version: 1.0.0

Module Path: `github.com/vyuvvaraj/pranor/flow`

Default Port: 8096

License: AGPL-3.0 (OSS) / Enterprise License (EE with BFT Raft & Visual Designer)

Overview

Pranor Flow is a stateful, DAG-based workflow orchestrator and Saga compensation engine for the Pranor ecosystem. It supports durable execution with checkpointing, WASM step functions, sub-workflow composition, per-execution tracing, a Dead Letter Workflow Queue with manual retry, and automatic reverse compensation on failure.

Pranor Flow can run as:

- A **standalone binary** with file-based checkpoint persistence
- An **integrated module** within the Pranor ecosystem with OTel tracing, Pranor Lock leader election, and Console visual designer

Key Features

Feature	Description
DAG Orchestration	Multi-step execution graphs with topological sort and parallel fan-out/fan-in
Saga Compensation	Automatic reverse compensation on failure — only completed steps are rolled back
Durable Execution	WAL checkpoint persistence; resume from last successful step after restarts
WASM Step Functions	Sandboxed WASI-compliant WebAssembly step execution (Rust, C, Go)
Sub-workflow Composition	Compose workflows from reusable sub-workflows with recursive nesting
Dead Letter Queue	Failed workflows moved to DLWQ with full context and manual retry
Step Output Propagation	Each step's output becomes the next step's input
Conditional Branching	Skip steps based on upstream output conditions
AI Cost Tracking	LLM token cost annotations on spans for AI workflow steps
Idempotent Replay	Skip already-completed steps on resume for safe replay

Architecture

```
graph TD
    subgraph API ["Workflow Control Interface"]
        Define["REST DAG Definition API"]
        Exec["Execution Manager API"]
    end
```

```
end

subgraph Core ["■ Core Distributed Saga Orchestrator"]
  Topo["Topological Sort and Dependency Evaluator"]
  HTTPExec["HTTP / REST Step Executor"]
  WASMExec["WASM Sandbox Step Executor"]
  SagaComp["Saga Reverse Compensation Handler"]
end

subgraph Storage ["■ Durable State Persistence"]
  WALStore["WAL Checkpoint Store"]
  DLWQ["Dead-Letter Workflow Queue"]
  BFTConsensus["BFT Raft State Consensus"]
end

Define --> Topo
Exec --> Topo
Topo --> HTTPExec
Topo --> WASMExec
HTTPExec --> WALStore
WASMExec --> WALStore
WALStore -->|On Failure| SagaComp
SagaComp -->|Max Retries Exhausted| DLWQ
WALStore --> BFTConsensus
```

Saga Execution & Compensation Sequence Flow

```
sequenceDiagram
    autonumber
    participant Client as Client Application
    participant Flow as Pranor Flow Orchestrator
    participant Inventory as Inventory Service
    participant Payment as Payment Gateway
    participant Shipping as Shipping Service
    participant WAL as WAL Checkpoint Store

    Client->>Flow: Execute Workflow (Order Checkout DAG)
    Flow->>Inventory: Step 1: POST /inventory/reserve
    Inventory-->>Flow: 200 OK (Reserved)
    Flow->>WAL: Checkpoint Step 1 Completed
    Flow->>Payment: Step 2: POST /payment/charge
    Payment-->>Flow: 500 Internal Error (Payment Failed)
    Flow->>WAL: Log Step 2 Execution Failure
    Note over Flow,Inventory: Trigger Reverse Compensation Rollback
    Flow->>Inventory: Compensate Step 1: POST /inventory/release
    Inventory-->>Flow: 200 OK (Inventory Unreserved)
    Flow->>WAL: Saga Rollback Completed
    Flow-->>Client: Workflow Execution Failed (Compensated)
```

Ecosystem Cross-Module Integration

Pranor Flow acts as the primary saga orchestrator across the Pranor ecosystem:

- **Pranor Pulse:** Dispatches asynchronous event triggers and listens to topic completions during long-running saga steps.
- **Pranor Trace:** Annotates every workflow execution and individual step with W3C traceparent headers, tracking LLM token costs and latency flamegraphs.
- **Pranor Lock:** Acquires distributed fencing token leases to ensure saga execution steps are evaluated by a single leader node during failover.
- **Pranor Console:** Provides a visual DAG designer, live workflow step progress tracking, and 1-click DLQ retry controls.

Installation & Deployment

Binary

```
cd pranor/flow
go build -o pranor-flow .
./pranor-flow --port 8096
```

Docker

```
docker run -p 8096:8096 \
-v flow-data:/data \
ghcr.io/vyuvaraj/pranor-flow:latest
```

With Checkpoint Persistence

```
./pranor-flow --port 8096 --checkpoint-dir /data/checkpoints
```

As Part of Pranol Ecosystem

When running under the Pranol platform, Flow integrates automatically with Lock (leader election), Trace (OTel spans), Console (visual designer), and Pulse (event triggers).

Configuration

Environment Variables

Variable	Default	Description
PRANOR_FLOW_PORT	8096	HTTP listener port
PRANOR_FLOW_CHECKPOINT_DIR	./checkpoints	Directory for workflow state checkpoint files
PRANOR_FLOW_OTEL_ENDPOINT	—	OpenTelemetry collector URL
PRANOR_FLOW_WASM_MODULES_DIR	./wasm	Directory for WASM step module files
PRANOR_FLOW_DLQ_MAX_SIZE	1000	Max workflows retained in DLQ

YAML Config (flow.yaml)

```
port: "8096"
checkpoint_dir: "/data/checkpoints"
otel_endpoint: "http://pranor-trace:8090"
wasm_modules_dir: "./wasm"
dlq_max_size: 1000
```

CLI Flags

Flag	Default	Description
--port	8096	HTTP listen port
--checkpoint-dir	./checkpoints	Checkpoint persistence directory

API Reference

Base URL: `http://localhost:8096`

POST /api/workflows/define

Define a new DAG workflow.

Request:

```
{
  "name": "order-fulfillment",
  "steps": [
    {
      "id": "reserve-inventory",
      "type": "http",
      "url": "http://inventory/reserve",
      "depends_on": [],
      "compensate_url": "http://inventory/release"
    },
    {
      "id": "charge-payment",
      "type": "http",
      "url": "http://payments/charge",
      "depends_on": ["reserve-inventory"],
      "compensate_url": "http://payments/refund"
    },
    {
      "id": "notify-customer",
      "type": "http",
      "url": "http://notifications/send",
      "depends_on": ["charge-payment"]
    }
  ]
}
```

Response (201):

```
{
  "id": "wf-def-001",
  "name": "order-fulfillment",
  "step_count": 3,
  "status": "registered"
}
```

POST /api/workflows/execute

Execute a workflow instance.

Request:

```
{
  "workflow": "order-fulfillment",
  "input": { "order_id": "ord-123", "amount": 99.99 }
}
```

Response (200):

```
{
  "instance_id": "wf-abc-001",
  "status": "running",
  "started_at": "2026-08-01T10:00:00Z"
}
```

GET /api/workflows/instances/{id}

Get execution status and step logs.

Response (200):

```
{
  "instance_id": "wf-abc-001",
  "workflow": "order-fulfillment",
  "status": "completed",
  "steps": [
    { "id": "reserve-inventory", "status": "success", "duration_ms": 45 },
    { "id": "charge-payment", "status": "success", "duration_ms": 230 },
    { "id": "notify-customer", "status": "success", "duration_ms": 12 }
  ]
}
```

POST /api/workflows/resume

Resume a workflow from its last checkpoint.

Request:

```
{
  "instance_id": "wf-abc-001"
}
```

Response (200):

```
{
  "status": "resumed",
  "resumed_from_step": "charge-payment"
}
```

GET /api/workflows/dlq

Browse Dead Letter Workflow Queue.

Response (200):

```
{
  "workflows": [
    {
      "instance_id": "wf-xyz-002",
      "workflow": "order-fulfillment",
      "failed_step": "charge-payment",
      "error": "connection timeout",
      "failed_at": "2026-08-01T09:30:00Z"
    }
  ]
}
```

POST /api/workflows/dlq/{id}/retry

Retry a DLQ workflow.

Response (200):

```
{
  "status": "retrying",
  "instance_id": "wf-xyz-002"
}
```

GET /healthz

Liveness probe.

```
{"status": "UP", "service": "pranor-flow", "version": "1.0.0"}
```

Security

Standalone Mode

In standalone mode, Flow runs without authentication. Workflow callbacks are dispatched without auth headers.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem:

1. **OTel Tracing** — every request and step execution gets a span
2. **Rate Limiting** — per-client request throttling

- 3. **JWT Auth** — validates Bearer tokens against Prnor Auth
- 4. **Tenant Isolation** — workflows scoped per tenant namespace
- 5. **Callback Auth** — configurable bearer token forwarded to step URLs

WASM Sandbox Security

WASM steps execute in a sandboxed environment:

- No filesystem access beyond stdin/stdout
- Per-step execution timeout prevents runaway processes
- Memory limits enforced per WASM module

Observability

Prometheus Metrics

Metric	Type	Description
----- ----- -----		
`prnor_flow_workflows_active`	Gauge	Currently executing workflows
`prnor_flow_steps_total`	Counter	Total step executions (labeled by status)
`prnor_flow_step_duration_ms`	Histogram	Step execution duration
`prnor_flow_compensations_total`	Counter	Saga compensation events
`prnor_flow_dlq_depth`	Gauge	Dead letter queue depth
`prnor_flow_checkpoints_total`	Counter	Checkpoint writes

OpenTelemetry Tracing

Every workflow and step generates OTel spans:

- `flow.workflow.execute` — root workflow span
- `flow.step.http` — HTTP step execution
- `flow.step.wasm` — WASM step execution
- `flow.saga.compensate` — compensation rollback
- `flow.dlq.enqueue` — DLQ enqueue event

Logging

Structured JSON logs with fields: `level`, `timestamp`, `trace\_id`, `instance\_id`, `step\_id`, `status`, `duration\_ms`.

Enterprise Edition

Feature	OSS	EE
----- :---: :---:		
DAG workflow orchestration	✓	✓
Saga compensation	✓	✓

| Checkpoint persistence | ✓ | ✓ |

| WASM step functions | ✓ | ✓ |

| Sub-workflow composition | ✓ | ✓ |

| Dead letter queue | ✓ | ✓ |

| OTel tracing | ✓ | ✓ |

| BFT Raft state consensus | — | ✓ |

| Visual DAG designer UI | — | ✓ |

| AI cost tracking per step | — | ✓ |

| Multi-cluster workflow federation | — | ✓ |

| Event-driven triggers (Pranor Pulse) | — | ✓ |

Operational Runbook

Workflow stuck in "running" state

1. Check ``/api/workflows/instances/{id}`` for step-level status
2. Identify which step is blocking — check its callback URL health
3. If step timed out, the workflow may be waiting for checkpoint write
4. Resume from checkpoint: ``POST /api/workflows/resume``
5. If stuck permanently, check disk space for checkpoint directory

Saga compensation failing

1. Check `compensate_url` endpoints are reachable and returning 200
2. Review logs for compensation step errors
3. Compensations are best-effort — if they fail, manual intervention required
4. Check ``pranor_flow_compensations_total`` metric for failure patterns

DLQ growing unbounded

1. Monitor ``pranor_flow_dlq_depth`` gauge
2. Review failed workflows in DLQ for common error patterns
3. Fix root cause (downstream service, timeout, etc.)
4. Retry workflows: ``POST /api/workflows/dlq/{id}/retry``
5. Adjust ``PRANOR_FLOW_DLQ_MAX_SIZE`` to prevent memory issues

WASM steps timing out

1. Check ``PRANOR_FLOW_WASM_MODULES_DIR`` for module availability
2. Review step timeout configuration in workflow definition
3. Check WASM module for infinite loops or excessive memory allocation
4. Monitor ``pranor_flow_step_duration_ms`` histogram for WASM steps

Pranor Gate — API Gateway & Ingress Router

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/gate`

Default Port: 8080

License: AGPL-3.0 (OSS) / Enterprise License (EE with eBPF, GraphQL Federation, Geo-IP Steering)

Overview

Pranor Gate is a high-performance API gateway and reverse proxy that routes, secures, and transforms traffic between clients and upstream services. It features WASM-based plugin extensibility, AI-aware traffic management (prompt guard, semantic caching, PII redaction, token billing), weighted canary/blue-green deployments with automatic promotion, circuit breaking, SSE passthrough, WebSocket proxying, and a declarative per-route configuration model.

Pranor Gate can run as:

- A **standalone binary** with local JSON configuration
- An **integrated module** within the Pranor ecosystem with S3-based dynamic config, JWT auth, OTel tracing, and Console visibility
- An **edge proxy** with Let's Encrypt auto-TLS or dynamic certificate fetching from Pranor Secret

Table of Contents

- [Key Features](#key-features)
- [Architecture](#architecture)
- [Installation & Deployment](#installation--deployment)
- [Configuration](#configuration)
- [API Reference](#api-reference)
- [Routing & Traffic Management](#routing--traffic-management)
- [WASM Plugin System](#wasm-plugin-system)
- [AI Guard & LLM Routing](#ai-guard--llm-routing)
- [Security](#security)
- [Observability](#observability)
- [Client Libraries & CLI](#client-libraries--cli)
- [Enterprise Edition](#enterprise-edition)
- [Operational Runbook](#operational-runbook)

Key Features

Feature	Description
WASM Plugin Middleware	Upload and hot-register WebAssembly request/response transform modules per route at runtime.

| **Rate Limiting** | Per-IP, per-route RPM limits with optional Redis-backed distributed enforcement. |

| **Circuit Breaker** | Automatic circuit breaking on upstream failure thresholds with half-open recovery. |

| **AI Prompt Guard** | Inspects and sanitizes inputs for prompt injection attacks on AI/LLM routes. |

| **Semantic Cache** | Embedding-based response cache for AI endpoints — returns cached responses for semantically similar prompts. |

| **PII Redaction** | Automatic detection and masking of personally identifiable information in AI payloads. |

| **Canary / Blue-Green** | Weighted traffic splitting with automated canary promotion and error-rate rollback. |

| **SSE Passthrough** | Transparent proxying of Server-Sent Events streams without buffering. |

| **WebSocket Proxy** | Full-duplex WebSocket proxying with connection tracking. |

| **mTLS to Upstreams** | Per-route mutual TLS client certificates for service-to-service authentication. |

| **Let's Encrypt Auto-TLS** | Zero-config HTTPS with automatic ACME certificate provisioning. |

| **Response Caching** | Configurable per-route TTL response cache for GET requests. |

| **Backpressure Control** | Concurrent request limiting with queue overflow protection. |

| **OpenAPI Validation** | Request payload validation against OpenAPI 3.0 spec per route. |

| **IP Allowlist/Blocklist** | Per-route network ACLs via CIDR ranges. |

| **Structured Access Logs** | JSONL access logging with request/response metadata. |

| **GitOps Config Sync** | Webhook-triggered git pull + config reload for GitOps workflows. |

| **Dynamic Policy Engine** | ServPolicy integration for fine-grained authorization rules compiled to WASM. |

| **AI Token Billing** | Per-route and per-tenant LLM token usage tracking with budget enforcement. |

| **Traffic Replay** | Record traffic to JSONL and replay against WASM middlewares or candidate backends. |

Architecture

```
graph TD
  subgraph Edge ["Global Ingress Layer"]
    DNS["Geo-IP Anycast DNS"]
    XDP["eBPF XDP Packet Filter"]
  end

  subgraph Security ["Zero-Trust Security and WASM Engine"]
    TLS["PCIe Hardware TLS Offload"]
    WASM["WASM Security Sandbox"]
    PromptGuard["AI Prompt Injection Guard"]
  end

  subgraph Core ["Proxy Router and Rate Limiter"]
    CRDT["Global CRDT Rate Limiter"]
    Router["Dynamic Reverse Proxy"]
  end
```

```

subgraph Upstream ["Upstream Microservices"]
  AIModel["LLM / Model Service"]
  Microservice["gRPC / REST Microservice"]
end

DNS --> XDP
XDP --> TLS
TLS --> WASM
WASM --> PromptGuard
PromptGuard --> CRDT
CRDT --> Router
Router -->|mTLS| AIModel
Router -->|mTLS| Microservice

```

Request Processing Sequence & WASM Execution Flow

```

sequenceDiagram
    autonumber
    participant Client as Client Application
    participant Gate as Pranor Gate Ingress
    participant Auth as Pranor Auth / JWT Validator
    participant WASM as WASM Plugin Sandbox
    participant AI as AI Prompt Guard
    participant Service as Upstream Microservice

    Client->>Gate: HTTP Request / POST /v1/ai/prompt
    Gate->>Auth: Validate JWT / SPIFFE SVID Token
    Auth-->>Gate: Token Validated (Claims + Tenant Context)
    Gate->>WASM: Execute Request Transformer (WASM)
    WASM-->>Gate: Transformed Headers & Body
    Gate->>AI: Evaluate Prompt Injection & PII Redaction
    AI-->>Gate: Sanitized Prompt & Risk Score (Passed)
    Gate->>Service: Forward Request (mTLS + Retrying Transport)
    Service-->>Gate: Response Stream (200 OK)
    Gate->>WASM: Execute Response Transformer (WASM)
    WASM-->>Gate: Final Formatted Payload
    Gate-->>Client: Streamed HTTP Response + X-Token-Cost

```

Ecosystem Cross-Module Integration

Pranor Gate acts as the front door for the entire Pranor platform, seamlessly interfacing with core infrastructure services:

- **Pranor Auth:** Automatically verifies incoming JWT signatures, SAML claims, and SPIFFE/SPIRE x509 workload identities ('PRANOR_JWT_SECRET').

- **Pranor Secret:** Dynamically fetches and auto-rotates TLS server certificates and client mTLS credentials without restarting the proxy.
- **Pranor Trace:** Generates W3C-compliant `traceparent` OpenTelemetry headers, emitting distributed trace spans for every proxied request.
- **Pranor Console:** Streams real-time throughput, p99 latency histograms, and active WASM plugin health metrics directly to the control plane dashboard.
- **Pranor Vault:** Pulls dynamic S3-backed JSON route configurations and uploads recorded JSONL traffic replay logs.

Installation & Deployment

Binary

```
cd pranor/gate
go build -o pranor-gate .
./pranor-gate
```

Docker

```
docker run -p 8080:8080 ghcr.io/vyuvaraj/pranor-gate:latest
```

Docker Compose

```
services:
  gate:
    image: ghcr.io/vyuvaraj/pranor-gate:latest
    ports:
      - "8080:8080"
    volumes:
      - ./config.json:/app/config.json
    environment:
      - PRANOR_JWT_SECRET=your-secret
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Gate integrates automatically with Auth (JWT/mTLS), Secret (dynamic certificates), Trace (OTel spans), and Console (dashboard visibility). Configuration can be pulled from an S3-compatible store for centralized management.

Configuration

JSON Config (`config.json`)

```
{
  "addr": ":8080",
  "auth_token": "gateway-secret-token",
  "tls_cert": "",
  "tls_key": "",
  "routes": [
    {
      "prefix": "/api/v1/services",
      "target": "http://127.0.0.1:8081",
      "middleware": "uppercase",
      "rate_limit_rpm": 120,
      "cache_ttl_seconds": 60,
      "access_log": true
    },
    {
      "prefix": "/ai/v1",
      "target": "http://127.0.0.1:11434",
      "enable_semantic_cache": true,
      "enable_prompt_guard": true,
      "semantic_token_limit_per_min": 10000
    }
  ]
}
```

Environment Variables

Variable	Default	Description
----- ----- -----		
`PRANOR_JWT_SECRET`	—	JWT signing key for token-based auth
`PRANOR_AUTO_TLS`	`false`	Enable Let's Encrypt auto-TLS
`PRANOR_AUTO_TLS_DOMAIN`	—	Domain for ACME certificate
`PRANOR_CONFIG_S3_BUCKET`	—	S3 bucket for remote config
`PRANOR_DISCOVERY`	—	Service discovery endpoint
`PRANOR_SECRET_URL`	—	Pranor Secret service URL for dynamic certs
`PRANOR_SECRET_API_KEY`	—	API key for Pranor Secret
`PRANOR_SECRET_TENANT_ID`	`default`	Tenant ID for secret lookup
`PRANOR_GATE_LIMITS_REDIS_URL`	—	Redis URL for distributed rate limiting
`PRANOR_REGISTRY`	`https://registry.pranor.org`	WASM middleware registry URL
`PRANOR_CLUSTER`	`default`	Cluster identifier for tenant policies

| `PRANOR\_REGION` | `us-east` | Region identifier for tenant policies |
 | `PRANOR\_OTLP\_ENDPOINT` | — | OpenTelemetry collector URL |

CLI Flags

| Flag | Default | Description |

|-----|-----|-----|

| `--config` | `config.json` | Path to configuration file |

CLI Subcommands

| Command | Description |

|-----|-----|

| `pranor-gate` | Start the gateway server |

| `pranor-gate dashboard` | Launch terminal TUI traffic dashboard |

| `pranor-gate replay --log FILE --middleware FILE.wasm` | Replay recorded traffic through WASM |

| `pranor-gate replay --shadow --log FILE --target URL` | Shadow diff replay against candidate backend |

| `pranor-gate install` | Install WASM middleware from registry |

| `pranor-gate policy compile -o` | Compile policy DSL to WASM |

API Reference

Base URL: `http://localhost:8080`

API Version: `/api/v1/` (recommended) or `/api/` (legacy)

GET /healthz

Liveness probe.

```
{"status":"UP","service":"pranor","version":"1.0.0"}
```

GET /readyz

Readiness probe. Same format as healthz.

GET /api/v1/routes

List all configured routes.

Response (200):

```
[
  {
    "prefix": "/api/v1/services",
    "target": "http://127.0.0.1:8081",
    "middleware": "uppercase",
    "rate_limit_rpm": 120
  }
]
```

```
}  
]
```

POST /api/v1/routes

Register or update a route dynamically.

Request:

```
{  
  "prefix": "/api/v2/users",  
  "target": "http://users-service:8080",  
  "rate_limit_rpm": 200,  
  "cache_ttl_seconds": 30,  
  "ip_allowlist": ["10.0.0.0/8"]  
}
```

Response (200):

```
Route registered successfully
```

DELETE /api/v1/routes?prefix=/api/v2/users

Remove a route.

Response (200):

```
Route deleted successfully
```

POST /api/v1/admin/middleware/{name}

Register a WASM middleware plugin at runtime.

Request: Raw `.wasm` binary as request body.

Response (200):

```
WASM Middleware auth-check compiled and registered
```

GET /api/v1/admin/connections

List active backend connections.

Response (200):

```
{  
  "http://127.0.0.1:8081": 5,  
  "http://127.0.0.1:8082": 2  
}
```

DELETE /api/v1/admin/cache?prefix=/api/v1/data

Invalidate response cache entries.

Response (200):

```
{
  "status": "success",
  "entries_invalidated": 12,
  "prefix": "/api/v1/data"
}
```

POST /api/v1/admin/policy/reload

Hot-reload the dynamic IAM policy schema.

Request (optional body): Policy schema JSON.

Response (200):

```
{"status": "success", "message": "Policy schema updated"}
```

POST /api/v1/admin/policy/revoke

Revoke all sessions for a user.

Request:

```
{"username": "compromised-user"}
```

Response (200):

```
{"status": "success", "message": "Session revoked for user compromised-user"}
```

GET /api/v1/admin/ai-billing

Retrieve AI token usage and cost metrics.

Response (200):

```
{
  "total_tokens": 1523400,
  "total_cost_usd": 4.57,
  "per_tenant": {
    "tenant-a": {"tokens": 800000, "cost_usd": 2.40}
  }
}
```

POST /api/v1/admin/ai-billing

Set per-tenant AI budget limits.

Request:

```
{
  "tenant_id": "tenant-a",
  "max_cost_per_day_usd": 10.00,
  "max_tokens_per_minute": 50000
}
```

GET /api/v1/admin/ai-cost-attribution

Per-route AI token and cost attribution dashboard.

Response (200):

```
{
  "routes": [
    {
      "prefix": "/ai/v1",
      "total_tokens": 500000,
      "total_cost_usd": 1.50,
      "estimated_savings": 0.30
    }
  ],
  "summary": {
    "total_cost_usd": 4.57,
    "total_tokens": 1523400,
    "estimated_savings": 0.91,
    "savings_percent": 16.6
  }
}
```

GET /api/v1/admin/metrics/ws

WebSocket endpoint streaming real-time gateway metrics (RPS, error rate, active connections).

POST /api/v1/admin/console/sync

Synchronize full route configuration from Pranor Console.

Request:

```
{"routes": [...]}
```

GET /api/v1/admin/console/sync

Get current gateway state snapshot (routes, connections, metrics).

POST /api/v1/gitops/webhook

Trigger a git pull + config reload for GitOps-managed configuration.

Response (200):

```
{
  "status": "success",
  "message": "GitOps config sync completed successfully",
  "git_output": "Already up to date."
}
```

POST /api/v1/routes/register

Register a route via the compiler connector (for Pranor Lang integration).

GET /api/docs

Embedded interactive API documentation page.

GET /api/docs/openapi.json

Auto-generated OpenAPI specification from current routes.

Routing & Traffic Management

Prefix-Based Matching

Routes are matched by longest-prefix on the request URL path. The first matching route wins.

Weighted Canary / Blue-Green Deployments

Distribute traffic between stable and canary targets by weight:

```
{
  "prefix": "/api/v1/orders",
  "targets_weighted": [
    {"url": "http://orders-v1:8080", "weight": 90},
    {"url": "http://orders-v2:8080", "weight": 10}
  ],
  "canary_auto_promote": true,
  "canary_promote_step": 10,
  "canary_promote_sec": 60,
  "canary_max_error_rate": 0.01
}
```

The canary engine automatically:

1. Increments canary weight by `canary\_promote\_step` every `canary\_promote\_sec` seconds

2. Monitors error rate on the canary target
3. Rolls back to 100% stable if error rate exceeds `canary\_max\_error\_rate`
4. Disables auto-promotion once canary reaches 100%

Load Balancing

Multiple targets support round-robin and least-connections strategies:

```
{
  "prefix": "/api/v1/users",
  "targets": ["http://users-1:8080", "http://users-2:8080", "http://users-3:8080"],
  "load_balancer": "least_conn"
}
```

Circuit Breaker

Automatically opens when upstream error rate exceeds threshold, preventing cascade failures. Half-open state probes recovery.

Backpressure Control

Per-route concurrency limiting with queue overflow:

```
{
  "max_concurrent_requests": 100,
  "max_queue_size": 500,
  "queue_timeout_ms": 5000
}
```

Returns `503 Service Unavailable` when queue is full, `504 Gateway Timeout` on queue timeout.

Response Caching

```
{
  "cache_ttl_seconds": 60,
  "cache_methods": ["GET"]
}
```

WASM Plugin System

Architecture

WASM middlewares are compiled via wazero (pure-Go WebAssembly runtime, no CGO). Plugins receive the request, can transform headers/body, and return modified content.

Registering a Plugin

```
# From registry
pranor-gate install jwt-auth

# Upload directly
curl -X POST http://localhost:8080/api/v1/admin/middleware/my-filter \
-H "Authorization: Bearer gateway-secret-token" \
--data-binary @my-filter.wasm
```

Per-Route Assignment

```
{
  "prefix": "/api/v1/data",
  "middleware": "my-filter",
  "response_middleware": "response-transform"
}
```

WASM A/B Testing

Split traffic between different WASM middleware versions:

```
{
  "wasm_split": {
    "targets": [
      {"middleware_name": "filter-v1", "weight": 80},
      {"middleware_name": "filter-v2", "weight": 20}
    ]
  }
}
```

Policy DSL Compilation

Write human-readable policies and compile to WASM:

```
# auth.policy
allow GET /api/public/*
deny POST /api/admin/* if header.role == "viewer"
allow * * if header.x-internal == "true"
```

```
pranor-gate policy compile auth.policy -o auth.wasm
```

AI Guard & LLM Routing

Prompt Guard

Detects and blocks prompt injection attempts on AI-routed traffic:

```
{"prefix": "/ai/v1", "prompt_guard": true}
```

PII Redaction

Masks sensitive data (emails, SSN, credit cards) before forwarding to LLM backends:

```
{"prefix": "/ai/v1", "pii_redact": true}
```

Semantic Cache

Caches LLM responses and returns cached versions for semantically similar prompts (cosine similarity > 0.85):

```
{"prefix": "/ai/v1", "semantic_cache": true}
```

LLM Routing with Fallback

Route to a primary model with automatic fallback on low confidence:

```
{
  "llm_routing": {
    "primary": {"url": "http://ollama:11434", "model": "llama3"},
    "fallback": {"url": "https://api.openai.com", "model": "gpt-4"},
    "confidence_header": "X-Confidence",
    "min_confidence": 0.7
  }
}
```

Semantic Rate Limiting

Token-based rate limiting for LLM routes (tokens-per-minute rather than requests-per-minute):

```
{"semantic_rate_limit": true, "semantic_token_limit_per_min": 10000}
```

Prompt A/B Testing

Route different prompt templates to measure response quality.

Security

Bearer Token Auth

Set `auth\_token` in config. All non-health endpoints require:

```
Authorization: Bearer gateway-secret-token
```

JWT Authentication

When `PRANOR\_JWT\_SECRET` is set, validates JWT Bearer tokens. Supports policy versioning — stale tokens get `X-Token-Refresh: true` header.

Dynamic Secret Fetching

Gate can fetch TLS certificates and JWT secrets dynamically from Pranor Secret at startup.

mTLS to Upstreams

Per-route client certificate for backend authentication:

```
{
  "client_cert_path": "/certs/client.crt",
  "client_key_path": "/certs/client.key",
  "root_ca_path": "/certs/backend-ca.crt"
}
```

Multi-Tenant API Keys

Per-key rate limits, route restrictions, and tenant isolation:

```
{"require_api_key": true, "allowed_tenants": ["tenant-a", "tenant-b"]}
```

IP Allowlist / Blocklist

```
{
  "ip_allowlist": ["10.0.0.0/8", "192.168.1.0/24"],
  "ip_blocklist": ["1.2.3.4"]
}
```

Request Body Size Limits

Per-route body size enforcement (default 10MB):

```
{"max_body_size": 5242880}
```

Session Revocation

Instant session revocation via admin API without waiting for token expiry.

Dynamic IAM Policy (ServPolicy)

Upload OPA-style policy schemas that Gate evaluates inline per request.

Observability

Metrics

Metric	Type	Description
total_requests	Counter	Total proxied requests
total_errors	Counter	Total upstream errors
request_rate	Gauge	Requests/second (1s window)

| `error\_rate` | Gauge | Errors/second (1s window) |
| `active\_connections` | Gauge | Per-target active connections |

WebSocket Live Metrics

Connect to `/api/v1/admin/metrics/ws` for 1-second streaming metrics updates.

Access Logging

Structured JSONL access logs per route:

```
{  
  "timestamp": "2026-01-15T10:00:00Z",  
  "method": "GET",  
  "path": "/api/v1/users/123",  
  "status": 200,  
  "latency_ms": 42,  
  "client_ip": "10.0.1.5",  
  "upstream": "http://users:8080"  
}
```

OpenTelemetry Tracing

Every proxied request gets an OTel span with method, route, status code, and upstream latency.

Terminal Dashboard

```
pranor-gate dashboard
```

Live TUI showing real-time RPS, P99 latency, circuit breaker state, cache hit rate.

```
---
```

Client Libraries & CLI

cURL

```
# Register a route
curl -X POST http://localhost:8080/api/v1/routes \
-H "Authorization: Bearer gateway-secret-token" \
-H "Content-Type: application/json" \
-d '{"prefix":"/api/v2/users","target":"http://users:8080","rate_limit_rpm":100}'

# Upload WASM middleware
curl -X POST http://localhost:8080/api/v1/admin/middleware/auth-check \
-H "Authorization: Bearer gateway-secret-token" \
--data-binary @auth-check.wasm

# Invalidate cache
curl -X DELETE "http://localhost:8080/api/v1/admin/cache?prefix=/api/v1/data" \
-H "Authorization: Bearer gateway-secret-token"
```

Pranor CLI

```
pranor gate routes list
pranor gate routes add --prefix /api/v2 --target http://backend:8080 --rate-limit 100
pranor gate middleware install jwt-auth
pranor gate dashboard
pranor gate replay --log traffic.jsonl --middleware filter.wasm
```

Enterprise Edition

| Feature | OSS | EE |

|-----|:---|:--|

| WASM plugin middleware | ✓ | ✓ |

| Rate limiting (local) | ✓ | ✓ |

| Rate limiting (Redis distributed) | ✓ | ✓ |

| Circuit breaker | ✓ | ✓ |

| Canary / Blue-Green deployments | ✓ | ✓ |

| AI Prompt Guard & PII Redaction | ✓ | ✓ |

| Semantic cache | ✓ | ✓ |

| SSE passthrough & WebSocket proxy | ✓ | ✓ |

| mTLS to upstreams | ✓ | ✓ |

| Let's Encrypt Auto-TLS | ✓ | ✓ |

| GitOps config sync | ✓ | ✓ |

| Traffic replay engine | ✓ | ✓ |

AI token billing & budgets	✓	✓
Kernel eBPF XDP DDoS bypass (100Gbps)	—	✓
Geo-IP latency anycast steering	—	✓
GraphQL schema stitching & federation	—	✓
SSL offloading (hardware acceleration)	—	✓
AI self-defending WAF	—	✓
Multi-cluster enterprise control plane	—	✓

Operational Runbook

Route not matching / 502 Bad Gateway

1. Check ``/api/v1/routes`` for the configured routes
2. Verify the request path has the correct prefix
3. Ensure the upstream target is reachable from the gateway
4. Check circuit breaker state via metrics

High latency on specific route

1. Check ``/api/v1/admin/connections`` for connection count
2. Review backpressure settings — ``max_concurrent_requests`` may be too low
3. Check if circuit breaker is in half-open state (probing slowly)
4. Look at upstream health via WebSocket metrics stream

Rate limiting kicking in unexpectedly

1. Verify ``rate_limit_rpm`` is set correctly on the route
2. Check if Redis-based distributed limiting is configured — all instances share state
3. Per-API-key limits may be more restrictive than route limits
4. Review semantic token rate limits for AI routes

WASM middleware failing

1. Check gateway logs for WASM compilation errors
2. Use ``pranor-gate replay --log traffic.jsonl --middleware broken.wasm`` to test offline
3. Verify WASM module exports the correct ABI functions
4. Check if the middleware registry URL is reachable

Canary deployment not promoting

1. Check error rate on canary target — exceeding ``canary_max_error_rate`` causes rollback
2. Verify ``canary_auto_promote`` is ``true``
3. Ensure at least 3 requests have hit the canary (minimum sample for error rate calculation)
4. Check ``canary_promote_sec`` interval — promotion may not have triggered yet

TLS certificate issues

1. If using auto-TLS, ensure port 80 is accessible for HTTP challenge
2. For Pranor Secret integration, verify `PRANOR\_SECRET\_URL` connectivity
3. Check certificate paths in config for file-based TLS
4. Review gateway startup logs for certificate loading errors

Versioning & Compatibility

- API is versioned at `/api/v1/`
- Legacy `/api/` paths continue to work (internally mapped to v1)
- Configuration format is backward-compatible across minor versions
- WASM ABI is stable — plugins compiled for v1.0 work on all v1.x releases

Pranor Hub — Package Registry & Artifact Store

Version: 1.0.0

Module Path: `github.com/vyuvvaraj/pranor/hub`

Default Port: 8088

License: AGPL-3.0 (OSS) / Enterprise License (EE with OCI backend & Air-gapped Mirror)

Overview

Pranor Hub is the lightweight, S3-backed package registry and artifact store for the Pranor ecosystem. It provides package publishing, semver resolution, dependency graph analysis, Cosign supply-chain verification, JWT-authenticated publishing, and a built-in landing dashboard for browsing packages.

- Pranor Hub can run as:
- A **standalone binary** with S3-compatible storage backend
 - An **integrated module** within the Pranor ecosystem with Pranor Vault storage, Auth RBAC, and OCI container image support

Key Features

Feature	Description
S3 / Pranor Vault Backend	Packages stored as tarballs in S3-compatible storage
Semver Resolution	Semantic versioning with dependency tree resolution
Cosign Verification	Sigstore supply-chain signature verification on publish
JWT Authorization	Token-based authentication for package publishing
Dependency Graph	Resolve and visualize full dependency trees
Package Search	Full-text search across package names and metadata
Version History	Browse all published versions per package
Landing Dashboard	Built-in web UI displaying packages, sizes, and versions
pranor.toml Manifests	Standard manifest format for package metadata
OCI Backend	Store and distribute packages as OCI artifacts

Architecture

```
graph TD
    subgraph PackageClients ["CLI and Package Registry API"]
        CLI["pranor-cli Package Manager"]
        PublishAPI["REST Package Publishing API"]
        RegistryDash["Package Registry Landing UI"]
    end
```



```

end

subgraph RegistryCore ["■ Package Resolver and Security Engine"]
ManifestParser["pranor.toml Manifest Inspector"]
DepResolver["Dependency Graph Resolver Engine"]
CosignVerifier["Cosign / Sigstore Supply-Chain Verification"]
JWTAuth["JWT Signature and Publisher Verifier"]
end

subgraph StorageLayer ["■ S3 and Vault Package Store"]
VaultStore["Pranor Vault S3 Bucket Tarball Storage"]
ColdArchive["Air-Gapped Private Package Mirror"]
end

CLI --> ManifestParser
PublishAPI --> ManifestParser
RegistryDash --> ManifestParser
ManifestParser --> DepResolver
DepResolver --> CosignVerifier
CosignVerifier --> JWTAuth
JWTAuth --> VaultStore
VaultStore --> ColdArchive

```

Package Publish & Dependency Resolution Sequence Flow

```

sequenceDiagram
autonumber

participant Developer as Module Developer
participant Hub as Pranor Hub Registry
participant Auth as Pranor Auth / Cosign
participant Resolver as Dependency Tree Resolver
participant Vault as Pranor Vault S3

Developer->>Hub: POST /publish (Package Tarball + pranor.toml)
Hub->>Auth: Verify JWT Token & Cosign Supply-Chain Signature
Auth-->>Hub: Publisher Identity & Cryptographic Proof Verified
Hub->>Resolver: Parse Manifest Dependencies & Resolve DAG Tree
Resolver-->>Hub: Dependency Graph Validated (No Conflicts)
Hub->>Vault: Store Package Tarball (packages/foo-1.2.0.tar.gz)
Vault-->>Hub: S3 Blob Persisted
Hub-->>Developer: Package Published Successfully

```

Ecosystem Cross-Module Integration

Pranor Hub acts as the official artifact and WebAssembly module registry for the Pranor platform:

- **Pranor Deploy:** Pulls signed WebAssembly security modules, OCI container images, and deployment manifests during canary rollouts.
- **Pranor Gate:** Downloads compiled WASM dynamic policy plugins published to Hub repositories.
- **Pranor Vault:** Serves as the high-availability S3 storage backend for all published package tarballs and signatures.
- **Pranor Auth:** Enforces RBAC permissions for organization-scoped package publishing and team access control.

Installation & Deployment

Binary

```
cd pranor/hub
go build -o pranor-hub .
./pranor-hub --addr :8088 --s3-endpoint http://localhost:9000
```

Docker

```
docker run -p 8088:8088 ghcr.io/vyuvaraj/pranor-hub:latest
```

With Pranor Vault Storage

```
./pranor-hub --addr :8088 \
--s3-endpoint http://pranor-vault:7070 \
--s3-access-key admin \
--s3-secret-key admin123
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Hub integrates automatically with Vault (storage), Auth (RBAC), Deploy (artifact pull), and Gate (WASM module distribution).

Configuration

Environment Variables

Variable	Default	Description
-----	-----	-----
`PORT`	`8088`	HTTP server port
`PRANOR_STORE_ENDPOINT`	`http://localhost:9000`	Pranor Vault or external S3 URL
`PRANOR_STORE_ACCESS_KEY`	`admin`	S3 access key
`PRANOR_STORE_SECRET_KEY`	`admin123`	S3 secret key
`PRANOR_JWT_SECRET`	—	JWT secret for publish authentication (disabled if unset)
`PRANOR_HUB_OTEL_ENDPOINT`	—	OpenTelemetry collector URL

YAML Config (`hub.yaml`)

```
port: "8088"
store_endpoint: "http://pranor-vault:7070"
store_access_key: "admin"
store_secret_key: "admin123"
jwt_secret: "my-signing-secret"
otel_endpoint: "http://pranor-trace:8090"
```

CLI Flags

Flag	Default	Description
----	-----	-----
<code>--addr</code>	<code>:8088</code>	HTTP listen address
<code>--s3-endpoint</code>	<code>http://localhost:9000</code>	S3-compatible storage endpoint

API Reference

Base URL: `http://localhost:8088`

API Version: `/api/v1/` (recommended) or `/api/` (legacy)

POST `/api/v1/publish`

Publish a package tarball.

- Headers:**
- `Authorization: Bearer` (required if `PRANOR_JWT_SECRET` is set)
 - `Content-Type: multipart/form-data`
- Request:** Multipart upload with `.tar.gz` file containing `pranor.toml` manifest.

Response (201):

```
{
  "status": "published",
  "package": "my-module",
  "version": "1.2.0",
  "checksum": "sha256:abc123..."
}
```

GET `/api/v1/packages`

List all packages in the registry.

Response (200):

```
{
  "packages": [
```

```
{ "name": "my-module", "latest_version": "1.2.0", "published_at": "2026-08-01T10:00:00Z" },
{ "name": "utils-lib", "latest_version": "0.5.3", "published_at": "2026-07-28T14:30:00Z" }
]
```

GET /api/v1/packages/{name}/versions

List all versions of a package.

Response (200):

```
{
  "name": "my-module",
  "versions": ["1.0.0", "1.1.0", "1.2.0"]
}
```

GET /api/v1/packages/{name}/deps

Resolve dependency tree for the latest version.

Response (200):

```
{
  "package": "my-module",
  "version": "1.2.0",
  "dependencies": [
    { "name": "utils-lib", "version": ">=0.5.0", "resolved": "0.5.3" },
    { "name": "crypto-core", "version": "^2.0.0", "resolved": "2.1.1" }
  ]
}
```

GET /api/v1/packages/search?q={query}

Search packages by name or metadata.

Response (200):

```
{
  "results": [
    { "name": "my-module", "description": "Core utility module", "latest_version": "1.2.0" }
  ]
}
```

GET /packages/{name}.tar.gz

Download the latest version tarball.

GET /packages/{name}/{version}/{name}-{version}.tar.gz

Download a specific version tarball.

GET /healthz

Liveness probe.

```
{"status":"UP","service":"pranor-hub","version":"1.0.0"}
```

Security

Standalone Mode

When `PRANOR_JWT_SECRET`` is unset, publishing is unauthenticated. Set the JWT secret to require token authentication for all publish operations.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem:

1. **JWT Auth** — validates Bearer tokens against Pranor Auth
2. **Cosign Verification** — supply-chain signature validation on published artifacts
3. **RBAC** — organization-scoped publish permissions via Pranor Auth roles
4. **Artifact Signing** — all published packages signed with Sigstore transparency log

Package Integrity

- Packages are checksummed (SHA-256) on upload
- Cosign signatures verify publisher identity and build provenance
- Immutable versions — once published, a version cannot be overwritten

Observability

Prometheus Metrics

Metric Type Description			
----- ----- -----			
`pranor_hub_packages_total`	Gauge	Total registered packages	
`pranor_hub_publishes_total`	Counter	Publish events (labeled by status)	
`pranor_hub_downloads_total`	Counter	Package downloads	
`pranor_hub_resolution_duration_ms`	Histogram	Dependency resolution time	
`pranor_hub_storage_bytes`	Gauge	Total storage used	

OpenTelemetry Tracing

Hub emits spans for:

- ``hub.publish`` — package publication
- ``hub.resolve`` — dependency tree resolution
- ``hub.download`` — package download
- ``hub.verify`` — Cosign signature verification

Logging

Structured JSON logs with fields: ``level``, ``timestamp``, ``trace_id``, ``package``, ``version``, ``action``, ``publisher``.

Enterprise Edition

| Feature | OSS | EE |

|-----|:---|:---|

| S3-backed package storage | ✓ | ✓ |

| Semver dependency resolution | ✓ | ✓ |

| JWT publish authentication | ✓ | ✓ |

| Package search | ✓ | ✓ |

| Landing dashboard UI | ✓ | ✓ |

| Cosign supply-chain verification | ✓ | ✓ |

| OCI container image backend | — | ✓ |

| Air-gapped private package mirror | — | ✓ |

| Organization-scoped RBAC publishing | — | ✓ |

| Vulnerability scanning on publish | — | ✓ |

| Package deprecation & yanking | — | ✓ |

Operational Runbook

Package publish failing with auth error

1. Verify ``PRANOR_JWT_SECRET`` is configured correctly
2. Check JWT token validity and expiration
3. Ensure the publishing user has the correct RBAC role
4. If using Cosign, verify the signing key is available

Dependency resolution failing

1. Check if all declared dependencies exist in the registry
2. Review version constraints in ``pranor.toml`` for conflicts
3. Check for circular dependency chains
4. Monitor ``pranor_hub_resolution_duration_ms`` for timeout issues

S3 storage backend unavailable

1. Verify `PRANOR\_STORE\_ENDPOINT` connectivity
2. Check S3 access key/secret key credentials
3. Verify the target bucket exists and has correct permissions
4. If using Pranol Vault, check Vault health endpoint

Slow package downloads

1. Check S3 backend latency and throughput
2. Review `pranol\_hub\_downloads\_total` for traffic spikes
3. Consider using a CDN or regional cache in front of Hub
4. Verify network bandwidth between Hub and storage backend

Pranor Lock — Distributed Lock Manager

Version: 1.0.0

Module Path: `github.com/vyuvvaraj/pranor/lock`

Default Port: 8089

License: AGPL-3.0 (OSS) / Enterprise License (EE with Raft consensus)

Overview

Pranor Lock is a lightweight, production-grade distributed lock manager that provides lease-based mutual exclusion for coordinating access to shared resources across services. It supports exclusive and shared lock modes, priority-based wait queues, deadlock detection, fencing tokens, reentrant locks, real-time event streaming, and client heartbeat monitoring.

Pranor Lock can run as:

- A **standalone binary** with zero external dependencies (memory or file-backed)
- An **integrated module** within the Pranor ecosystem with mTLS, RBAC, and OTel tracing

Table of Contents

- [Key Features](#key-features)
- [Architecture](#architecture)
- [Installation & Deployment](#installation--deployment)
- [Configuration](#configuration)
- [API Reference](#api-reference)
- [Lock Semantics](#lock-semantics)
- [Storage Backends](#storage-backends)
- [Security](#security)
- [Observability](#observability)
- [Client Libraries](#client-libraries)
- [Enterprise Edition](#enterprise-edition)
- [Operational Runbook](#operational-runbook)

Key Features

Feature	Description
Lease-based TTL locks	Every lock has an expiry. No permanent deadlocks from crashed clients.
Exclusive & Shared modes	Read-write lock semantics. Multiple shared readers, single exclusive writer.
Fencing tokens	Monotonically increasing tokens prevent stale clients from corrupting state.
Reentrant locks	Same owner+client_id can re-acquire without blocking. Reentrancy count tracked.

| **Priority wait queues** | Waiters are served in priority order. Higher priority clients jump the queue. |

| **Deadlock detection** | Cycle detection in the wait-for graph prevents distributed deadlocks. |

| **Blocking acquire** | Optional `wait\_ms` parameter blocks until lock is available or timeout. |

| **Real-time SSE events** | Subscribe to lock lifecycle events (released, expired) via Server-Sent Events. |

| **Client heartbeats** | Dead client detection — locks auto-release when heartbeats stop. |

| **File persistence** | Optional file-backed storage survives process restarts. |

| **Zombie lock alerts** | Logs a warning when locks are held longer than 5 seconds. |

| **Prometheus metrics** | Active locks, waiter count, deadlock counter. |

Architecture

```
graph TD
classDef client fill:#1e293b,stroke:#38bdf8,stroke-width:2px,color:#fff;
classDef engine fill:#0f172a,stroke:#0d9488,stroke-width:2px,color:#fff;
classDef storage fill:#1e1b4b,stroke:#6366f1,stroke-width:2px,color:#fff;
classDef monitor fill:#1e293b,stroke:#64748b,stroke-width:1px,color:#fff;

subgraph API ["■ Access & Stream Interface"]
  REST["HTTP REST API<br/><i>(Acquire / Release / Renew)</i>"] :::client
  Auth["Auth & Security Layer<br/><i>(mTLS / JWT / API Key)</i>"] :::client
  SSE["SSE Pub/Sub Stream<br/><i>(Real-Time Lock Events)</i>"] :::client
end

subgraph Core ["■ Core Distributed Lock Engine"]
  Reentrant["Reentrancy & Lease Engine<br/><i>(Exclusive & Shared Modes)</i>"] :::engine
  Fencing["Monotonic Fencing Token Generator"] :::engine
  Deadlock["Deadlock Cycle Detector<br/><i>(Wait-For Graph Evaluator)</i>"] :::engine
  Priority["Priority Wait Queue Manager"] :::engine
end

subgraph Backend ["■ Persisted Lock Store"]
  MemStore["In-Memory Lock Store<br/><i>(Zero-Allocation)</i>"] :::storage
  FileStore["File-Backed Lease Store"] :::storage
  RaftStore["Raft Consensus Engine<br/><i>(Enterprise EE)</i>"] :::storage
end

subgraph Background ["■ Background Monitors"]
  TTLCleaner["TTL Lease Evictor<br/><i>(500ms Sweep)</i>"] :::monitor
  Heartbeat["Client Heartbeat Monitor"] :::monitor
end

REST --> Auth
```

```

Auth --> Reentrant
Reentrant --> Fencing
Fencing --> Deadlock
Deadlock --> Priority
Priority --> MemStore
Priority --> FileStore
Priority --> RaftStore
TTLCleaner --> MemStore
Heartbeat --> Reentrant
Reentrant --> SSE

```

Lease Acquisition & Fencing Token Sequence Flow

```

sequenceDiagram
    autonumber
    participant Worker as Client / Worker Instance
    participant Lock as Pranor Lock Manager
    participant Deadlock as Deadlock Cycle Evaluator
    participant Storage as Raft / File Lock Store
    participant DB as Target Storage / Database

    Worker->>Lock: POST /api/locks/acquire (key="orders/process", duration_ms=10000)
    Lock->>Deadlock: Evaluate Wait-For Graph (Cycle Detection)
    Deadlock-->>Lock: Cycle Free (No Deadlock)
    Lock->>Storage: Issue Monotonic Fencing Token (Token=1042)
    Storage-->>Lock: Lock State Persisted & Lease TTL Set
    Lock-->>Worker: Lock Granted (Fencing Token = 1042)
    Worker->>DB: Write Record with Fencing Token = 1042
    DB-->>Worker: Write Validated (Token 1042 > Previous 1041)
    Worker->>Lock: POST /api/locks/renew (Heartbeat Keepalive)
    Lock-->>Worker: TTL Extended (10,000ms refreshed)
    Worker->>Lock: POST /api/locks/release (Fencing Token = 1042)
    Lock-->>Worker: Lock Released & Next Waiter Notified via SSE

```

Ecosystem Cross-Module Integration

Pranor Lock provides distributed synchronization across all core ecosystem components:

- **Pranor Chrono:** Uses exclusive fencing token locks to ensure distributed cron jobs trigger on exactly one node during multi-replica deployments.
- **Pranor Flow:** Manages saga execution state locks, preventing concurrent workers from processing duplicate saga compensation steps.
- **Pranor Pool:** Coordinates online database DDL migrations, ensuring zero-downtime schema changes are executed by a single leader node.

- **Pranor Auth:** Enforces single-session user login restrictions across clusters when configured in strict single-tenant security mode.
- **Pranor Trace:** Emits lock contention metrics, wait-queue durations, and deadlock cycle detections directly to OpenTelemetry traces.

Installation & Deployment

Binary

```
cd pranor/lock
go build -o pranor-lock .
./pranor-lock --port 8089
```

Docker

```
docker run -p 8089:8089 ghcr.io/vyuvaraj/pranor-lock:latest
```

With Config File

```
./pranor-lock --config lock.yaml
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Lock integrates automatically with Auth (JWT/mTLS), Trace (OTel spans), and Console (dashboard visibility).

Configuration

YAML Config (`lock.yaml`)

```
port: "8089"
backend: "file" # "memory" or "file"
file_path: "leases.json" # Only used when backend is "file"
api_key: "your-secret" # Optional: standalone API key auth
tls_cert: "" # Path to TLS certificate
tls_key: "" # Path to TLS private key
client_ca: "" # Path to CA cert for mTLS client verification
```

Environment Variables

Variable	Default	Description
----- ----- -----		
`PRANOR_LOCK_API_KEY`	—	API key for standalone auth
`PRANOR_OTLP_ENDPOINT`	—	OpenTelemetry collector URL

CLI Flags

Flag	Default	Description
----- ----- -----		
`--port`	`8089`	HTTP listen port
`--config`	—	Path to YAML config file

API Reference

Base URL: ``http://localhost:8089``

API Version: ``/api/v1`` (recommended) or ``/api`` (legacy)

POST `/api/locks/acquire`

Acquire a distributed lock.

Request:

```
{
  "key": "orders/processing",
  "owner": "worker-1",
  "client_id": "instance-abc",
  "duration_ms": 30000,
  "wait_ms": 5000,
  "mode": "exclusive",
  "priority": 10
}
```

Field	Type	Required	Description
----- ----- -----			
`key`	string	✓	Lock identifier (namespace/resource)
`owner`	string	✓	Who is requesting the lock
`client_id`	string		Instance identifier (enables reentrancy)
`duration_ms`	int		Lease TTL in ms (default: 10000)
`wait_ms`	int		Block until lock available (0 = fail immediately)
`mode`	string		“exclusive” (default) or “shared”
`priority`	int		Higher = served first in wait queue

Success Response (200):

```
{
  "status": "success",
  "lock": {
    "key": "orders/processing",
    "owner": "worker-1",
```

```
"client_id": "instance-abc",
"reentrancy_count": 1,
"fencing_token": 42,
"expires_at": "2026-08-01T10:00:30Z",
"mode": "exclusive",
"acquired_at": "2026-08-01T10:00:00Z"
}
}
```

Conflict Response (409):

```
{
  "status": "failed",
  "message": "lock conflict: key \"orders/processing\" is held in mode \"exclusive\""
}
```

Deadlock Response (409):

```
{
  "status": "failed",
  "message": "deadlock detected: cycle in lock wait queue"
}
```

POST /api/locks/release

Release a held lock.

Request:

```
{
  "key": "orders/processing",
  "owner": "worker-1",
  "fencing_token": 42
}
```

Field	Type	Required	Description
key	string	✓	Lock to release
owner	string	✓	Must match the lock holder
fencing_token	int64		If provided, must match (prevents stale releases)

Response (200):

```
{
  "status": "success",
  "message": "Lock released successfully"
}
```

POST /api/locks/renew

Extend the lease of an active lock.

Request:

```
{
  "key": "orders/processing",
  "owner": "worker-1",
  "fencing_token": 42,
  "duration_ms": 30000
}
```

Response (200):

```
{
  "status": "success",
  "message": "Lock lease renewed successfully"
}
```

POST /api/locks/heartbeat

Ping to indicate client is alive. If heartbeats stop for >5s, all locks held by that client are auto-released.

Request:

```
{
  "client_id": "instance-abc"
}
```

Response (200):

```
{
  "status": "success"
}
```

GET /api/locks/observability

List all active locks with their waiters.

Response (200):

```
[
  {
    "key": "orders/processing",
    "owner": "worker-1",
    "fencing_token": 42,
    "expires_at": "2026-08-01T10:00:30Z",
  }
]
```

```
"waiters": ["worker-2", "worker-3"]
}
```

GET /api/locks/metrics

Prometheus-compatible metrics endpoint.

Response (200 text/plain):

```
# HELP pranor_lock_active_locks Number of active locks currently held
# TYPE pranor_lock_active_locks gauge
pranor_lock_active_locks 3

# HELP pranor_lock_waiters_count Total number of clients waiting for locks
# TYPE pranor_lock_waiters_count gauge
pranor_lock_waiters_count 1

# HELP pranor_lock_deadlocks_total Total number of deadlocks detected
# TYPE pranor_lock_deadlocks_total counter
pranor_lock_deadlocks_total 0
```

GET /api/locks/subscribe

Server-Sent Events stream for real-time lock lifecycle events.

Response (text/event-stream):

```
: keep-alive

data: {"key":"orders/processing","action":"released"}

data: {"key":"inventory/update","action":"expired"}
```

Events:

- `released` — lock explicitly released by owner
- `expired` — lock TTL expired or client heartbeat timed out

GET /healthz

Liveness probe.

```
{"status":"UP","service":"pranor","version":"1.0.0"}
```

GET /readyz

Readiness probe. Same format as healthz.

Lock Semantics

Exclusive Mode (Default)

Only one owner can hold the lock. All other acquire attempts either fail immediately or block (if `wait\_ms` > 0).

```
Worker-1: acquire("key", exclusive) → ✓ granted
Worker-2: acquire("key", exclusive) → ✗ conflict (or blocks)
Worker-1: release("key") → ✓
Worker-2: (if waiting) → ✓ auto-granted
```

Shared Mode

Multiple owners can hold a shared lock simultaneously. Exclusive requests block until all shared locks are released.

```
Reader-1: acquire("key", shared) → ✓ granted
Reader-2: acquire("key", shared) → ✓ granted (concurrent)
Writer-1: acquire("key", exclusive) → ✗ blocks (shared locks active)
Reader-1: release → ✓
Reader-2: release → ✓
Writer-1: → ✓ auto-granted (all readers done)
```

Reentrancy

If the same `owner` + `client\_id` acquires a lock they already hold, the reentrancy count increments. The lock is only fully released when the count reaches zero.

```
Worker-1: acquire("key") → reentrancy_count: 1
Worker-1: acquire("key") → reentrancy_count: 2 (no block)
Worker-1: release("key") → reentrancy_count: 1 (still held)
Worker-1: release("key") → reentrancy_count: 0 (fully released)
```

Fencing Tokens

Every lock acquisition generates a monotonically increasing fencing token. Downstream systems should validate the token to reject operations from stale lock holders:

```
Worker-1: acquire → fencing_token: 41
Worker-1: crashes, lock expires
Worker-2: acquire → fencing_token: 42
Worker-1: wakes up, tries write with token 41 → REJECTED
Worker-2: writes with token 42 → ACCEPTED
```

Deadlock Detection

When `wait\_ms` > 0, the engine checks for cycles in the wait-for graph before queueing:

```
Worker-A holds Lock-X, waiting for Lock-Y
Worker-B holds Lock-Y, waiting for Lock-X
```



```
→ Cycle detected → "deadlock detected" error returned immediately
```

Priority Queue

When multiple waiters exist for a lock, they are served in descending priority order (higher number = higher priority):

```
Worker-A (priority: 1) waiting
Worker-B (priority: 10) waiting
Worker-C (priority: 5) waiting
Lock released → Worker-B gets it first
```

Storage Backends

InMemory (Default)

- Zero configuration
- All state in memory
- Lost on restart
- Best for: development, testing, ephemeral workloads

File-Backed

- Persists leases to JSON file (`leases.json`)
- Survives process restarts
- Loads non-expired leases on startup
- Best for: single-node production, edge deployments

```
backend: "file"
file_path: "/var/pranor/lock/leases.json"
```

Raft Consensus (Enterprise)

- Multi-node replication
- Strong consistency
- Automatic leader election
- Best for: production HA deployments

Security

Standalone Mode (API Key)

Set `PRANOR\_LOCK\_API\_KEY` or configure in YAML. Clients authenticate via:

```
X-API-Key: your-secret
```

or:

```
Authorization: Bearer your-secret
```

Health endpoints (`/healthz`, `/readyz`) are unauthenticated.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem (no API key configured), the full middleware chain activates:

- 1. **OTel Tracing** — every request gets a span
- 2. **Rate Limiting** — per-client request throttling
- 3. **CORS** — cross-origin request handling
- 4. **Max Body Size** — 10MB request body limit
- 5. **JWT Auth** — validates Bearer tokens against Pranor Auth
- 6. **Tenant Isolation** — multi-tenant namespace enforcement

mTLS

Enable mutual TLS for service-to-service authentication:

```
tls_cert: "/certs/lock.crt"
tls_key: "/certs/lock.key"
client_ca: "/certs/ca.crt"

---
```

Observability

Metrics

Metric	Type	Description
pranor_lock_active_locks	Gauge	Currently held locks
pranor_lock_waiters_count	Gauge	Clients waiting in queues
pranor_lock_deadlocks_total	Counter	Total deadlocks detected

Real-time Events (SSE)

Connect to `/api/locks/subscribe` for real-time lock state changes. Useful for building dashboards or triggering downstream workflows.

Zombie Lock Alerts

Locks held longer than 5 seconds generate a log warning:

```
[Warning] Zombie Lock Alert: Lock on "orders/processing" was held for 12.3s
```

Heartbeat Monitoring

If a client stops sending heartbeats for >5 seconds, all its locks are automatically released and an `expired` event is broadcast.

Client Libraries

Go (via Pranor Core)

```
import "github.com/vyuvaraj/pranor/core"

client := core.NewLockClient("http://localhost:8089", "your-api-key")
lock, err := client.Acquire("orders/processing", "worker-1", 30*time.Second)
defer client.Release(lock)
```

cURL

```
# Acquire
curl -X POST http://localhost:8089/api/v1/locks/acquire \
-H "X-API-Key: your-secret" \
-H "Content-Type: application/json" \
-d '{"key":"my-resource","owner":"worker-1","duration_ms":30000}'

# Renew
curl -X POST http://localhost:8089/api/v1/locks/renew \
-H "X-API-Key: your-secret" \
-d '{"key":"my-resource","owner":"worker-1","duration_ms":30000}'

# Release
curl -X POST http://localhost:8089/api/v1/locks/release \
-H "X-API-Key: your-secret" \
-d '{"key":"my-resource","owner":"worker-1"}'
```

Pranor CLI

```
pranor lock acquire --key orders/processing --owner worker-1 --ttl 30s
pranor lock renew --key orders/processing --owner worker-1 --ttl 30s
pranor lock release --key orders/processing --owner worker-1
pranor lock list
```

Enterprise Edition

| Feature | OSS | EE |

|-----|:---|:---|

| InMemory backend | ✓ | ✓ |

| File-backed persistence | ✓ | ✓ |

| API Key auth | ✓ | ✓ |

| Shared/Exclusive modes | ✓ | ✓ |

| Deadlock detection | ✓ | ✓ |

| Priority queues | ✓ | ✓ |

SSE event stream	✓	✓
Client heartbeats	✓	✓
Raft consensus replication	—	✓
Multi-node HA	—	✓
Automatic failover	—	✓

Operational Runbook

Lock stuck / not releasing

1. Check ``/api/locks/observability`` for the lock state
2. Verify the owner's heartbeat is active
3. If owner is dead, wait for TTL expiry (or heartbeat timeout)
4. As last resort, release manually via API with matching owner

High waiter count

1. Check ``/api/locks/metrics`` for ``pranor_lock_waiters_count``
2. Identify hot keys via ``/api/locks/observability``
3. Consider:
 - Increasing lock TTL (reduce churn)
 - Switching to shared mode if readers dominate
 - Sharding the resource key

Deadlocks increasing

1. Monitor ``pranor_lock_deadlocks_total``
2. Review client code for multi-key acquisition patterns
3. Enforce consistent lock ordering across all services
4. Consider using shorter TTLs so deadlocked chains resolve via expiry

Process restart (file backend)

On restart, the file store loads all non-expired leases from ``leases.json``. Locks that expired during downtime are automatically cleaned up.

Versioning & Compatibility

- API is versioned at ``/api/v1/``
- Legacy ``/api/`` paths continue to work (internally rewritten to v1)
- Fencing tokens are monotonically increasing and never reset (even across restarts with file backend)

Pranor Mesh — Intelligent Service Mesh

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/mesh`

Default Port: 8089

License: AGPL-3.0 (OSS) / Enterprise License (EE with WireGuard & mTLS Attestation)

Overview

Pranor Mesh is the intelligent service mesh for the Pranor ecosystem, providing latency-aware Power-of-Two-Choices (P2C) load balancing, distributed rate limiting, live topology telemetry, circuit breaking, mTLS, and chaos fault injection — all without requiring sidecar proxies.

Pranor Mesh can run as:

- A **standalone binary** providing load balancing and service discovery
- An **integrated module** within the Pranor ecosystem with distributed rate limiting via Cache, topology push to Console, and mTLS via Auth

Key Features

Feature	Description
P2C Load Balancing	Power-of-Two-Choices with latency-aware backend selection
Locality Preference	Prefer backends in the same AZ before spilling to remote nodes
Distributed Rate Limiting	Global rate limits via Pranor Cache token buckets
Circuit Breaking	Automatic circuit open/half-open/closed state per backend
Live Topology	Real-time service dependency graph pushed to Console
Chaos Fault Injection	Latency injection, error simulation, network partition
Health-aware Routing	Unhealthy backends excluded with exponential recovery probing
mTLS	Mutual TLS for encrypted service-to-service communication
Traffic Flow Visualization	Edges annotated with RPS, error rate, and p99 latency
Microsegmentation	eBPF L4/L7 policy enforcement between services

Architecture

```
graph TD
    subgraph ServiceTraffic ["Encrypted Service Connectivity"]
        ClientService["Client Service Pod / Host"]
        mTLSSidecar["mTLS Auto-Inject Sidecar Proxy"]
    end
```

```

WireGuardMesh["WireGuard Private Network Mesh Overlay"]
end

subgraph MeshCore ["■ Zero-Trust Control and Microsegmentation"]
P2CRouter["Power-of-Two-Choices (P2C) Load Balancer"]
Microseg["eBPF Layer 4/7 Microsegmentation Policy Engine"]
BFTRaft["Byzantine Fault Tolerant (BFT) Raft Control Plane"]
ChaosEngine["In-Situ Chaos Experiment Injector"]
end

subgraph PlatformSync ["■ Ecosystem Sync and Observability"]
CacheLimit["Pranor Cache Shared Token Bucket"]
ConsoleTopology["Pranor Console Live Topology Emitter"]
end

ClientService --> mTLSSidecar
mTLSSidecar --> WireGuardMesh
WireGuardMesh --> P2CRouter
P2CRouter --> Microseg
Microseg --> BFTRaft
BFTRaft --> ChaosEngine
ChaosEngine --> CacheLimit
ChaosEngine --> ConsoleTopology

```

Power-of-Two-Choices (P2C) Routing & Microsegmentation Sequence Flow

```

sequenceDiagram
    autonumber
    participant Caller as Caller Service A
    participant Mesh as Pranor Mesh Control Plane
    participant eBPF as eBPF Microsegmentation Guard
    participant Backend as Selected Target Service B

    Caller->>Mesh: POST /api/v1/route (Service B, Locality Zone: "us-east-1a")
    Mesh->>eBPF: Validate L4/L7 Zero-Trust Microsegmentation Policy
    eBPF-->>Mesh: Traffic Authorized (Policy Passed)
    Mesh->>Mesh: Pick 2 Random Candidate Endpoints & Evaluate p99 Latency (P2C)
    Mesh->>Backend: Route Mutual TLS Request (WireGuard Overlay)
    Backend-->>Mesh: Response Payload + Health Status
    Mesh-->>Caller: Selected Endpoint Response (Sub-millisecond Latency)

```

Ecosystem Cross-Module Integration

Pranor Mesh manages secure inter-service communication across all ecosystem components:

- **Pranor Gate:** Acts as the external ingress target for Mesh WireGuard overlay tunnels and mTLS sidecar proxies.

- **Pranor Cache:** Shares token bucket rate-limiting counters across all cluster Mesh nodes for global traffic shaping.
- **Pranor Auth:** Enforces SPIFFE/SPIRE workload identities and mutual TLS (mTLS) certificate verification per service route.
- **Pranor Console:** Renders live service topology dependency graphs, real-time latency heatmaps, and active chaos experiment controls.

Installation & Deployment

Binary

```
cd pranor/mesh
go build -o pranor-mesh .
./pranor-mesh --port 8089
```

Docker

```
docker run -p 8089:8089 ghcr.io/vyuvaraj/pranor-mesh:latest
```

With Distributed Rate Limiting

```
docker run -p 8089:8089 \
-e PRANOR_MESH_PRANOR_CACHE_URL=http://pranor-cache:8086 \
-e PRANOR_MESH_PRANOR_CONSOLE_WS_URL=ws://pranor-console:8083/ws/topology \
ghcr.io/vyuvaraj/pranor-mesh:latest
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Mesh integrates automatically with Cache (rate limiting), Console (topology), Auth (mTLS), and Trace (OTel spans).

Configuration

Environment Variables

Variable	Default	Description
PRANOR_MESH_PORT	8089	HTTP listener port
PRANOR_MESH_PRANOR_CACHE_URL		Pranor Cache URL for distributed rate limit state
PRANOR_MESH_PRANOR_CONSOLE_WS_URL		Pranor Console WebSocket URL for topology push
PRANOR_MESH_LOCALITY_ZONE		Availability zone for locality-preference routing
PRANOR_MESH_OTEL_ENDPOINT		OpenTelemetry collector URL

YAML Config (`mesh.yaml`)

```
port: "8089"
cache_url: "http://pranor-cache:8086"
console_ws_url: "ws://pranor-console:8083/ws/topology"
locality_zone: "us-east-1a"
otel_endpoint: "http://pranor-trace:8090"
circuit_breaker:
failure_threshold: 5
recovery_timeout: "30s"
```

CLI Flags

Flag	Default	Description
--port	8089	HTTP listen port

API Reference

Base URL: `http://localhost:8089`

POST /api/v1/services

Register a service endpoint.

Request:

```
{
  "name": "orders-api",
  "endpoints": ["http://orders-1:3000", "http://orders-2:3000", "http://orders-3:3000"],
  "locality_zone": "us-east-1a"
}
```

Response (201):

```
{
  "status": "registered",
  "service": "orders-api",
  "endpoint_count": 3
}
```

POST /api/v1/route

Route a request via P2C selection.

Request:

```
{
```



```
"service": "orders-api",
"caller_zone": "us-east-1a"
}
```

Response (200):

```
{
"selected_endpoint": "http://orders-2:3000",
"latency_p99_ms": 12,
"locality_match": true
}
```

POST /api/v1/ratelimit/policy

Set rate limit policy for a service.

Request:

```
{
"service": "orders-api",
"requests_per_second": 500,
"burst": 1000
}
```

Response (200):

```
{
"status": "applied",
"service": "orders-api"
}
```

POST /api/v1/chaos/inject

Inject a chaos fault.

Request:

```
{
"target_service": "payments-api",
"fault_type": "latency",
"latency_ms": 200,
"percentage": 30,
"duration": "5m"
}
```

Response (201):

```
{
"id": "exp-123",
```

```
"status": "active",
"expires_at": "2026-08-01T10:05:00Z"
}
```

GET /api/v1/topology

Current topology graph snapshot.

Response (200):

```
{
  "services": ["orders-api", "payments-api", "inventory-api"],
  "edges": [
    { "from": "orders-api", "to": "payments-api", "rps": 120, "p99_ms": 45 }
  ]
}
```

GET /healthz

Liveness probe.

```
{"status":"UP","service":"pranor-mesh","version":"1.0.0"}
```

Security

Standalone Mode

In standalone mode, Mesh provides unauthenticated load balancing and service discovery.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem:

1. **mTLS** — mutual TLS for all service-to-service traffic
2. **SPIFFE/SPIRE** — workload identity attestation per service
3. **eBPF Microsegmentation** — L4/L7 zero-trust policy enforcement
4. **WireGuard Overlay** — encrypted mesh network between nodes
5. **Token-bucket rate limiting** — global enforcement via Pranor Cache

Circuit Breaking

Mesh implements circuit breaking per backend:

- **Closed**: Normal traffic flow
- **Open**: All requests fast-fail (after failure threshold)
- **Half-Open**: Limited probe requests to test recovery

Observability

Prometheus Metrics

Metric	Type	Description
----- ----- -----		
`pranor_mesh_routing_decisions_total`	Counter	Total P2C routing decisions
`pranor_mesh_rate_limit_hits_total`	Counter	Rate limit rejections
`pranor_mesh_chaos_faults_active`	Gauge	Active chaos experiments
`pranor_mesh_circuit_breaker_state`	Gauge	Circuit state per backend (0=closed, 1=open, 2=half-open)
`pranor_mesh_backend_latency_ms`	Histogram	Backend response latency
`pranor_mesh_topology_edges`	Gauge	Active service-to-service edges

OpenTelemetry Tracing

- Mesh emits spans for:
- `mesh.route` — P2C routing decision
 - `mesh.ratelimit.check` — rate limit evaluation
 - `mesh.chaos.inject` — chaos fault injection
 - `mesh.circuit.trip` — circuit breaker state change

Logging

Structured JSON logs with fields: `level`, `timestamp`, `trace\_id`, `service`, `endpoint`, `latency\_ms`, `action`.

Enterprise Edition

Feature	OSS	EE
----- :--- :--		
P2C load balancing	✓	✓
Service registration & discovery	✓	✓
Locality-aware routing	✓	✓
Distributed rate limiting (via Cache)	✓	✓
Chaos fault injection	✓	✓
Circuit breaking	✓	✓
Live topology telemetry	✓	✓
WireGuard kernel tunnel mesh	—	✓
SPIFFE/SPIRE mTLS workload attestation	—	✓
eBPF L4/L7 microsegmentation	—	✓
BFT Raft control plane	—	✓

Operational Runbook

High tail latency on routed requests

1. Check ``pranor_mesh_backend_latency_ms`` histogram for p99 spikes
2. Review which backends are being selected — P2C should prefer faster ones
3. Verify locality zone configuration matches actual deployment topology
4. Check if circuit breaker is tripping on slow backends
5. Look for active chaos experiments affecting the target service

Rate limiting blocking legitimate traffic

1. Check ``pranor_mesh_rate_limit_hits_total`` for unexpected rejections
2. Review rate limit policy: ``GET /api/v1/ratelimit/policy``
3. Verify Pranol Cache connectivity — rate limit state is shared globally
4. Increase burst allowance if traffic is legitimately spiky

Topology graph missing services

1. Verify services are registered: ``GET /api/v1/services``
2. Check Console WebSocket connectivity (``PRANOR_MESH_PRANOR_CONSOLE_WS_URL``)
3. Ensure services are actually making calls through Mesh (not direct)
4. Review Mesh logs for registration errors

Chaos experiment not auto-expiring

1. Check experiment status: ``GET /api/v1/chaos/active``
2. Verify system clock is accurate (expiry is time-based)
3. Manually abort: ``POST /api/v1/chaos/abort/{id}``
4. Review duration configuration in the inject request

Pranor Notify — Multi-Channel Notification Engine

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/notify`

Default Port: 8094

License: AGPL-3.0 (OSS) / Enterprise License (EE with AI Deliverability & WebPush)

Overview

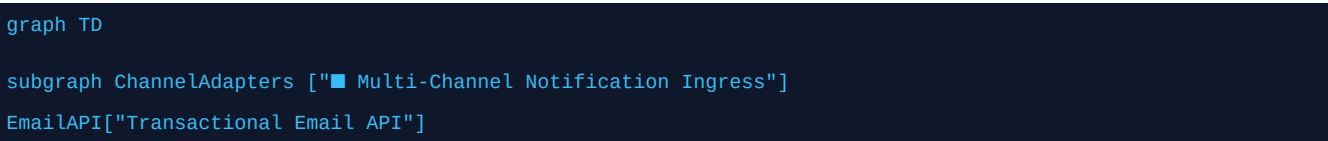
Pranor Notify is the transactional email, SMS, and push notification service for the Pranor ecosystem. It handles sending, receiving, bounce management, unsubscribe compliance (RFC 8058), DMARC/SPF/DKIM enforcement, inbound email routing, and provides a rich templating DSL with delivery analytics.

- Pranor Notify can run as:
- A **standalone binary** with SMTP relay configuration for email delivery
 - An **integrated module** within the Pranor ecosystem with multi-channel dispatch, OTel tracing, and Console analytics
-

Key Features

Feature	Description
Transactional Email	REST API for HTML/plain text emails with attachments, CC/BCC
Template DSL	Variable interpolation, conditionals, loops, partials, and layouts
SMTP Relay	Route via SendGrid, AWS SES, Mailgun, or custom SMTP
Inbound Routing	Route incoming emails to HTTP webhooks based on rules
Bounce Management	Auto-suppression list with hard/soft bounce classification
DMARC Enforcement	SPF/DKIM/DMARC alignment checking and aggregate reports
RFC 8058 Unsubscribe	One-click unsubscribe headers on all bulk emails
SMS Gateway	Twilio and multi-carrier SMS delivery
WebPush / APNs	Browser push and Apple Push Notification delivery
Delivery Analytics	Per-campaign rates, opens, clicks, bounces, complaints
Suppression List	Automatic and manual address suppression management

Architecture



```

PushAPI["WebPush and APNs Provider"]
SMSAPI["Twilio and Multi-Carrier SMS Gateway"]
end

subgraph DispatchEngine ["■ Template and Deliverability Engine"]
TemplateEngine["HTML / DSL Template Rendering Engine"]
DMARCVa1["DMARC / SPF / DKIM Inspector and Alignment"]
SuppressionList["Automatic Bounce and Suppression Filter"]
AIOptimizer["AI Deliverability and Send-Time Optimizer"]
end

subgraph Relays ["■ Provider Relays and Analytics"]
SMTPRelay["Outbound SMTP Relay Pool"]
WebhookRouter["Inbound Webhook and RFC 8058 Unsubscribe Router"]
end

EmailAPI --> TemplateEngine
PushAPI --> TemplateEngine
SMSAPI --> TemplateEngine
TemplateEngine --> DMARCVa1
DMARCVa1 --> SuppressionList
SuppressionList --> AIOptimizer
AIOptimizer --> SMTPRelay
SMTPRelay --> WebhookRouter

```

Notification Dispatch & Bounce Suppression Sequence Flow

```

sequenceDiagram
autonumber
participant App as Application Microservice
participant Notify as Pranor Notify Engine
participant Suppression as Suppression List
participant Template as DSL Template Renderer
participant Gateway as SMTP / SMS / Push Gateway
participant Analytics as Pranor Console Analytics

App->>Notify: POST /api/v1/send/template (Template: "welcome-email", User Email)
Notify->>Suppression: Check Address against Hard-Bounce Suppression List
Suppression-->>Notify: Clean Record (Not Suppressed)
Notify->>Template: Inject Payload Variables into DSL Template
Template-->>Notify: Rendered HTML Body + List-Unsubscribe-Post Header
Notify->>Gateway: Relay Encrypted Payload via Outbound SMTP / Push Gateway
Gateway-->>Notify: Delivery Acknowledgment (Message ID: msg-7718)
Notify->>Analytics: Push Delivery Telemetry & Open/Click Trackers

```

Ecosystem Cross-Module Integration

Pranor Notify delivers multi-channel communications across the Pranor platform:

- **Pranor Auth:** Sends one-time password (OTP) codes for multi-factor authentication (MFA) step-up login challenges.
- **Pranor Trace:** Annotates notification dispatch events with OpenTelemetry traces, recording deliverability latency flamegraphs.
- **Pranor Flow:** Triggers customer communication steps in saga workflows (e.g., order confirmation emails, shipment SMS alerts).
- **Pranor Console:** Renders live deliverability analytics, bounce rate histograms, and template editor UI.

Installation & Deployment

Binary

```
cd pranor/notify
go build -o pranor-notify .
./pranor-notify --port 8094
```

Docker

```
docker run -p 8094:8094 ghcr.io/vyuvaraj/pranor-notify:latest
```

With SMTP Configuration

```
docker run -p 8094:8094 \
-e PRANOR_NOTIFY_SMTP_HOST=smtp.sendgrid.net \
-e PRANOR_NOTIFY_SMTP_PORT=587 \
-e PRANOR_NOTIFY_SMTP_USER=apikey \
-e PRANOR_NOTIFY_SMTP_PASS=SG.xxxxx \
-e PRANOR_NOTIFY_FROM_DOMAIN=yourapp.com \
ghcr.io/vyuvaraj/pranor-notify:latest
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Notify integrates automatically with Auth (MFA OTP), Trace (OTel spans), Flow (saga steps), and Console (analytics dashboard).

Configuration

Environment Variables

Variable	Default	Description
----- ----- -----		
`PRANOR_NOTIFY_PORT`	`8094`	HTTP listener port

`PRANOR_NOTIFY_SMTP_HOST` — Outbound SMTP relay host
`PRANOR_NOTIFY_SMTP_PORT` `587` Outbound SMTP relay port
`PRANOR_NOTIFY_SMTP_USER` — SMTP authentication username
`PRANOR_NOTIFY_SMTP_PASS` — SMTP authentication password
`PRANOR_NOTIFY_FROM_DOMAIN` — Default sending domain
`PRANOR_NOTIFY_INBOUND_PORT` — SMTP port for inbound mail reception
`PRANOR_NOTIFY_DMARC_ENABLED` `true` Enable DMARC enforcement
`PRANOR_NOTIFY_OTEL_ENDPOINT` — OpenTelemetry collector URL

YAML Config (`notify.yaml`)

```
port: "8094"
smtp:
  host: "smtp.sendgrid.net"
  port: 587
  user: "apikey"
  pass: "SG.xxxxx"
  from_domain: "yourapp.com"
  inbound_port: 25
  dmarc_enabled: true
  otel_endpoint: "http://pranor-trace:8090"
```

CLI Flags

Flag	Default	Description
----	-----	-----
`--port`	`8094`	HTTP listen port

API Reference

Base URL: `http://localhost:8094`

POST `/api/v1/send`

Send a transactional email.

Request:

```
{
  "to": "alice@example.com",
  "from": "noreply@yourapp.com",
  "subject": "Order Confirmation",
  "html": "<h1>Thanks for your order!</h1>",
  "text": "Thanks for your order!",
  "cc": ["admin@yourapp.com"],
```



```
"attachments": []
}
```

Response (200):

```
{
  "status": "sent",
  "message_id": "msg-7718",
  "delivered_at": "2026-08-01T10:00:01Z"
}
```

POST /api/v1/send/template

Send using a named template.

Request:

```
{
  "template": "welcome-email",
  "to": "alice@example.com",
  "variables": {
    "user": { "name": "Alice", "verified": true }
  }
}
```

Response (200):

```
{
  "status": "sent",
  "message_id": "msg-7719",
  "template": "welcome-email"
}
```

POST /api/v1/templates

Create or update an email template.

Request:

```
{
  "name": "welcome-email",
  "subject": "Welcome, {{ user.name }}!",
  "html": "<h1>Welcome, {{ user.name }}!</h1>\n{% if user.verified %}<p>Verified.</p>{% endif %}"
}
```

Response (201):

```
{
  "status": "created",
}
```

```
"name": "welcome-email"
}
```

GET /api/v1/suppression

List suppressed addresses.

Response (200):

```
{
  "addresses": [
    { "email": "bad@example.com", "reason": "hard_bounce", "suppressed_at": "2026-07-30T08:00:00Z" }
  ]
}
```

POST /api/v1/inbound/rules

Create an inbound routing rule.

Request:

```
{
  "name": "support-tickets",
  "match": { "to_pattern": "support@yourapp.com" },
  "forward_to": "http://helpdesk/api/tickets",
  "priority": 10
}
```

Response (201):

```
{
  "status": "created",
  "rule_id": "rule-001"
}
```

GET /api/v1/dmarc/report

Generate DMARC aggregate report.

Response (200):

```
{
  "period": "2026-07",
  "total_messages": 15420,
  "aligned": 15100,
  "failed_spf": 120,
  "failed_dkim": 200
}
```

GET /healthz

Liveness probe.

```
{"status": "UP", "service": "pranor-notify", "version": "1.0.0"}
```

Security

Standalone Mode

In standalone mode, Notify connects directly to a configured SMTP relay. No authentication required for API access.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem:

- 1. **JWT Auth** — validates Bearer tokens against Pranor Auth
- 2. **Rate Limiting** — per-client send rate throttling
- 3. **DMARC Enforcement** — incoming mail validated against SPF/DKIM/DMARC
- 4. **Suppression List** — automatic blocking of bounced/complained addresses
- 5. **OTel Tracing** — every send generates a trace span

Email Security

- **SPF alignment** — validates sender IP against domain's SPF record
- **DKIM signing** — signs outgoing emails with domain key
- **DMARC reporting** — generates and sends RUA aggregate reports
- **TLS encryption** — STARTTLS for all outbound SMTP connections

Observability

Prometheus Metrics

Metric	Type	Description
pranor_notify_sent_total	Counter	Emails sent (labeled by channel, status)
pranor_notify_bounces_total	Counter	Bounce events (labeled by type: hard/soft)
pranor_notify_suppressed_total	Counter	Suppressed sends (address on suppression list)
pranor_notify_delivery_latency_ms	Histogram	Time to SMTP acknowledgment
pranor_notify_templates_active	Gauge	Registered templates
pranor_notify_inbound_routed_total	Counter	Inbound emails routed

OpenTelemetry Tracing

Notify emits spans for:

- ``notify.send`` — email dispatch
- ``notify.template.render`` — template rendering
- ``notify.suppression.check`` — suppression list lookup
- ``notify.dmarc.validate`` — DMARC alignment check
- ``notify.inbound.route`` — inbound email routing

Logging

Structured JSON logs with fields: ``level``, ``timestamp``, ``trace_id``, ``message_id``, ``to``, ``template``, ``channel``, ``status``.

Enterprise Edition

Feature OSS EE
----- :---: :---:
Transactional email via SMTP ✓ ✓
Template DSL (variables, conditionals, loops) ✓ ✓
Bounce management & suppression ✓ ✓
DMARC/SPF/DKIM enforcement ✓ ✓
Inbound email routing ✓ ✓
RFC 8058 one-click unsubscribe ✓ ✓
Mailing list management ✓ ✓
SMS gateway (Twilio, multi-carrier) — ✓
WebPush / APNs push notifications — ✓
AI deliverability & send-time optimizer — ✓
Delivery analytics dashboard — ✓
Per-recipient event tracking — ✓

Operational Runbook

Emails not being delivered

1. Check SMTP relay connectivity (``PRANOR_NOTIFY_SMTP_HOST``)
2. Verify SMTP credentials are correct
3. Check suppression list — recipient may be suppressed
4. Review DMARC/SPF/DKIM alignment for the sending domain
5. Check ``pranor_notify_delivery_latency_ms`` for SMTP timeout issues

High bounce rate

1. Monitor ``pranor_notify_bounces_total`` metric by type
2. Hard bounces indicate invalid addresses — clean your list
3. Soft bounces (mailbox full) will auto-retry with backoff
4. Review suppression list growth: ``GET /api/v1/suppression``
5. Check domain reputation via external tools (Google Postmaster)

Inbound routing not matching

1. List rules: ``GET /api/v1/inbound/rules``
2. Verify rule patterns match incoming email headers
3. Check priority ordering — higher priority rules match first
4. Verify the ``forward_to`` webhook URL is reachable
5. Check inbound SMTP port is accessible (``PRANOR_NOTIFY_INBOUND_PORT``)

Template rendering errors

1. Verify template exists: ``GET /api/v1/templates/{name}``
2. Check variable names match the payload structure
3. Review DSL syntax for unclosed conditionals or loops
4. Test with minimal variables to isolate the issue

Pranor Pool — Database Connection Proxy

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/pool`

Default Port: 8097

License: AGPL-3.0 (OSS) / Enterprise License (EE with pgvector accelerator & multi-dialect)

Overview

Pranor Pool is an intelligent, observable database connection pool manager for the Pranor ecosystem. It provides read/write splitting, connection health validation, leak detection, query telemetry, prepared statement caching, pool saturation alerting, and multi-dialect support for PostgreSQL, MySQL, and SQLite.

Pranor Pool can run as:

- A **standalone binary** providing connection pooling for any PostgreSQL/MySQL application
- An **integrated module** within the Pranor ecosystem with OTel tracing, Console dashboards, and Lock-coordinated DDL migrations

Key Features

Feature	Description
Read/Write Split	Auto-routes SELECTs to replicas, writes to primary
Replica Weighting	Configurable traffic distribution across replicas
Transaction Pinning	All queries within a transaction pinned to primary
Replica Lag Awareness	Skip replicas exceeding configurable lag threshold
Pre-checkout Validation	Ping + validation query before handing connections to callers
Leak Detection	Age-based and activity-based detection with goroutine stack traces
Query Analytics	Per-query p50/p75/p90/p99 latency histograms
Slow Query Logger	Queries exceeding threshold logged with full context
Prepared Statement Cache	Per-connection cache with automatic invalidation on schema change
Saturation Alerting	Pool utilization and wait queue depth alerts to Console

Architecture



```

end

subgraph PoolCore ["■ Core Connection Routing and Health Engine"]
  RWRouter["Read/Write Query Router"]
  HealthCheck["Pre-Checkout Validation Engine"]
  LeakDetector["Connection Leak and Goroutine Stack Tracker"]
  StmtCache["Per-Connection Prepared Statement Cache"]
  VectorOffload["PostgreSQL pgvector Accelerator"]
end

subgraph DBClusters ["■ Heterogeneous Relational DB Tier"]
  PrimaryDB["Primary RDBMS"]
  ReplicaPool["Weighted Replica Pool"]
end

App --> PoolClient
PoolClient --> RWRouter
RWRouter --> HealthCheck
HealthCheck --> LeakDetector
LeakDetector --> StmtCache
StmtCache --> VectorOffload
VectorOffload --> PrimaryDB
VectorOffload --> ReplicaPool

```

Connection Checkout, Read/Write Split & Leak Detection Sequence Flow

```

sequenceDiagram
    autonumber
    participant App as Application Microservice
    participant Pool as Pranor Pool Manager
    participant Leak as Goroutine Leak Tracker
    participant Stmt as Prepared Statement Cache
    participant DB as Target RDBMS (Primary / Replica)

    App->>Pool: Checkout Connection (Query: "SELECT * FROM users WHERE id = $1")
    Pool->>Pool: Inspect SQL Query Type (Read Query -> Route to Replica Pool)
    Pool->>Leak: Register Goroutine Stack & Start 30s Max-Hold Timer
    Pool->>Stmt: Lookup Cached Prepared Statement ("stmt_users_by_id")
    Stmt-->>Pool: Prepared Statement Handle Ready
    Pool->>DB: Execute Query on Replica DB Instance
    DB-->>Pool: Query Result Set Returned (p99 latency: 1.2ms)
    Pool->>Leak: Cancel Max-Hold Leak Timer & Return Connection to Pool
    Pool-->>App: Connection Released & Stats Updated

```

Ecosystem Cross-Module Integration

Pranor Pool provides intelligent database proxying across the Pranor ecosystem:

- **Pranor Lock:** Coordinates zero-downtime online DDL schema migrations, holding exclusive fencing token leases during migrations.
- **Pranor Trace:** Annotates SQL queries with OpenTelemetry spans, recording query normalization histograms and slow query stack traces.
- **Pranor Vault:** Connects seamlessly to PostgreSQL `pgvector` instances, managing connection pools for S3 vector metadata storage.
- **Pranor Console:** Displays real-time database connection saturation heatmaps, active wait-queue depth, and 1-click connection leak reclaims.

Installation & Deployment

Binary

```
cd pranor/pool
go build -o pranor-pool .
./pranor-pool --port 8097
```

Docker

```
docker run -p 8097:8097 ghcr.io/vyuvaraj/pranor-pool:latest
```

With OTel and Console

```
docker run -p 8097:8097 \
-e PRANOR_POOL_OTEL_ENDPOINT=http://pranor-trace:8090 \
-e PRANOR_POOL_PRANOR_CONSOLE_URL=http://pranor-console:8083 \
ghcr.io/vyuvaraj/pranor-pool:latest
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Pool integrates automatically with Lock (DDL coordination), Trace (query spans), and Console (saturation dashboards).

Configuration

Environment Variables

Variable	Default	Description
PRANOR_POOL_PORT	8097	HTTP listener port
PRANOR_POOL_OTEL_ENDPOINT		OpenTelemetry collector URL
PRANOR_POOL_PRANOR_CONSOLE_URL		Pranor Console URL for saturation alerts

| `PRANOR\_POOL\_DEFAULT\_MAX\_CONN` | `25` | Default max connections per pool |
 | `PRANOR\_POOL\_LEAK\_CHECK\_INTERVAL` | `30s` | How often to run leak detection sweep |

YAML Config (`pool.yaml`)

```
port: "8097"
otel_endpoint: "http://pranor-trace:8090"
console_url: "http://pranor-console:8083"
default_max_connections: 25
leak_check_interval: "30s"
slow_query_threshold_ms: 100
```

CLI Flags

Flag	Default	Description
`--port` `8097` HTTP listen port		

API Reference

Base URL: `http://localhost:8097`

POST /api/v1/pools

Create a connection pool.

Request:

```
{
  "name": "orders-db",
  "primary": "postgres://user:pass@primary:5432/orders",
  "replicas": [
    { "dsn": "postgres://user:pass@replica1:5432/orders", "weight": 70 },
    { "dsn": "postgres://user:pass@replica2:5432/orders", "weight": 30 }
  ],
  "max_connections": 50,
  "min_idle": 5,
  "validation_query": "SELECT 1",
  "max_checkout_duration": "30s",
  "slow_query_threshold_ms": 100
}
```

Response (201):

```
{
  "status": "created",
  "name": "orders-db",
```

```
"max_connections": 50
}
```

GET /api/v1/pools/{name}/stats

Pool stats — utilization, wait queue, active connections.

Response (200):

```
{
  "name": "orders-db",
  "total": 50,
  "active": 38,
  "idle": 12,
  "wait_queue": 2,
  "utilization_pct": 76
}
```

GET /api/v1/pools/{name}/leaks

List detected connection leaks.

Response (200):

```
{
  "leaks": [
    {
      "conn_id": "conn-42",
      "held_since": "2026-08-01T10:00:00Z",
      "duration_s": 45,
      "goroutine": "main.go:84",
      "stack_trace": "goroutine 42 [running]:\nmain.handleOrder(...)"
    }
  ]
}
```

POST /api/v1/pools/{name}/reclaim

Force-reclaim all leaked connections.

Response (200):

```
{
  "status": "reclaimed",
  "reclaimed_count": 3
}
```

GET /api/v1/pools/{name}/query-stats

Per-query latency histograms.

Response (200):

```
{
  "queries": [
    {
      "signature": "SELECT * FROM orders WHERE id = ?",
      "p50_ms": 3,
      "p75_ms": 8,
      "p90_ms": 22,
      "p99_ms": 45,
      "count": 10234
    }
  ]
}
```

GET /api/v1/pools/{name}/slow-queries

Recent slow queries.

Response (200):

```
{
  "queries": [
    {
      "query": "SELECT * FROM orders JOIN items ON ...",
      "duration_ms": 340,
      "timestamp": "2026-08-01T10:01:30Z",
      "caller": "handlers.go:156"
    }
  ]
}
```

GET /healthz

Liveness probe.

```
{"status": "UP", "service": "pranor-pool", "version": "1.0.0"}
```

Security

Standalone Mode

In standalone mode, Pool provides unauthenticated connection pooling. DSN credentials are stored in memory only.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem:

- 1. **JWT Auth** — validates Bearer tokens for pool management API
- 2. **Tenant Isolation** — pools scoped per tenant namespace
- 3. **OTel Tracing** — every query generates a trace span
- 4. **Credential Injection** — DSN passwords can be sourced from Pranor Secret

Connection Security

- **TLS to database** — supports `sslmode=require` in PostgreSQL DSNs
- **Credential rotation** — integrates with Pranor Secret for dynamic password rotation
- **No credential exposure** — DSN passwords never exposed in API responses

Observability

Prometheus Metrics

Metric	Type	Description
pranor_pool_connections_active	Gauge	Currently checked-out connections
pranor_pool_connections_idle	Gauge	Idle connections in pool
pranor_pool_wait_queue_depth	Gauge	Callers waiting for a connection
pranor_pool_utilization_pct	Gauge	Pool utilization percentage
pranor_pool_query_duration_ms	Histogram	Query execution latency
pranor_pool_leaks_detected_total	Counter	Connection leaks detected
pranor_pool_stmt_cache_hits_total	Counter	Prepared statement cache hits
pranor_pool_slow_queries_total	Counter	Slow queries logged

OpenTelemetry Tracing

Pool emits spans for:

- `pool.checkout` — connection checkout with routing decision
- `pool.query` — SQL query execution
- `pool.leak.detect` — leak detection event
- `pool.health.validate` — connection validation

Logging

Structured JSON logs with fields: `level`, `timestamp`, `trace_id`, `pool`, `query_signature`, `duration_ms`, `connection_id`.

Enterprise Edition

Feature OSS EE
----- :--- :---
Connection pooling (PostgreSQL, MySQL, SQLite) ✓ ✓
Read/write split routing ✓ ✓
Replica weighting ✓ ✓
Pre-checkout validation ✓ ✓
Leak detection ✓ ✓
Query analytics ✓ ✓
Slow query logger ✓ ✓
Prepared statement cache ✓ ✓
Saturation alerting ✓ ✓
PostgreSQL pgvector accelerator — ✓
Multi-dialect federation (cross-DB routing) — ✓
AI query optimization advisor — ✓
Connection pool sharding — ✓

Operational Runbook

Pool saturation (high utilization)

1. Check ``/api/v1/pools/{name}/stats`` for utilization percentage
2. Monitor ``pranor_pool_wait_queue_depth`` — if growing, pool is undersized
3. Increase ``max_connections`` in pool configuration
4. Check for connection leaks: ``GET /api/v1/pools/{name}/leaks``
5. Force reclaim leaks: ``POST /api/v1/pools/{name}/reclaim``

Connection leaks accumulating

1. Monitor ``pranor_pool_leaks_detected_total`` metric
2. Review leak stack traces: ``GET /api/v1/pools/{name}/leaks``
3. Identify code paths that checkout but don't release connections
4. Reduce ``max_checkout_duration`` to catch leaks sooner
5. Ensure all ``defer conn.Close()`` patterns are correct in application code

Replica lag causing stale reads

1. Check replica lag via database metrics
2. Configure lag threshold in pool — lagging replicas auto-excluded
3. Monitor how many queries fall back to primary due to lag

4. Consider adding more replicas or optimizing replication

Slow queries increasing

1. Review `/api/v1/pools/{name}/slow-queries`` for patterns
2. Check ``pranor_pool_query_duration_ms`` histogram for p99 growth
3. Use query normalization to identify expensive query signatures
4. Work with DBA to add indexes or optimize queries
5. Consider prepared statement cache to reduce parse overhead

Pranor Pulse — Async Event Broker & Message Queue

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/pulse`

Default Ports: 8082 (HTTP), 61613 (STOMP)

License: AGPL-3.0 (OSS) / Enterprise License (EE with Raft, MirrorMaker, KMS Encryption)

Overview

Pranor Pulse is a multi-protocol message broker and event streaming platform that supports STOMP, Kafka wire protocol, and MQTT v5 simultaneously. It provides durable WAL-based persistence, WASM-powered message transforms, dead letter queues with intelligent triage, consumer groups, partitioned topics, priority queues, delayed/scheduled messages, schema validation, tiered cold storage offloading, and browser-native OPFS queue support.

Pranor Pulse can run as:

- A **standalone binary** with zero external dependencies (WAL file-backed)
- An **integrated module** within the Pranor ecosystem with mTLS, RBAC, OTel tracing, and Console visibility
- A **Kafka-compatible broker** accepting native Kafka producer/consumer clients
- A **browser-embedded queue** via OPFS for offline-first PWAs

Table of Contents

- [Key Features](#key-features)
- [Architecture](#architecture)
- [Installation & Deployment](#installation--deployment)
- [Configuration](#configuration)
- [API Reference](#api-reference)
- [Messaging Semantics](#messaging-semantics)
- [Protocol Support](#protocol-support)
- [Storage & Durability](#storage--durability)
- [Security](#security)
- [Observability](#observability)
- [Client Libraries & SDKs](#client-libraries--sdks)
- [Enterprise Edition](#enterprise-edition)
- [Operational Runbook](#operational-runbook)

Key Features

Feature	Description
Multi-Protocol	STOMP 1.2, Kafka wire protocol, MQTT v5 — all on a single broker.

| **WAL Persistence** | Write-ahead log ensures zero message loss across restarts. |

| **WASM Transforms** | Per-topic WebAssembly transform pipelines for filtering, enrichment, or routing. |

| **Dead Letter Queues** | Automatic DLQ routing on transform failures with triage and requeue APIs. |

| **Consumer Groups** | Round-robin message dispatch across group members with rebalancing. |

| **Partitioned Topics** | FNV-1a key-based partitioning with partition-level subscribers. |

| **Priority Queues** | Priority-ordered message delivery — higher priority messages dispatched first. |

| **Delayed Messages** | Schedule message delivery N milliseconds in the future via TimeWheel. |

| **Message Deduplication** | Idempotent message-ID and producer-sequence-number dedup. |

| **Schema Registry** | Per-topic schema validation — reject non-conforming payloads at publish time. |

| **Tiered Storage** | Automatic offload of closed WAL segments to S3-compatible cold storage. |

| **Message TTL** | Per-message expiry — expired messages route to DLQ instead of delivery. |

| **Topic Compaction** | Key-based log compaction retaining only the latest value per key. |

| **Wildcard Subscriptions** | MQTT-style wildcard topics (`sensors.\*`, `events.#`). |

| **Backpressure** | Queue capacity limits with configurable overflow behavior. |

| **Rate Limiting** | Token-bucket publish rate limiting per broker. |

| **WebSocket Subscriptions** | Real-time browser consumption via WebSocket upgrade. |

| **SSE Subscriptions** | Server-Sent Events stream for lightweight real-time consumption. |

| **Offset Management** | Consumer group offset commit/fetch with replay-from-offset support. |

| **Time-Based Seek** | Seek to a timestamp offset for event replay. |

| **CDC (Change Data Capture)** | Database change event capture and publishing. |

| **OPFS Browser Queue** | Offline-first browser queue using Origin Private File System. |

| **Batch Publish** | Multi-message atomic publish in a single request. |

| **DLQ AI Triage** | Intelligent DLQ classification and suggested remediation. |

Architecture

```
graph TD
  subgraph Adapters ["Multi-Protocol Wire Interface"]
    STOMP["STOMP 1.2 Listener :61613"]
    Kafka["Kafka Wire Decoder :9092"]
    MQTT["MQTT v5 Broker :1883"]
    HTTP["HTTP REST API :8082"]
  end

  subgraph Core ["Core Event Streaming Broker"]
    Registry["Topic Registry and Wildcard Matcher"]
    Dedup["Idempotent Dedup Window"]
    Schema["Schema Compatibility Inspector"]
  end
```



```
WASM["WASM Transform Pipeline"]
Dispatch["Partition and Consumer Group Dispatcher"]
end

subgraph Storage ["WAL and Tiered Persistence Engine"]
WAL["Write-Ahead Log Engine"]
DLQ["Dead-Letter Queue Storage"]
ColdStore["S3 Cold Storage Offloader"]
end

subgraph Timers ["Delayed Delivery and Recovery"]
TimeWheel["TimeWheel Delayed Scheduler"]
OffsetStore["Consumer Group Offset Store"]
end

STOMP --> Registry
Kafka --> Registry
MQTT --> Registry
HTTP --> Registry
Registry --> Dedup
Dedup --> Schema
Schema --> WASM
WASM --> Dispatch
Dispatch --> WAL
WAL --> DLQ
WAL --> ColdStore
TimeWheel --> Dispatch
OffsetStore --> Dispatch
```

Event Streaming & Consumer Dispatch Sequence Flow

```
sequenceDiagram
    autonumber
    participant Producer as Event Producer (Kafka / STOMP / REST)
    participant Pulse as Pranor Pulse Broker Core
    participant Dedup as Sliding Dedup Window
    participant WASM as WASM Transform Sandbox
    participant WAL as Hardware AES-NI WAL Storage
    participant Consumer as Consumer Group Subscriber
    participant DLQ as Dead-Letter Queue (DLQ)

    Producer->>Pulse: Publish Event (Topic: "orders.created", Payload)
    Pulse->>Dedup: Verify Message ID & Producer Sequence Number
    Dedup-->>Pulse: Unique Payload (Passed)
    Pulse->>WASM: Execute Topic Transform Pipeline (WASM)
    alt Transform Succeeded
        WASM-->>Pulse: Enriched Event Payload
        Pulse->>WAL: Append Payload to Active WAL Segment
        WAL-->>Pulse: Log Offset Committed
        Pulse->>Consumer: Dispatch Event Payload via Consumer Group Round-Robin
        Consumer-->>Pulse: Acknowledge Event Commit (Offset Updated)
    else Transform Failed / Processing Expiry
        WASM-->>Pulse: Exception / Transform Error
        Pulse->>DLQ: Route Event Payload to Dead-Letter Queue
        DLQ-->>Pulse: DLQ Entry Logged & AI Triage Suggested
    end
end
```

Ecosystem Cross-Module Integration

Pranor Pulse serves as the primary asynchronous message bus across the Pranor platform:

- **Pranor Flow:** Dispatches saga workflow execution steps, compensation triggers, and human-in-the-loop task events via Pulse topics.
- **Pranor Vault:** Receives closed WAL segments offloaded automatically to S3 object buckets for long-term cold archive retention.
- **Pranor Trace:** Propagates trace context headers across message boundaries, tracking event latency flamegraphs end-to-end.
- **Pranor Console:** Provides real-time event throughput dashboards, consumer group rebalance monitors, and 1-click DLQ message replay UI.
- **Pranor Gate:** Relays event streams to web clients via WebSocket upgrader and SSE stream passthrough.

Installation & Deployment

Binary

```
cd pranor/pulse
go build -o pranor-pulse .
./pranor-pulse
```

Docker

```
docker run -p 8082:8082 -p 61613:61613 ghcr.io/vyuvaraj/pranor-pulse:latest
```

Docker Compose

```
services:
pulse:
image: ghcr.io/vyuvaraj/pranor-pulse:latest
ports:
- "8082:8082"
- "61613:61613"
environment:
- PRANOR_PULSE_WAL_PATH=/data/queue.wal
volumes:
- pulse-data:/data
volumes:
pulse-data:
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Pulse integrates automatically with Auth (JWT/mTLS), Trace (OTel spans), and Console (dashboard visibility). Multi-tenant topic namespacing is enforced automatically.

Configuration

Environment Variables

Variable	Default	Description
PRANOR_PULSE_WAL_PATH	queue.wal	Path to write-ahead log file
PRANOR_PULSE_PUBLISH_RATE	100	Token bucket publish rate (messages/sec)
PRANOR_PULSE_PUBLISH_CAPACITY	100	Token bucket burst capacity
PRANOR_PULSE_BACKPRESSURE_LIMIT	1000	Max messages in per-topic queue before backpressure
PRANOR_PULSE_S3_ENDPOINT		S3 endpoint for cold storage offloading
PRANOR_PULSE_S3_BUCKET		S3 bucket for WAL segment offload
PRANOR_PULSE_S3_TOKEN		S3 auth token for offloader

`PRANOR\_JWT\_SECRET`	—	JWT signing key for token auth
`PRANOR\_OTLP\_ENDPOINT`	—	OpenTelemetry collector URL
`TLS\_CERT\_FILE`	—	Path to TLS certificate
`TLS\_KEY\_FILE`	—	Path to TLS private key

STOMP Credentials

Default credentials (configured in code, overridable via ecosystem auth):

- Username: `admin`
- Password: `secret`

HTTP API Auth Token

Default: `secret-token` (standalone mode). In ecosystem mode, full JWT/mTLS auth chain is used.

API Reference

Base URL: `http://localhost:8082`
API Version: `/api/v1/` (recommended) or `/api/` (legacy)

GET /healthz

Liveness probe.

```
{ "status": "UP", "service": "pranor", "version": "1.0.0" }
```

GET /readyz

Readiness probe.

POST /api/v1/publish

Publish a message to a topic.

Request:

```
{
  "topic": "orders.created",
  "payload": "{ \"order_id\": \"abc-123\", \"amount\": 99.99 }",
  "key": "abc-123",
  "priority": 5,
  "delay_ms": 0,
  "message_id": "msg-unique-001",
  "ttl_ms": 60000
}
```

Field	Type	Required	Description
-----	-----	-----	-----

| `topic` | string | ✓ | Destination topic |

| `payload` | string | ✓ | Message content (JSON string) |

| `key` | string | | Partition key (FNV-1a hash for partition assignment) |

| `priority` | int | | Higher = dispatched first (default: 0) |

| `delay\_ms` | int | | Delay delivery by N milliseconds |

| `message\_id` | string | | Unique ID for deduplication |

| `ttl\_ms` | int | | Message expires after N ms (routes to DLQ) |

| `producer\_id` | string | | Producer identity for sequence dedup |

| `sequence\_number` | int | | Monotonic sequence for producer dedup |

Response (200):

```
{
  "status": "success",
  "topic": "orders.created",
  "processed_payload": "{\"order_id\":\"abc-123\",\"amount\":99.99}"
}
```

Backpressure Response (503):

```
{
  "error": "queue capacity exceeded: backpressure active",
  "code": "ERR_BACKPRESSURE"
}
```

POST /api/v1/publish/batch

Publish multiple messages atomically.

Request:

```
{
  "messages": [
    {"topic": "events.user", "payload": "{\"action\":\"login\"}"},
    {"topic": "events.user", "payload": "{\"action\":\"page_view\"}"},
  ]
}
```

GET /api/v1/topics

List all topics with metadata.

Response (200):

```
{
  "topics": [
    {
```

```
"name": "orders.created",
"subscribers": 3,
"partitions": 3,
"has_transform": true,
"dlq_topic": "orders.created.dlq"
},
"count": 1
}
```

POST /api/v1/topics/{topic}/transform

Register a WASM transform for a topic.

Request: Raw `.wasm` binary as request body.

Response (200):

```
WASM transform registered for topic orders.created
```

To clear a transform, send an empty body.

POST /api/v1/topics/{topic}/dlq

Register a Dead Letter Queue for a topic.

Request:

```
{"dlq_topic": "orders.created.dlq"}
```

GET /api/v1/topics/{topic}/dlq

List DLQ messages for a topic.

Response (200):

```
{
  "messages": [
    {
      "message_id": "dlq-1234567890",
      "source_topic": "orders.created",
      "original_payload": "{\"bad\":\"data\"}",
      "failure_reason": "WASM transform error: invalid field",
      "timestamp": 1706000000,
      "retry_count": 1
    }
  ],
  "total": 1,
  "dlq_topic": "orders.created.dlq"
}
```

```
}

```

GET /api/v1/topics/{topic}/dlq/summary

AI-powered DLQ analysis with failure pattern clustering.

GET /api/v1/topics/{topic}/dlq/triage

Intelligent DLQ triage with remediation suggestions.

POST /api/v1/topics/{topic}/dlq/requeue

Requeue a DLQ message (optionally patched) back to its source topic.

Request:

```
{
  "message_id": "dlq-1234567890",
  "payload": "{\"fixed\":\"data\"}"
}
```

POST /api/v1/topics/{topic}/schema

Register a validation schema for a topic.

Request:

```
{
  "order_id": "string",
  "amount": "number",
  "status": "string"
}
```

Messages that fail schema validation are rejected at publish time.

GET /api/v1/topics/{topic}/anomalies

Detect anomalous message patterns (spike detection, schema drift).

GET /api/v1/subscribe/{topic}

Subscribe via Server-Sent Events for real-time message consumption.

Response (text/event-stream):

```
data: {"order_id":"abc-123","amount":99.99}

data: {"order_id":"def-456","amount":45.00}

```

GET /ws/subscribe/{topic}

Subscribe via WebSocket for real-time bidirectional message consumption.

GET /api/v1/tail?topic={topic}

Tail the latest N messages from a topic (useful for debugging).

Query Parameters:

Param	Default	Description
-----	-----	-----

| `topic` | — | Topic to tail |
| `n` | `10` | Number of recent messages |

GET /api/v1/stats

Broker statistics and metrics.

Response (200):

```
{
  "messages_published": 152340,
  "wasm_executions": 45000,
  "wasm_execution_errors": 12,
  "wasm_avg_duration_ns": 250000,
  "topics_count": 8,
  "wal_entries": 152340
}
```

GET /api/v1/stats/ws

WebSocket endpoint streaming real-time broker stats every second.

POST /api/v1/replay

Replay messages from a specific offset for a consumer group.

Request:

```
{
  "topic": "orders.created",
  "start_offset": 100,
  "group_name": "analytics-group"
}
```

Response (200):

```
{"replayed": 52}
```

POST /api/v1/replay/time

Seek to a timestamp-based offset.

Request:

```
{"topic": "orders.created", "timestamp": 1706000000000}
```

Response (200):

```
{"offset": 1523}
```

GET /api/v1/offsets

Get committed offsets for a consumer group.

POST /api/v1/offsets

Commit an offset for a consumer group.

Request:

```
{"group": "analytics", "topic": "orders.created", "offset": 1523}
```

GET /api/v1/consumers/lag

Get consumer group lag (difference between latest offset and committed offset).

POST /api/v1/topics/retention

Configure topic retention policy.

POST /api/v1/admin/offloader

Configure tiered storage offloader.

Request:

```
{
  "s3_endpoint": "http://vault:9000",
  "s3_bucket": "pulse-cold-storage",
  "s3_token": "auth-token"
}
```

GET /metrics

Prometheus-compatible metrics.

```
# HELP pranor_pulse_messages_published_total Total messages published
# TYPE pranor_pulse_messages_published_total counter
pranor_pulse_messages_published_total 152340

# HELP pranor_pulse_wasm_executions_total Total WASM transform executions
# TYPE pranor_pulse_wasm_executions_total counter
pranor_pulse_wasm_executions_total 45000
```

GET /api/v1/events/{topic}

Event sourcing API — list events with filtering.

POST /api/v1/sqlite/query

Query broker metadata via embedded SQLite interface.

Messaging Semantics

Publish/Subscribe (Fan-Out)

Every subscriber on a topic receives every message:

```
Producer → publish("events.user", msg)
■■■ Subscriber-A receives msg
■■■ Subscriber-B receives msg
■■■ Subscriber-C receives msg
```

Consumer Groups (Competing Consumers)

Messages are round-robin dispatched to one member per group:

```
Producer → publish("orders", msg1)
Group "processors":
■■■ Worker-1 receives msg1
■■■ Worker-2 receives msg2 (next message)
■■■ Worker-3 receives msg3 (next message)
```

Partitioned Topics

Key-based partitioning ensures ordering per key:

```
publish("orders", key="customer-A", msg1) → Partition 0
publish("orders", key="customer-A", msg2) → Partition 0 (same key = same partition)
publish("orders", key="customer-B", msg3) → Partition 2 (different key)
```

Priority Queues

Messages with higher priority are dispatched first regardless of arrival order:

```
publish("tasks", payload="low", priority=1)
publish("tasks", payload="high", priority=10)
publish("tasks", payload="medium", priority=5)
→ Consumer receives: "high", "medium", "low"
```

Delayed Messages

Schedule delivery N milliseconds in the future:

```
publish("reminders", payload="Check order status", delay_ms=300000)
→ Message delivered after 5 minutes
```

The TimeWheel implementation provides 10ms resolution with O(1) scheduling.

Message Deduplication

Two dedup mechanisms:

1. **Message-ID dedup:** Same `message\_id` within 5-minute window is dropped
2. **Producer-sequence dedup:** Per `producer\_id`, any `sequence\_number ≤ last\_seen` is dropped

Message TTL / Expiry

Messages with `ttl\_ms` expire and route to the DLQ instead of delivering to consumers:

```
publish("events", payload="time-sensitive", ttl_ms=5000)
→ If not consumed within 5s, routes to DLQ with reason "message TTL expired"
```

Topic Compaction

For compacted topics, only the latest message per key is retained:

```
publish("state", key="user-1", payload="v1")
publish("state", key="user-1", payload="v2")
publish("state", key="user-2", payload="v1")
→ Compacted state: {"user-1": "v2", "user-2": "v1"}
```

Wildcard Subscriptions

MQTT-style topic patterns:

- `\*` matches exactly one level: `sensors.\*` matches `sensors.temp` but not `sensors.temp.room1`
- `#` matches zero or more levels: `events.#` matches `events`, `events.user`, `events.user.login`

Dead Letter Queues

When a WASM transform fails, the original message routes to the registered DLQ topic with an envelope containing:

- Original payload
- Source topic
- Failure reason
- Message ID
- Retry count

Protocol Support

STOMP 1.2 (Port 61613)

Full STOMP 1.2 implementation with username/password authentication. Compatible with any STOMP client (ActiveMQ clients, Spring Messaging, etc.).

```
import stomp
conn = stomp.Connection([('localhost', 61613)])
conn.connect('admin', 'secret', wait=True)
```

```
conn.subscribe('/topic/orders', id=1)
conn.send('/topic/orders', '{"order": "123"}')
```

Kafka Wire Protocol (Port 9092)

Native Kafka producer/consumer compatibility. Existing Kafka applications can point to Pulse without code changes.

MQTT v5 (Port 1883)

Full MQTT v5 support for IoT workloads — QoS levels, retained messages, topic aliases, and shared subscriptions.

HTTP REST API (Port 8082)

JSON-based publish/subscribe with SSE and WebSocket real-time delivery.

OPFS Browser Queue

Client-side JavaScript SDK using Origin Private File System for offline message queuing with automatic sync on reconnection.

Storage & Durability

Write-Ahead Log (WAL)

Every published message is appended to the WAL before acknowledgment. The WAL provides:

- Crash recovery — replays unprocessed messages on restart
- Segment rotation — closed segments can be offloaded to cold storage
- Sequential I/O — optimized for throughput

Tiered Storage Offloading

Configure S3-compatible cold storage for WAL segment archival:

```
export PRANOR_PULSE_S3_ENDPOINT=http://vault:9000
export PRANOR_PULSE_S3_BUCKET=pulse-archive
export PRANOR_PULSE_S3_TOKEN=auth-token
```

When a WAL segment rotates, it's automatically uploaded to the configured S3 bucket.

Offset Persistence

Consumer group offsets are stored in-memory with WAL backing. Consumers can:

- Commit offsets explicitly via API
- Replay from any historical offset
- Seek to a timestamp-based position

Security

Standalone Mode (API Token)

HTTP endpoints require:

```
Authorization: Bearer secret-token
```

STOMP connections authenticate with username/password.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem, the full middleware chain activates:

1. **OTel Tracing** — every request gets a span
2. **Rate Limiting** — per-client request throttling
3. **CORS** — cross-origin handling
4. **Max Body Size** — 10MB limit
5. **JWT Auth** — validates Bearer tokens
6. **Tenant Isolation** — topic namespacing (``tenant:topic``)

Multi-Tenant Isolation

Topics are automatically namespaced with tenant ID. Tenant A cannot see or publish to Tenant B's topics:

```
Tenant "acme" publishes to "orders" → stored as "acme:orders"
Tenant "acme" listing topics → only sees topics prefixed "acme:"
```

TLS Encryption

Enable TLS on both STOMP and HTTP listeners:

```
export TLS_CERT_FILE=/certs/pulse.crt
export TLS_KEY_FILE=/certs/pulse.key

---
```

Observability

Prometheus Metrics

Metric	Type	Description
pranor_pulse_messages_published_total	Counter	Total messages published
pranor_pulse_wasm_executions_total	Counter	Total WASM transform runs
pranor_pulse_wasm_errors_total	Counter	WASM execution failures
pranor_pulse_wasm_duration_ns	Histogram	WASM transform latency
pranor_pulse_topics_count	Gauge	Active topic count
pranor_pulse_subscribers_count	Gauge	Connected subscriber count
pranor_pulse_dlq_messages_total	Counter	Messages routed to DLQ

OpenTelemetry Tracing

Every publish operation generates an OTel span with:

- ``messaging.system``: ``pranor-pulse``

- ``messaging.destination``: topic name
- ``messaging.payload_len``: payload size
- Child spans for WASM transforms and DLQ routing

Real-time Stats WebSocket

Connect to ``/api/v1/stats/ws`` for streaming broker stats (1-second updates).

Embedded Web UI

Access ``/ui/`` on the HTTP port for a management dashboard showing topics, subscribers, DLQ state, and real-time throughput graphs.

Grafana Dashboard

Import the bundled ``grafana_dashboard.json`` for a pre-built Pulse monitoring dashboard.

Client Libraries & SDKs

Go

```
import "github.com/vyuvaraj/pranor/pulse/sdks/go"

client := pulse.NewClient("http://localhost:8082", "secret-token")

// Publish
err := client.Publish("orders.created", `{"order_id":"abc"}`, pulse.WithPriority(5))

// Subscribe
ch, err := client.Subscribe("orders.created")
for msg := range ch {
    fmt.Println("Received:", msg)
}
```

Python (STOMP)

```
import stomp

class MyListener(stomp.ConnectionListener):
    def on_message(self, frame):
        print(f"Received: {frame.body}")

conn = stomp.Connection([('localhost', 61613)])
conn.set_listener('', MyListener())
conn.connect('admin', 'secret', wait=True)
conn.subscribe('/topic/orders.created', id=1, ack='auto')
conn.send('/topic/orders.created', '{"order_id":"abc-123"}')
```

TypeScript (WebSocket)

```
const ws = new WebSocket('ws://localhost:8082/ws/subscribe/orders.created');
ws.onmessage = (event) => {
  const order = JSON.parse(event.data);
  console.log('New order:', order);
};
```

cURL

```
# Publish
curl -X POST http://localhost:8082/api/v1/publish \
-H "Authorization: Bearer secret-token" \
-H "Content-Type: application/json" \
-d '{"topic":"orders.created","payload":{"order_id":"abc"}}'

# List topics
curl http://localhost:8082/api/v1/topics \
-H "Authorization: Bearer secret-token"

# Register WASM transform
curl -X POST http://localhost:8082/api/v1/topics/orders.created/transform \
-H "Authorization: Bearer secret-token" \
--data-binary @enrich.wasm

# Replay from offset
curl -X POST http://localhost:8082/api/v1/replay \
-H "Authorization: Bearer secret-token" \
-d '{"topic":"orders.created","start_offset":0,"group_name":"replay-group}"'
```

Pranor CLI

```
pranor pulse publish --topic orders.created --payload '{"id":"abc"}'
pranor pulse subscribe --topic orders.created
pranor pulse topics list
pranor pulse dlq list --topic orders.created
pranor pulse dlq requeue --topic orders.created --message-id dlq-123
pranor pulse replay --topic orders.created --offset 0 --group analytics
```

Enterprise Edition

| Feature | OSS | EE |

|-----|:---|:---|

| STOMP / Kafka / MQTT protocols | ✓ | ✓ |

| WAL persistence & recovery | ✓ | ✓ |

| WASM transforms | ✓ | ✓ |

| Dead letter queues | ✓ | ✓ |

| Consumer groups & partitions | ✓ | ✓ |

| Priority queues & delayed messages | ✓ | ✓ |

| Message deduplication | ✓ | ✓ |

| Schema validation | ✓ | ✓ |

| Tiered S3 storage offload | ✓ | ✓ |

| WebSocket & SSE subscriptions | ✓ | ✓ |

| Topic compaction | ✓ | ✓ |

| OPFS browser queue | ✓ | ✓ |

| Raft consensus replication | — | ✓ |

| Multi-region MirrorMaker sync | — | ✓ |

| Hardware KMS/HSM payload encryption | — | ✓ |

| Schema registry breaking change guard | — | ✓ |

| Federated cross-cluster topic routing | — | ✓ |

| Advanced DLQ AI triage & auto-remediation | — | ✓ |

Operational Runbook

Messages not being delivered

1. Check ``/api/v1/topics`` to confirm the topic exists and has subscribers
2. Verify the publisher is authenticated and targeting the correct tenant namespace
3. Check for backpressure — if queue is full, publishes return 503
4. Review WASM transform logs — transform failures route to DLQ silently
5. Check dedup — same ``message_id`` within 5 minutes is dropped

DLQ filling up

1. Check ``/api/v1/topics/{topic}/dlq/summary`` for failure pattern clusters
2. Review the WASM transform for bugs — most DLQ entries come from transform errors
3. Use ``/api/v1/topics/{topic}/dlq/triage`` for AI-suggested remediation
4. Fix the transform, then requeue messages via ``/api/v1/topics/{topic}/dlq/requeue``

High publish latency

1. Check ``pranor_pulse_wasm_duration_ns`` — slow transforms add latency
2. Review backpressure limit — increase ``PRANOR_PULSE_BACKPRESSURE_LIMIT`` if queue is healthy
3. Check WAL disk I/O — WAL append is synchronous
4. Verify S3 offloader isn't blocking rotation (network issues to cold storage)

Consumer group rebalancing

1. Check subscriber count on the topic — new/removed consumers trigger rebalance
2. Verify consumer heartbeats are active
3. Review offset commits — stale offsets cause replay on rejoin

WAL recovery after crash

On restart, Pulse automatically:

1. Opens the WAL file
2. Recovers all non-expired entries
3. Re-publishes them through the broker engine
4. Resumes normal operation

No manual intervention required.

Tiered storage not offloading

1. Verify S3 endpoint connectivity: ``curl $PRANOR_PULSE_S3_ENDPOINT/healthz``
2. Check S3 credentials and bucket existence
3. WAL segments only offload on rotation — ensure enough write volume to trigger rotation
4. Check broker logs for offloader errors

Versioning & Compatibility

- HTTP API is versioned at ``/api/v1/``
- Legacy ``/api/`` paths continue to work
- STOMP protocol follows STOMP 1.2 specification
- Kafka wire protocol maintains compatibility with Kafka 2.x+ clients
- MQTT follows MQTT v5.0 specification
- WAL format is forward-compatible within major versions

Pranor Secret — Secret & Credential Management

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/secret`

Default Port: 8091

License: AGPL-3.0 (OSS) / Enterprise License (EE with HSM & Multi-Cloud KMS)

Overview

Pranor Secret is the centralized secrets, credentials, and configuration protection engine for the Pranor ecosystem. It provides tenant-isolated secret storage encrypted at rest using AES-256-GCM, Shamir secret splitting, dynamic injection into services, automatic rotation policies, and leak detection scanning.

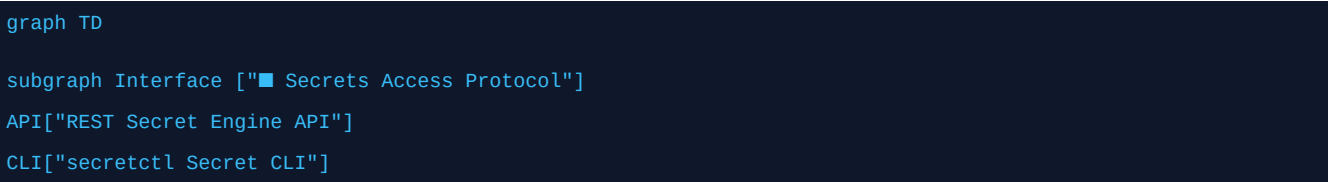
Pranor Secret can run as:

- A **standalone binary** with local encrypted file storage and a master key
 - An **integrated module** within the Pranor ecosystem with Pranor Core middleware, multi-tenant isolation, HSM integration, and dynamic rotation
-

Key Features

Feature	Description
AES-256-GCM Encryption	All secrets encrypted at rest with envelope encryption
Tenant Isolation	Secrets organized per tenant with namespace enforcement
Shamir Splitting	Master key split across multiple key holders (2-of-3 quorum)
Dynamic Injection	Services retrieve secrets at runtime via API
Automatic Rotation	Configurable TTL-based rotation with zero-downtime rollover
Leak Detection	Scan codebases and logs for accidentally exposed secrets
KMS Federation	Multi-cloud KMS sync (AWS KMS, GCP KMS, Azure Key Vault)
FIPS 140-3 HSM	Hardware security module adapter for key operations
Encrypted File Store	Local persistence in encrypted `secrets.enc` file
Vault Backend	Optional Pranor Vault encrypted key store

Architecture



```

end

subgraph Core ["■ Cryptographic Key and Secret Engine"]
  AESGCM["AES-256-GCM Envelope Encryption Engine"]
  FIPS140["FIPS 140-3 Cryptographic HSM Adapter"]
  KMSFed["Multi-Cloud KMS Federation Sync"]
  MPC["Zero-Knowledge MPC Key Splitter"]
end

subgraph Persistence ["■ Encrypted Secret Storage"]
  FileStore["Encrypted Local Store"]
  VaultStore["Pranor Vault Encrypted Key Store"]
end

API --> AESGCM
CLI --> AESGCM
AESGCM --> FIPS140
FIPS140 --> KMSFed
KMSFed --> MPC
MPC --> FileStore
MPC --> VaultStore

```

Cryptographic Secret Envelope & Key Unsealing Sequence Flow

```

sequenceDiagram
    autonumber
    participant App as Microservice / Gateway
    participant Secret as Pranor Secret Engine
    participant HSM as FIPS 140-3 Hardware HSM
    participant KMS as Multi-Cloud KMS Federation
    participant Store as Encrypted Secrets Store

    App->>Secret: GET /api/v1/secrets/database-password (X-Tenant-ID)
    Secret->>HSM: Unseal Envelope Master Key via FIPS 140-3 Module
    HSM->>KMS: Combine MPC Threshold Key Shares (2-of-3 quorum)
    KMS-->>Secret: Reconstructed Decryption Key
    Secret->>Store: Read Ciphertext Payload from secrets.enc
    Store-->>Secret: Encrypted Data Ciphertext + AES-GCM Nonce
    Secret->>Secret: Decrypt Payload in Memory-Isolated Buffer
    Secret-->>App: Plaintext Secret Value + Dynamic Rotation TTL

```

Ecosystem Cross-Module Integration

Pranor Secret provides master key management and secret protection across all Pranor modules:

- **Pranor Gate:** Dynamically provisions and auto-rotates TLS server certificates and client mTLS credentials without restarting proxy instances.
- **Pranor Auth:** Secures private RSA/ECDSA JWT signing keys, WebAuthn passkey seeds, and OIDC client secrets.
- **Pranor Vault:** Stores client-side envelope encryption keys and S3 cloud storage access credentials.
- **Pranor Console:** Renders the visual Secret Management Webview, unsealing vaults and inspecting rotation policies securely.

Installation & Deployment

Binary

```
cd pranor/secret
go build -o pranor-secret .
./pranor-secret --port 8091 --file secrets.enc
```

Docker

```
docker run -p 8091:8091 \
-e PRANOR_SECRET_MASTER_KEY="000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f" \
-v secret-data:/data \
ghcr.io/vyuvaraj/pranor-secret:latest
```

With Master Key

```
export PRANOR_SECRET_MASTER_KEY="000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f"
./pranor-secret --port 8091 --file /data/secrets.enc
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Secret integrates automatically with Auth (JWT key storage), Gate (TLS cert rotation), Console (secret management UI), and Core middleware (tenant isolation).

Configuration

Environment Variables

Variable	Default	Description
PRANOR_SECRET_PORT	8091	HTTP listener port
PRANOR_SECRET_MASTER_KEY		32-byte hex-encoded master encryption key
PRANOR_SECRET_FILE	secrets.enc	Path to encrypted secrets file
PRANOR_SECRET_ROTATION_INTERVAL		Default rotation interval for secrets
PRANOR_SECRET_OTEL_ENDPOINT		OpenTelemetry collector URL

YAML Config (`secret.yaml`)

```
port: "8091"
master_key: "" # Set via env var for security
file: "/data/secrets.enc"
rotation_interval: "24h"
otel_endpoint: "http://pranor-trace:8090"
```

CLI Flags

Flag	Default	Description
<code>--port</code>	<code>8091</code>	HTTP listen port
<code>--file</code>	<code>secrets.enc</code>	Encrypted secrets file path

API Reference

Base URL: `http://localhost:8091`

POST `/api/v1/secrets`

Set or update a secret.

Headers:

- `X-Tenant-ID: tenant-a`
- `Authorization: Bearer`

Request:

```
{
  "key": "database-password",
  "value": "super-secret-passphrase"
}
```

Response (201):

```
{
  "key": "database-password",
  "status": "stored",
  "encrypted": true
}
```

GET `/api/v1/secrets/{key}`

Retrieve a secret value.

Headers:

- `X-Tenant-ID: tenant-a`

- `Authorization: Bearer `

Response (200):

```
{
  "key": "database-password",
  "value": "super-secret-passphrase"
}
```

GET /api/v1/secrets

List stored secret keys (values not exposed).

Response (200):

```
{
  "keys": ["database-password", "api-key-stripe", "jwt-signing-key"]
}
```

DELETE /api/v1/secrets/{key}

Delete a secret.

Response (200):

```
{
  "status": "deleted",
  "key": "database-password"
}
```

GET /healthz

Liveness probe.

```
{"status": "UP", "service": "pranor-secret", "version": "1.0.0"}
```

Security

Standalone Mode

Provide a 32-byte hex-encoded master key via `PRANOR\_SECRET\_MASTER\_KEY`. If unset, a temporary random key is generated at startup (secrets won't persist across restarts).

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem:

1. **OTel Tracing** — every secret access generates a span
2. **Rate Limiting** — per-client request throttling

- 3. **JWT Auth** — validates Bearer tokens against Pranor Auth
- 4. **Tenant Isolation** — secrets scoped per X-Tenant-ID header
- 5. **Audit Logging** — all read/write/delete operations logged

Encryption Details

- **Algorithm:** AES-256-GCM (Galois/Counter Mode)
- **Nonce:** Unique random nonce per encryption operation
- **Key derivation:** Master key used for envelope encryption
- **Memory safety:** Plaintext secrets held only in memory-isolated buffers, zeroed after use

Shamir Secret Splitting (EE)

Master key can be split into N shares with M-of-N threshold for unsealing:

- Default: 2-of-3 quorum required to reconstruct master key
- Key holders each possess one share
- No single point of compromise

Observability

Prometheus Metrics

Metric	Type	Description
pranor_secret_reads_total	Counter	Secret read operations
pranor_secret_writes_total	Counter	Secret write operations
pranor_secret_deletes_total	Counter	Secret delete operations
pranor_secret_rotations_total	Counter	Automatic rotation events
pranor_secret_keys_active	Gauge	Currently stored secrets
pranor_secret_decrypt_duration_ms	Histogram	Decryption latency

OpenTelemetry Tracing

Secret emits spans for:

- `secret.read` — secret retrieval (key name logged, value never logged)
- `secret.write` — secret storage
- `secret.delete` — secret deletion
- `secret.rotate` — rotation event

Logging

Structured JSON logs with fields: `level`, `timestamp`, `trace_id`, `tenant_id`, `key`, `action`. Secret values are **never** logged.

Enterprise Edition

Feature OSS EE
----- :--- :---
AES-256-GCM encrypted storage ✓ ✓
Tenant-isolated secrets ✓ ✓
REST API for CRUD ✓ ✓
File-backed persistence ✓ ✓
Graceful shutdown ✓ ✓
Shamir secret splitting (2-of-N quorum) — ✓
FIPS 140-3 HSM adapter — ✓
Multi-cloud KMS federation (AWS/GCP/Azure) — ✓
Automatic rotation with zero-downtime rollover — ✓
Leak detection scanner — ✓
Dynamic injection into running services — ✓
Pranor Vault encrypted backend — ✓

Operational Runbook

Cannot decrypt secrets after restart

1. Verify `PRANOR\_SECRET\_MASTER\_KEY` is set correctly (same key as when secrets were written)
2. If no master key was provided initially, secrets used a temporary key and are lost
3. Check file permissions on `secrets.enc`
4. Verify the secrets file isn't corrupted (check file size > 0)

Rotation failing

1. Check `pranor\_secret\_rotations\_total` metric for errors
2. Verify rotation interval configuration
3. Ensure services consuming rotated secrets are polling for updates
4. Check OTel spans for `secret.rotate` errors

High decryption latency

1. Monitor `pranor\_secret\_decrypt\_duration\_ms` histogram
2. If using HSM, check HSM connectivity and load
3. Consider caching decrypted values in-memory with short TTL
4. Review concurrent access patterns — may need connection pooling to HSM

Suspected secret leak

1. Enable leak detection scanner (EE feature)
2. Rotate compromised secrets immediately via API
3. Audit access logs for unauthorized reads (`pranor\_secret\_reads\_total`)

4. Review which services accessed the leaked secret via trace spans
5. Invalidate downstream tokens/credentials that used the leaked secret

Pranor Trace — Distributed Tracing & Continuous Profiling

Version: 1.0.0

Module Path: `github.com/vyuvaraj/pranor/trace`

Default Port: 8090

License: AGPL-3.0 (OSS) / Enterprise License (EE with AI Anomaly Detection & SIEM Streaming)

Overview

Pranor Trace is the distributed tracing and continuous profiling service for the Pranor ecosystem. It ingests OTLP-format traces, assembles waterfall hierarchies, provides SLO burn rate alerting, delivers eBPF-powered flamegraph profiling with automatic OTel correlation, critical path analysis, and anomaly detection.

- Pranor Trace can run as:
- A **standalone binary** accepting OTLP/HTTP traces with in-memory storage
 - An **integrated module** within the Pranor ecosystem with eBPF profiling, Console integration, and SIEM streaming
-

Key Features

Feature	Description
OTLP Ingestion	Standard `/v1/traces` endpoint compatible with all OpenTelemetry SDKs
Span Reassembly	Groups spans by trace ID, links parent-child relationships
Waterfall UI	Full span waterfall with nested children and duration bars
SLO Burn Rate	Dual-window burn rate alerting with error budget tracking
eBPF Flamegraphs	Kernel-level CPU/memory profiling without code instrumentation
Trace-to-Flamegraph	Correlate slow spans to flamegraph profiles
Critical Path Analysis	Identify the longest-latency path across distributed traces
Prometheus Exemplars	OpenMetrics with trace exemplar links in histograms
Dependency Map	Auto-discovered service call graph from trace data
Anomaly Detection	AI-powered latency anomaly baseline comparison

Architecture

```
graph TD
    subgraph Ingestion ["Telemetry Ingestion Layer"]
        OTLP["OTLP / gRPC / HTTP Collector"]
        eBPFProf["Kernel eBPF Continuous Profiler"]
    end
```

```

subgraph Processing ["■ Span Reassembly and AI Engine"]
  Reassembly["Span Grouping and Trace ID Linker"]
  CriticalPath["Critical Path Evaluator"]
  AIAutoTune["Autonomous AI Anomaly Auto-Tuner"]
  SLOEngine["SLO Burn Rate Alerting Engine"]
end

subgraph Storage ["■ In-Memory and SIEM Storage"]
  MemStore["In-Memory Evicting Trace Store"]
  SIEMStreamer["Encrypted SIEM Streamer"]
end

OTLP --> Reassembly
eBPFProf --> Reassembly
Reassembly --> CriticalPath
CriticalPath --> AIAutoTune
AIAutoTune --> SLOEngine
SLOEngine --> MemStore
MemStore --> SIEMStreamer

```

Telemetry Processing & Flamegraph Correlation Sequence Flow

```

sequenceDiagram
    autonumber
    participant SDK as Microservice OTLP SDK
    participant Trace as Pranor Trace Collector
    participant eBPF as Kernel eBPF Profiler
    participant AI as AI Anomaly Engine
    participant Console as Pranor Console UI

    SDK->>Trace: POST /v1/traces (Span Tree + TraceID: 0x9918)
    eBPF->>Trace: Push Kernel CPU Stack Samples
    Trace->>Trace: Group Spans by TraceID & Link Parent-Child Tree
    Trace->>AI: Evaluate Span Latency against Baseline
    alt Latency Anomaly Detected
    AI-->>Trace: Raise Burn Rate Alert & Identify Root-Cause Span
    Trace->>Console: Stream Correlated Flamegraph + Log Evidence
    else Standard Trace
    Trace-->>Console: Update Live Waterfall Graph & Dependency Map
    end
end

```

Ecosystem Cross-Module Integration

Pranor Trace serves as the central telemetry and observability hub across the Pranor ecosystem:

- **Pranor Gate:** Ingests W3C `traceparent` headers, attributing gateway latency and AI prompt token costs to backend trace spans.
- **Pranor Flow:** Captures individual workflow step execution spans, linking saga compensation steps to root trace IDs.
- **Pranor Console:** Renders live interactive CPU flamegraphs, distributed service dependency graphs, and SLO burn rate dashboards.
- **Pranor Notify:** Triggers incident notifications to PagerDuty or Slack when SLO burn rates exceed fast/slow window thresholds.

Installation & Deployment

Binary

```
cd pranor/trace
go build -o pranor-trace .
./pranor-trace --port 8090
```

Docker

```
docker run -p 8090:8090 ghcr.io/vyuvaraj/pranor-trace:latest
```

With eBPF Profiling

```
docker run -p 8090:8090 \
--privileged \
-e PRANOR_TRACE_EBPF_ENABLED=true \
-e PRANOR_TRACE_MAX_TRACES=50000 \
ghcr.io/vyuvaraj/pranor-trace:latest
```

As Part of Pranor Ecosystem

When running under the Pranor platform, Trace integrates automatically with all services via the `PRANOR\_OTLP\_ENDPOINT` env var. Console connects for waterfall rendering and flamegraph display.

Configuration

Environment Variables

Variable	Default	Description
----- ----- -----		
`PRANOR_TRACE_PORT`	`8090`	HTTP listener port
`PRANOR_TRACE_MAX_TRACES`	`10000`	Max traces in memory before eviction
`PRANOR_TRACE_EBPF_ENABLED`	`false`	Enable eBPF continuous profiling
`PRANOR_TRACE_OTEL_EXPORT`	—	Re-export spans to another OTLP collector
`PRANOR_TRACE_SLO_ALERT_WEBHOOK`	—	Webhook URL for SLO burn rate alerts

YAML Config (`trace.yaml`)

```
port: "8090"
max_traces: 50000
ebpf_enabled: true
otel_export: ""
slo_alert_webhook: "http://pranor-notify:8094/api/v1/send"
```

CLI Flags

Flag	Default	Description
----	-----	-----
<code>--port</code>	<code>8090</code>	HTTP listen port

API Reference

Base URL: `http://localhost:8090`

POST `/v1/traces`

OTLP/HTTP trace ingestion (standard OpenTelemetry endpoint).

Request: Standard OTLP ExportTraceServiceRequest (protobuf or JSON).

Response (200):

```
{}
```

GET `/api/v1/traces`

List recent traces.

Query parameters: `service`, `status`, `min_duration_ms`, `limit`

Response (200):

```
{
  "traces": [
    {
      "trace_id": "abc123def456",
      "root_service": "orders-api",
      "root_operation": "POST /orders",
      "duration_ms": 234,
      "span_count": 8,
      "status": "ok",
      "started_at": "2026-08-01T10:00:00Z"
    }
  ]
}
```

```
}
```

GET /api/v1/traces/{traceID}

Get full trace with span waterfall hierarchy.

Response (200):

```
{
  "trace_id": "abc123def456",
  "spans": [
    {
      "span_id": "span-001",
      "parent_span_id": null,
      "service": "orders-api",
      "operation": "POST /orders",
      "duration_ms": 234,
      "status": "ok",
      "children": [
        {
          "span_id": "span-002",
          "service": "payments-api",
          "operation": "charge",
          "duration_ms": 180
        }
      ]
    }
  ]
}
```

GET /api/v1/traces/{traceID}/critical-path

Critical path analysis for a trace.

Response (200):

```
{
  "trace_id": "abc123def456",
  "critical_path": [
    { "service": "orders-api", "operation": "POST /orders", "self_time_ms": 54 },
    { "service": "payments-api", "operation": "charge", "self_time_ms": 180 }
  ],
  "bottleneck": "payments-api"
}
```

POST /api/v1/slo

Define an SLO for a service.

Request:

```
{
  "service": "orders-api",
  "slo_name": "availability",
  "target_ratio": 0.999,
  "windows": [
    { "name": "fast", "duration": "1h", "burn_rate_threshold": 14.4 },
    { "name": "slow", "duration": "6h", "burn_rate_threshold": 6.0 }
  ]
}
```

Response (201):

```
{
  "status": "created",
  "slo_id": "slo-001"
}
```

GET /api/v1/slo/{service}/burn-rate

SLO burn rate for a service.

Response (200):

```
{
  "slo": "availability",
  "budget_remaining": 0.82,
  "burn_rate_1h": 2.1,
  "burn_rate_6h": 0.8,
  "alerting": false
}
```

GET /api/v1/flamegraph/{service}

Latest eBPF flamegraph for a service.

Response (200): SVG or JSON flamegraph data.

GET /healthz

Liveness probe.

```
{ "status": "UP", "service": "pranor-trace", "version": "1.0.0" }
```

Security

Standalone Mode

In standalone mode, Trace accepts OTLP spans without authentication. Suitable for development and internal networks.

Ecosystem Mode (Full Auth Stack)

When running within the Pranor ecosystem:

- 1. **JWT Auth** — management APIs require Bearer token
- 2. **OTel ingestion** — `/v1/traces`` can be optionally auth-gated
- 3. **Tenant Isolation** — traces scoped per tenant
- 4. **SIEM Streaming** — encrypted export to external SIEM systems
- 5. **Data Retention** — configurable max traces with oldest-first eviction

eBPF Security

eBPF profiling requires `--privileged`` Docker flag or `CAP_SYS_ADMIN`` + `CAP_BPF`` capabilities. In production, use a dedicated profiling sidecar with minimal permissions.

Observability

Prometheus Metrics

Metric	Type	Description
pranor_trace_spans_ingested_total`	Counter	Total spans received
pranor_trace_traces_stored`	Gauge	Traces currently in memory
pranor_trace_slo_burn_rate`	Gauge	Current burn rate per service/SLO
pranor_trace_slo_alerts_fired_total`	Counter	SLO alert triggers
pranor_trace_flamegraph_samples_total`	Counter	eBPF stack samples collected
pranor_trace_evictions_total`	Counter	Traces evicted from memory

OpenTelemetry Self-Telemetry

Trace emits its own spans for:

- `trace.ingest`` — span ingestion pipeline
- `trace.reassemble`` — trace ID grouping
- `trace.slo.evaluate`` — burn rate calculation
- `trace.flamegraph.correlate`` — span-to-flamegraph correlation

Logging

Structured JSON logs with fields: `level`, `timestamp`, `trace\_id`, `service`, `operation`, `duration\_ms`, `alert`.

Enterprise Edition

Feature	OSS	EE
----- :--- :---		
OTLP/HTTP trace ingestion	✓	✓
Span reassembly & waterfall	✓	✓
SLO burn rate alerting	✓	✓
Critical path analysis	✓	✓
Service dependency map	✓	✓
Prometheus exemplars	✓	✓
In-memory evicting store	✓	✓
eBPF flamegraph profiling	—	✓
AI anomaly detection auto-tuner	—	✓
Encrypted SIEM streaming	—	✓
Trace-to-flamegraph correlation	—	✓
Multi-cluster trace federation	—	✓

Operational Runbook

Traces being evicted too quickly

- 1. Check `pranor\_trace\_traces\_stored` gauge vs `PRANOR\_TRACE\_MAX\_TRACES`
- 2. Increase `PRANOR\_TRACE\_MAX\_TRACES` or add more memory
- 3. Consider exporting to external storage via `PRANOR\_TRACE\_OTEL\_EXPORT`
- 4. Review if unnecessary high-cardinality spans are being ingested

SLO alerts firing incorrectly

- 1. Check `pranor\_trace\_slo\_burn\_rate` metric for the service
- 2. Verify SLO definition — is the target ratio correct?
- 3. Review burn rate window configuration (fast: 1h, slow: 6h)
- 4. Check if a deployment or incident caused a legitimate spike
- 5. Adjust thresholds if alerting is too sensitive

eBPF profiling not producing data

- 1. Verify `PRANOR\_TRACE\_EBPF\_ENABLED=true`
- 2. Check container has `--privileged` or necessary capabilities

3. Verify kernel version supports BPF (Linux 4.15+)
4. Check `pranor\_trace\_flamegraph\_samples\_total` metric
5. Review logs for BPF program load errors

High span ingestion latency

1. Monitor span ingestion rate vs processing capacity
2. Check `pranor\_trace\_spans\_ingested\_total` rate
3. If store is full, eviction adds overhead — increase capacity
4. Consider sampling at the SDK level to reduce volume
5. Review if SIEM streaming is creating backpressure

Pranor Tunnel — Secure Dev Tunneling

Version: 0.1.0

Module Path: `github.com/vyuvaraj/pranor/tunnel`

Default Port: 8443

License: AGPL-3.0 (OSS) / Enterprise License (EE with WireGuard E2E & Custom Domains)

Overview

Pranor Tunnel is a secure, instant tunneling service for exposing local services to the internet during development and testing. One command creates a public URL that forwards requests to your local machine via WebSocket multiplexing — ideal for webhook testing, OAuth callbacks, mobile app dev, and sharing work in progress.

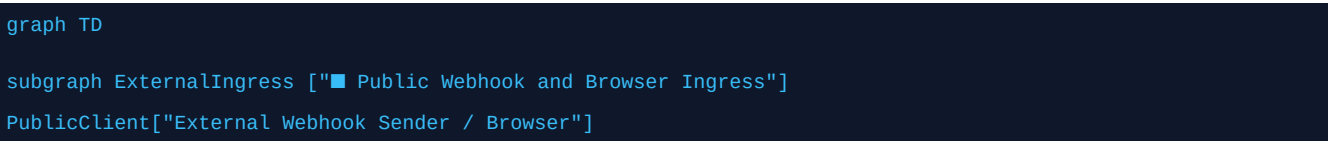
Pranor Tunnel can run as:

- A **server** (relay) accepting incoming public traffic and routing to connected clients
- A **client** (daemon) running on developer machines, connecting to the relay and forwarding to localhost

Key Features

Feature	Description
Subdomain Routing	Each tunnel gets a unique subdomain (e.g., `myapp.pranor.net`)
WebSocket Multiplexing	Binary-framed streams over a single WebSocket connection
Request Inspection	Ring-buffer captures all requests/responses for debugging
Request Replay	Replay any captured request with one API call
JWT Auth Gating	Require valid JWT to open tunnel connections
Shareable URLs	Time-limited shareable tunnel URLs with auto-expiry
Git Branch Auto-subdomain	Automatically derives subdomain from current Git branch
Multi-port Tunneling	Expose multiple local ports with a single config file
Custom Domains	Map custom domains to tunnels (DNS CNAME)
OTel Propagation	`traceparent` headers forwarded through the tunnel
Reconnection	Persistent reconnect with exponential backoff and jitter

Architecture



```

SubdomainRouter["Public Subdomain Ingress Router"]
end

subgraph TunnelServer ["■ Tunnel Multiplexer and Inspection Engine"]
WSMux["WebSocket Connection Multiplexer"]
Inspections["Ring-Buffer Request Capturer and Inspection"]
E2EEncryption["Zero-Trust WireGuard E2E Encryption"]
ReplayEngine["Request Replay Engine"]
end

subgraph LocalMachine ["■ Private Local Workload"]
TunnelClient["Pranor Tunnel Daemon CLI Client"]
LocalSvc["Local Microservice / Webhook Receiver"]
end

PublicClient --> SubdomainRouter
SubdomainRouter --> WSMux
WSMux --> Inspections
Inspections --> E2EEncryption
E2EEncryption --> ReplayEngine
ReplayEngine --> TunnelClient
TunnelClient --> LocalSvc

```

Public Webhook Proxying & Request Replay Sequence Flow

```

sequenceDiagram
    autonumber
    participant External as Stripe / GitHub Webhook Sender
    participant Server as Pranor Tunnel Server
    participant Buffer as Inspection Ring Buffer
    participant Client as Pranor Tunnel Local CLI
    participant Local as Local Host Service (localhost:3000)

    External->>Server: POST https://myapp.pranor.net/webhooks (Stripe Signature Header)
    Server->>Buffer: Store Request Headers & Body Payload in Ring Buffer
    Server->>Client: Forward Stream Payload over Multiplexed WebSocket
    Client->>Local: HTTP POST http://localhost:3000/webhooks
    Local-->>Client: 200 OK (Processed locally)
    Client-->>Server: Forward Response Frame over WebSocket
    Server-->>External: 200 OK (Proxy Complete)

    Note over External,Local: Developer triggers manual 1-Click Request Replay
    Client->>Server: POST /api/v1/tunnels/{id}/replay/{reqID}
    Server->>Local: Replay Captured Request to Local Host

```

Ecosystem Cross-Module Integration

Pranor Tunnel provides secure localhost exposure across the Pranor platform:

- **Pranor Gate:** Relays public HTTPS ingress routes into multiplexed WebSocket tunnels for dev preview environments.
- **Pranor Trace:** Generates `traceparent` OpenTelemetry headers, tracing requests from public webhooks through tunnels into local code.
- **Pranor Deploy:** Exposes ephemeral branch preview environments securely without public IP addresses.
- **Pranor Console:** Renders the visual Request Inspector UI, enabling 1-click webhook replays and live packet inspection.

Installation & Deployment

Server (Self-hosted Relay)

```
cd pranor/tunnel
go build -o pranor-tunnel .
./pranor-tunnel server --port 8443 --domain pranor.net
```

Docker (Server)

```
docker run -p 8443:8443 \
-e PRANOR_TUNNEL_DOMAIN=pranor.net \
-e PRANOR_TUNNEL_JWT_SECRET=my-secret \
ghcr.io/vyuvaraj/pranor-tunnel:latest server
```

Client (Local Machine)

```
# Install
go install github.com/vyuvaraj/pranor/tunnel@latest

# Expose local port 3000
pranor-tunnel client 3000 --relay ws://tunnel.pranor.net:8443/ws/connect --subdomain myapp
```

Multi-port Config File

```
# tunnel.yaml
relay: "ws://tunnel.pranor.net:8443/ws/connect"
token: "my-auth-token"
tunnels:
- port: "3000"
  subdomain: "frontend"
- port: "8080"
  subdomain: "api"
- port: "5432"
  subdomain: "db-admin"
```

```
pranor-tunnel client --config tunnel.yaml

---
```

Configuration

Server Environment Variables

Variable	Default	Description
PRANOR_TUNNEL_ADDR	:8443	Server listen address
PRANOR_TUNNEL_DOMAIN	localhost	Base domain for subdomains
PRANOR_TUNNEL_JWT_SECRET		JWT signing secret for auth gating
PRANOR_TUNNEL_MAX_RING_BUFFER	100	Max captured requests per tunnel
PRANOR_TUNNEL_OTEL_ENDPOINT		OpenTelemetry collector URL
PRANOR_TUNNEL_TLS_CERT		TLS certificate path
PRANOR_TUNNEL_TLS_KEY		TLS key path

Client Environment Variables

Variable	Default	Description
PRANOR_TUNNEL_RELAY	ws://localhost:8443/ws/connect	Relay WebSocket URL
PRANOR_TUNNEL_TOKEN		Authentication token

YAML Config (tunnel.yaml)

```
# Server config
addr: ":8443"
domain: "pranor.net"
jwt_secret: "my-secret"
max_ring_buffer: 100
tls_cert: "/certs/tunnel.crt"
tls_key: "/certs/tunnel.key"
otel_endpoint: "http://pranor-trace:8090"
```

CLI Flags (Server)

Flag	Default	Description
--port, -p	8443	Listen port
--domain, -d	localhost	Base domain for subdomains

CLI Flags (Client)

Flag	Default	Description
------	---------	-------------

```
|-----|-----|-----|
| `--relay`, `-r` | `ws://localhost:8443/ws/connect` | Relay WebSocket URL |
| `--subdomain`, `-s` | (auto-generated) | Requested subdomain |
| `--custom-domain`, `-c` | — | Custom domain mapping |
| `--token`, `-t` | — | Authentication token |
| `--inspect-port`, `-i` | `4040` | Local inspection web UI port |
| `--share-auth`, `-a` | — | Basic auth to protect public tunnel |
| `--config` | — | Path to YAML config file |
---
```

API Reference

Base URL: `http://localhost:8443`

POST /api/v1/tunnels

Create a new tunnel (server-side).

Request:

```
{
  "subdomain": "myapp",
  "target": "localhost:3000",
  "auth_required": true
}
```

Response (201):

```
{
  "id": "tun-abc-123",
  "url": "https://myapp.pranor.net",
  "status": "active",
  "created_at": "2026-08-01T10:00:00Z"
}
```

GET /api/v1/tunnels/{id}/requests

Browse captured requests from ring buffer.

Response (200):

```
{
  "requests": [
    {
      "id": "req-001",
      "method": "POST",
      "path": "/webhooks",

```

```
"status": 200,  
"latency_ms": 43,  
"timestamp": "2026-08-01T10:01:00Z"  
}  
]  
}
```

POST /api/v1/tunnels/{id}/replay/{reqID}

Replay a captured request to the local service.

Response (200):

```
{  
  "status": "replayed",  
  "response_status": 200,  
  "latency_ms": 38  
}
```

POST /api/v1/tunnels/{id}/share

Generate a shareable URL with expiry.

Request:

```
{  
  "expires_in": "1h",  
  "one_time": false  
}
```

Response (200):

```
{  
  "url": "https://myapp.pranor.net?token=xyz789",  
  "expires_at": "2026-08-01T11:00:00Z"  
}
```

GET /healthz

Liveness probe.

```
{"status": "UP", "service": "pranor-tunnel", "version": "0.1.0"}
```

GET /readyz

Readiness probe.

```
{"status": "UP", "service": "pranor-tunnel", "version": "0.1.0"}
```

Security

Authentication

- **JWT auth gating:** Set ``PRANOR_TUNNEL_JWT_SECRET`` to require valid JWT for tunnel connections
- **API key:** Pass a static token via ``--token`` flag or ``Authorization: Bearer `` header
- **Basic auth protection:** Use ``--share-auth usr:pwd`` to add HTTP Basic Auth to the public tunnel URL

Shareable URLs

- Time-limited URLs with configurable expiry
- One-time access tokens that invalidate after first use
- Shareable links include embedded auth tokens

TLS

Configure TLS for encrypted public-facing connections:

- Set ``PRANOR_TUNNEL_TLS_CERT`` and ``PRANOR_TUNNEL_TLS_KEY``
- Wildcard certificate recommended for ``*.pranor.net``

DNS Configuration

Configure a wildcard DNS record: ``*.pranor.net`` → `tunnel-server-ip``

Observability

Prometheus Metrics

Metric	Type	Description
----- ----- -----		
<code>`pranor_tunnel_active_connections`</code>	Gauge	Active WebSocket tunnel connections
<code>`pranor_tunnel_requests_proxied_total`</code>	Counter	Total requests forwarded
<code>`pranor_tunnel_request_latency_ms`</code>	Histogram	End-to-end proxy latency
<code>`pranor_tunnel_reconnections_total`</code>	Counter	Client reconnection events
<code>`pranor_tunnel_ring_buffer_size`</code>	Gauge	Captured requests in buffer

OpenTelemetry Tracing

Tunnel propagates ``traceparent`` and ``tracestate`` headers through the tunnel. Additionally emits:

- ``tunnel.proxy`` — request proxy span
- ``tunnel.replay`` — request replay span
- ``tunnel.connect`` — WebSocket connection establishment

Logging

Real-time request log in terminal client:

```
[2026-08-01 11:42:00] POST /webhook/payment 200 43ms
[2026-08-01 11:42:01] GET /api/orders/123 200 12ms
[2026-08-01 11:42:03] POST /webhook/payment 500 89ms ← error
```

Enterprise Edition

| Feature | OSS | EE |

|-----|:---|:---|

| Subdomain-based routing | ✓ | ✓ |

| WebSocket multiplexing | ✓ | ✓ |

| Request inspection & replay | ✓ | ✓ |

| JWT / API key auth gating | ✓ | ✓ |

| Shareable URLs with expiry | ✓ | ✓ |

| Git branch auto-subdomain | ✓ | ✓ |

| Multi-port tunneling (config file) | ✓ | ✓ |

| Persistent reconnect with backoff | ✓ | ✓ |

| WireGuard end-to-end encryption | — | ✓ |

| Custom domain mapping | — | ✓ |

| Team tunnel sharing (RBAC) | — | ✓ |

| Rate limiting per tunnel | — | ✓ |

| Request throttling | — | ✓ |

Operational Runbook

Client cannot connect to relay

1. Verify relay URL is correct (`--relay ws://...`)
2. Check if JWT token is required and valid
3. Verify network allows WebSocket connections (port 8443)
4. Check if firewall/proxy is stripping `Upgrade: websocket` headers
5. Try with explicit `--subdomain` to rule out auto-generation issues

Tunnel URL returning 502

1. Verify local service is running on the specified port
2. Check client terminal for connection errors
3. Verify WebSocket connection is active (not reconnecting)
4. Check ring buffer for request/response details
5. Review local service logs for errors

Requests not appearing in inspection buffer

1. Check `PRANOR\_TUNNEL\_MAX\_RING\_BUFFER` isn't set to 0
2. Verify inspection port is accessible (default: 4040)
3. Old requests may have been evicted (buffer is fixed-size ring)
4. Ensure the request went through the tunnel (not direct)

Reconnection loop (client keeps disconnecting)

1. Check server logs for auth rejection
2. Verify token hasn't expired
3. Check network stability between client and relay
4. Review reconnection backoff settings (max retries, max delay)
5. If the server restarted, subdomain may have been reassigned

Pranor Vault — S3-Compatible Object Storage

Version: 2.0.0

Module Path: ``github.com/vyuvaraj/pranor/vault``

Default Ports: 9000 (S3 API), 9001 (Admin Console)

License: AGPL-3.0 (OSS) / Enterprise License (EE with Multi-Region Replication, CoW Branching, Envelope Encryption)

Overview

Pranor Vault is a production-grade S3-compatible object storage engine with embedded vector search, time-travel versioning, erasure coding, bucket branching, WASM transform pipelines, tiered cold storage, and a full admin console. It implements the AWS S3 API specification enabling drop-in compatibility with existing S3 clients, SDKs, and tools.

Pranor Vault can run as:

- A **standalone daemon** (``pranor-vaultd``) with zero external dependencies
- An **integrated module** within the Pranor ecosystem with mTLS, RBAC, OTel tracing, and Console visibility
- A **Kubernetes-native store** via CSI driver and Helm charts
- A **distributed cluster** with Raft consensus, consistent hashing, and erasure coding

Table of Contents

- [Key Features](#key-features)
- [Architecture](#architecture)
- [Installation & Deployment](#installation--deployment)
- [Configuration](#configuration)
- [API Reference](#api-reference)
- [S3 Compatibility](#s3-compatibility)
- [Storage Engine](#storage-engine)
- [Security](#security)
- [Observability](#observability)
- [Client Libraries & CLI](#client-libraries--cli)
- [Enterprise Edition](#enterprise-edition)
- [Operational Runbook](#operational-runbook)

Key Features

Feature	Description
Full S3 API	GET, PUT, DELETE, HEAD, ListBuckets, ListObjects, Multipart Upload, S3 Select.
Vector Search	Embedded HNSW index for semantic similarity search over stored objects.

| **Time-Travel Versioning** | Access any historical version of an object — full version history with delete markers. |

| **Erasure Coding** | Reed-Solomon data/parity sharding across cluster nodes for fault tolerance. |

| **Bucket Branching** | Copy-on-Write (CoW) branch terabyte buckets instantly for sandbox development. |

| **WASM Pipelines** | Transform objects in-flight using WebAssembly modules (resize, transcode, redact). |

| **Tiered Cold Storage** | Automatic lifecycle rules sweeping objects to cold storage tier. |

| **Object Locking (WORM)** | Immutable object retention for compliance — legal hold and governance modes. |

| **Bucket Lifecycle** | Configurable expiration and transition rules per bucket. |

| **S3 Select** | Query object content with SQL (CSV, JSON, Parquet). |

| **Event Notifications** | Webhook and STOMP-based notifications on object create/delete events. |

| **Batch Operations** | Bulk copy, delete, and tag operations across large object sets. |

| **Object Tagging** | Key-value metadata tags on objects for classification and lifecycle filtering. |

| **Geo-Placement** | Per-bucket geographic data residency placement policies. |

| **Federation** | Cross-cluster bucket routing via pattern-based federation rules. |

| **Rate Limiting** | Per-tenant token-bucket rate limiting with Retry-After headers. |

| **SQL Metadata Query** | Query bucket metadata using SQL syntax. |

| **Conversational Query** | Natural language "ask" interface for semantic object discovery. |

| **CSI Driver** | Kubernetes Container Storage Interface for pod-mounted object storage. |

| **Helm Charts** | Production Helm charts for Kubernetes deployment. |

| **Access Audit Logging** | Structured access logs stored in `system-access-logs` bucket. |

| **Console Web UI** | Built-in admin console for bucket management and monitoring. |

| **Static Site Hosting** | Serve any bucket as a static website with MIME detection and index fallback. |

Architecture

```
graph TD
    subgraph API ["S3-Compatible API Layer"]
        S3["S3 REST API :9000"]
        Admin["Admin API :9001"]
        Console["Web Console /ui/"]
    end

    subgraph Auth ["Auth and RBAC"]
        SigV4["AWS Signature V4 Verification"]
        RBAC["Policy-Based Access Control"]
        RateLimit["Per-Tenant Rate Limiter"]
    end

    subgraph Engine ["Object Processing Engine"]
        S3Ops["S3 Operations Engine"]
    end
```

```
Vector["Vector Search HNSW Index"]
WASMPipe["WASM Transform Pipeline"]
Federation["Federation Router"]
end

subgraph Cluster ["Distributed Cluster Layer"]
  Raft["Raft Consensus Leader Election"]
  HashRing["Consistent Hash Ring Placement"]
  Erasure["Reed-Solomon Erasure Coding"]
  CRR["Cross-Region Replication"]
end

subgraph Storage ["Persistence Layer"]
  LocalStore["Content-Addressed Local Store"]
  Versioning["Version Metadata Engine"]
  ColdTier["S3 Cold Storage Tier"]
  WAL["Write-Ahead Log"]
end

S3 --> SigV4
Admin --> SigV4
Console --> SigV4
SigV4 --> RBAC
RBAC --> RateLimit
RateLimit --> S3Ops
RateLimit --> Vector
RateLimit --> WASMPipe
S3Ops --> Raft
Raft --> HashRing
HashRing --> Erasure
Erasure --> LocalStore
LocalStore --> Versioning
LocalStore --> ColdTier
Versioning --> WAL
Federation --> CRR
```

Object Lifecycle Sequence Flow

```
sequenceDiagram
    autonumber
    participant Client as S3 Client
    participant Gate as S3 API Gateway
    participant Auth as SigV4 Auth Layer
    participant Engine as S3 Operations Engine
    participant Cluster as Cluster Placement
    participant Store as Storage Engine
    participant Notify as Event Notifier

    Client->>Gate: PUT /bucket/key (Object Upload)
    Gate->>Auth: Verify AWS Signature V4
    Auth-->>Gate: Authenticated (Access Key + Policy)
    Gate->>Engine: Process PutObject Request
    Engine->>Cluster: Determine Placement via Hash Ring
    Cluster->>Store: Write Object Data + Version Metadata
    Store-->>Cluster: Write Committed (ETag Generated)
    Cluster-->>Engine: Placement Confirmed
    Engine->>Notify: Emit s3:ObjectCreated Event
    Notify-->>Engine: Webhook Dispatched
    Engine-->>Gate: 200 OK (ETag, VersionId)
    Gate-->>Client: HTTP 200 with ETag Header
```

Ecosystem Cross-Module Integration

Pranor Vault serves as the primary data persistence layer across the Pranor platform:

- **Pranor Pulse:** Receives closed WAL segments offloaded to S3 buckets for cold archive retention. Vault also emits object event notifications to Pulse topics.
- **Pranor Auth:** Validates JWT tokens and enforces RBAC bucket policies. OIDC and LDAP integration for enterprise environments.
- **Pranor Trace:** Every S3 operation generates an OTel span with trace context propagation across cluster nodes.
- **Pranor Console:** Provides bucket management dashboard, storage capacity monitoring, and object browsing UI.
- **Pranor Hub:** Uses Vault as the backing store for package artifacts (tarballs, WASM modules, metadata).
- **Pranor Secret:** Fetches encryption keys for server-side object encryption (SSE-KMS mode).

Installation & Deployment

Binary

```
cd pranor/vault
CGO_ENABLED=0 go build -o pranor-vaultd ./cmd/pranor-vaultd
CGO_ENABLED=0 go build -o pranor-vault ./cmd/pranor-vault
./pranor-vaultd -port :9000 -admin-port :9001
```

Docker

```
docker run -p 9000:9000 -p 9001:9001 -v vault-data:/data \
ghcr.io/vyuvaraj/pranor-vault:latest
```

Docker Compose

```
services:
  vault:
    image: ghcr.io/vyuvaraj/pranor-vault:latest
    ports:
      - "9000:9000"
      - "9001:9001"
    volumes:
      - vault-data:/data
    environment:
      - AWS_ACCESS_KEY_ID=minioadmin
      - AWS_SECRET_ACCESS_KEY=minioadmin
    volumes:
      vault-data:
```

Kubernetes (Helm)

```
helm install pranor-vault ./deploy/helm \
--set storage.size=100Gi \
--set replication.factor=3 \
--set erasure.enabled=true
```

CSI Driver

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: pranor-vault-csi
  provisioner: vault.csi.pranor.io
  parameters:
    bucket: my-app-data
  endpoint: http://pranor-vault:9000
```


As Part of Pranor Ecosystem

When running under the Pranor platform, Vault integrates automatically with Auth (JWT/mTLS), Secret (encryption keys), Trace (OTel spans), Pulse (event notifications), and Console (dashboard visibility).

Configuration

JSON Config (`config.json`)

```
{
  "addr": ":9000",
  "admin_addr": ":9001",
  "data_dir": "./data",
  "enable_web_admin": true,
  "default_buckets": ["default-bucket"]
}
```

Environment Variables

Variable	Default	Description
-----	-----	-----
<code>`AWS_ACCESS_KEY_ID`</code>	<code>`minioadmin`</code>	S3 access key for authentication
<code>`AWS_SECRET_ACCESS_KEY`</code>	<code>`minioadmin`</code>	S3 secret key for authentication
<code>`PORT`</code>	<code>`:9000`</code>	S3 API listening port
<code>`ADMIN_PORT`</code>	<code>`:9001`</code>	Admin console listening port
<code>`PRANOR_OTLP_ENDPOINT`</code>	—	OpenTelemetry collector URL
<code>`PRANOR_VAULT_DATA_DIR`</code>	<code>`./data`</code>	Storage data directory

CLI Flags

Flag	Default	Description
-----	-----	-----
<code>`-port`</code>	<code>`:9000`</code>	S3 API listening port
<code>`-admin-port`</code>	<code>`:9001`</code>	Admin console listening port
<code>`-config`</code>	<code>`config.json`</code>	Path to configuration file
<code>`-version`</code>	—	Show version and exit

API Reference

S3-Compatible API (Port 9000)

Pranor Vault implements the AWS S3 REST API. All standard S3 clients work without modification.

Authentication: AWS Signature V4 (compatible with `aws-cli`, boto3, MinIO client).

GET / — List Buckets

```
<?xml version="1.0" encoding="UTF-8"?>
<ListAllMyBucketsResult>
  <Owner>
    <ID>pranor-vault-owner</ID>
    <DisplayName>Pranor Vault Admin</DisplayName>
  </Owner>
  <Buckets>
    <Bucket>
      <Name>my-bucket</Name>
      <CreationDate>2026-01-15T10:00:00Z</CreationDate>
    </Bucket>
  </Buckets>
</ListAllMyBucketsResult>
```

PUT /{bucket} — Create Bucket

```
aws s3 mb s3://my-bucket --endpoint-url http://localhost:9000
```

DELETE /{bucket} — Delete Bucket

```
aws s3 rb s3://my-bucket --endpoint-url http://localhost:9000
```

GET /{bucket} — List Objects

Query parameters: `prefix`, `delimiter`, `max-keys`, `continuation-token`

```
aws s3 ls s3://my-bucket/ --endpoint-url http://localhost:9000
```

PUT /{bucket}/{key} — Put Object

```
aws s3 cp ./file.txt s3://my-bucket/path/file.txt --endpoint-url http://localhost:9000
```

Response includes `ETag` and optional `x-amz-version-id`.

GET /{bucket}/{key} — Get Object

```
aws s3 cp s3://my-bucket/path/file.txt ./file.txt --endpoint-url http://localhost:9000
```

Query param `?versionId=` retrieves a specific historical version.

DELETE /{bucket}/{key} — Delete Object

Creates a delete marker (versioned) or permanently removes (unversioned).

HEAD /{bucket}/{key} — Head Object

Returns metadata without body (Content-Type, Content-Length, ETag, version headers).

Multipart Upload

For large objects (>5MB recommended):

```
# Initiate
POST /{bucket}/{key}?uploads

# Upload parts
PUT /{bucket}/{key}?uploadId={id}&partNumber;={n}

# Complete
POST /{bucket}/{key}?uploadId={id}

# Abort
DELETE /{bucket}/{key}?uploadId={id}
```

POST /{bucket}/{key}?select — S3 Select

Query object contents with SQL:

```
{
  "Expression": "SELECT s.name, s.age FROM S3Object s WHERE s.age > 30",
  "InputSerialization": {"JSON": {"Type": "LINES"}},
  "OutputSerialization": {"JSON": {}}
}
```

PUT /{bucket}?versioning — Enable Versioning

```
<VersioningConfiguration>
<Status>Enabled</Status>
</VersioningConfiguration>
```

GET /{bucket}?versions — List Object Versions

Returns all versions including delete markers for time-travel access.

PUT /{bucket}/{key}?lock — Object Lock (WORM)

Enable immutable retention on an object.

PUT /{bucket}/{key}?tagging — Object Tagging

```
<Tagging>
<TagSet>
```

```
<Tag><Key>environment</Key><Value>production</Value></Tag>
</TagSet>
</Tagging>
```

PUT /{bucket}?lifecycle — Bucket Lifecycle Rules

Configure expiration and tier transitions:

```
<LifecycleConfiguration>
  <Rule>
    <ID>expire-old-logs</ID>
    <Status>Enabled</Status>
    <Expiration><Days>90</Days></Expiration>
    <Filter><Prefix>logs</Prefix></Filter>
  </Rule>
</LifecycleConfiguration>
```

PUT /{bucket}?cold-tier — Configure Cold Tier

Set up tiered storage for infrequently accessed objects.

POST /{bucket}?cold-tier&sweep; — Run Cold Sweep

Manually trigger cold storage sweep for a bucket.

PUT /{bucket}?notification — Event Notifications

```
{
  "bucket": "uploads",
  "events": ["s3:ObjectCreated:*", "s3:ObjectRemoved:*"],
  "webhook": "https://myapp.com/hook"
}
```

PUT /{bucket}?triggers — Bucket Triggers

Configure WASM triggers that execute on object events.

PUT /{bucket}?geo-placement — Geo-Placement Policy

Set geographic data residency requirements per bucket.

POST /{bucket}?pipeline — WASM Pipeline

Execute a WASM transform pipeline on objects in a bucket.

POST /{bucket}/{key}?transform&target-key;={output} — WASM Transform

Transform a single object using a registered WASM module.

POST /{bucket}?delete — Batch Delete

Delete multiple objects in a single request:

```
<Delete>
<Object><Key>file1.txt</Key></Object>
<Object><Key>file2.txt</Key></Object>
</Delete>
```

GET /{bucket}?ask={query} — Conversational Query

Natural language semantic search over bucket contents:

```
GET /my-bucket?ask=find+all+invoices+from+2024
```

GET /{bucket}?sql={query} — SQL Metadata Query

Query bucket metadata using SQL syntax.

Admin API (Port 9001)

GET /api/v1/health

```
{
  "status": "UP",
  "version": "2.0.0",
  "uptime_sec": 3600.5,
  "bucket_count": 5,
  "daemon": "pranor-vaultd"
}
```

GET /api/v1/buckets

List all bucket names.

```
["default-bucket", "uploads", "archive"]
```

POST /api/v1/buckets

Create a bucket.

```
{"name": "new-bucket"}
```

POST /api/v1/events/subscribe

Subscribe to bucket event webhooks.

```
{
  "bucket": "uploads",

```

```
"events": ["s3:ObjectCreated:*"],
"webhook": "https://myapp.com/hook"
}

---

#### GET /ui/
Built-in web console for bucket management, object browsing, and monitoring.

---

#### GET /metrics
Prometheus-compatible metrics endpoint.

---

#### POST /admin/backup/restore
Trigger a backup restore operation.

#### POST /admin/federation
Register a federation routing rule.

#### POST /admin/batch
Create a batch operations job (bulk copy, delete, tag).

#### GET /admin/batch/{jobId}
Check batch job status.

---

#### POST /console/login
Authenticate to the web console.

#### POST /console/logout
End console session.

#### GET /console/session
Validate current console session.

---
```

S3 Compatibility

Supported Operations

Operation	Status	Notes
----- ----- -----		
ListBuckets	✓	Full support
CreateBucket	✓	Full support
DeleteBucket	✓	Must be empty
HeadBucket	✓	Full support
ListObjects (v1/v2)	✓	Prefix, delimiter, pagination
PutObject	✓	With ETag, versioning

|GetObject | ✓ | Range requests, version selection |

|DeleteObject | ✓ | Delete markers for versioned buckets |

|HeadObject | ✓ | Full metadata |

|CopyObject | ✓ | Cross-bucket copy |

|Multipart Upload | ✓ | Initiate, Upload Part, Complete, Abort |

|Object Versioning | ✓ | Enable/Suspend, list versions |

|Object Tagging | ✓ | Put, Get, Delete tags |

|Object Lock (WORM) | ✓ | Governance and compliance modes |

|Bucket Lifecycle | ✓ | Expiration, transitions |

|S3 Select | ✓ | SQL on JSON/CSV/Parquet |

|Batch Delete | ✓ | Multi-object delete |

|Bucket Notifications | ✓ | Webhook + STOMP |

|Pre-signed URLs | ✓ | Standard AWS signature |

Compatible Clients

- **AWS CLI** — `aws s3 --endpoint-url http://localhost:9000``
- **boto3** (Python) — set ``endpoint_url`` parameter
- **MinIO Client** (`mc``) — `mc alias set vault http://localhost:9000 minioadmin minioadmin``
- **Go AWS SDK** — custom endpoint configuration
- **s3cmd** — configure with Vault endpoint
- **rclone** — S3-compatible provider

Storage Engine

Local Storage (Default)

Content-addressed object storage on the local filesystem. Objects are stored in a PebbleDB-backed engine with:

- B-tree indexed metadata
- Content-addressable deduplication
- Atomic write guarantees
- Crash-safe recovery

Erasure Coding

Reed-Solomon erasure coding distributes data across cluster nodes:

Default: 2 data shards + 1 parity shard

Any 2 of 3 shards can reconstruct the original object. Configured via:

`NewGateway(store, auth, raftNode, clusterMgr, replicationFactor, erasureEnabled, dataShards, parityShards)`

Consistent Hash Ring

Objects are placed on cluster nodes using a consistent hash ring. The ring determines:

- Which nodes own a given object (bucket/key hash)
- Replication targets (next N nodes on ring)
- Request routing (proxy to owner if not local)

Raft Consensus

Leader election and log replication for strong consistency of bucket-level operations (create, delete, versioning config).

Time-Travel Versioning

When versioning is enabled, every PUT creates a new version. Previous versions remain accessible by `versionId`:

```
PUT /bucket/key → version v1
PUT /bucket/key → version v2 (v1 still accessible)
DELETE /bucket/key → delete marker (v1, v2 still accessible)
GET /bucket/key?versionId=v1 → returns original content
```

Cold Storage Tiering

Lifecycle rules automatically move infrequently accessed objects to a cold storage tier:

```
Hot tier (SSD/local) → 30 days → Cold tier (S3-compatible remote)
```

Security

AWS Signature V4 Authentication

Standard S3 authentication using access key / secret key pairs:

```
export AWS_ACCESS_KEY_ID=minioadmin
export AWS_SECRET_ACCESS_KEY=minioadmin
```

RBAC Authorization

Role-based access control with per-bucket and per-action policies. Evaluated after authentication.

Rate Limiting

Per-tenant token-bucket rate limiting:

```
X-Pranor-Vault-Namespace: tenant-a
→ Rate limited independently per tenant
→ 429 Too Many Requests with Retry-After header on exhaustion
```

Object Lock (WORM)

Immutable object retention for regulatory compliance:

- **Governance mode** — privileged users can override
- **Compliance mode** — no one can delete until retention expires
- **Legal hold** — indefinite immutability flag

Access Audit Logging

Every S3 operation is logged to the ``system-access-logs`` bucket:

```
{
  "request_id": "trace-id-abc",
  "timestamp": "2026-01-15T10:00:00Z",
  "requester": "admin",
  "bucket": "uploads",
  "key": "data/file.csv",
  "operation": "GET",
  "source_ip": "10.0.1.5:54321",
  "status": 200
}
```

TLS / mTLS

Configure TLS for the S3 API endpoint. In ecosystem mode, mTLS is available for service-to-service communication.

Console Authentication

The web admin console has its own session-based login separate from S3 credentials.

Observability

Prometheus Metrics

Metric	Type	Description
pranor_vault_http_requests_total	Counter	Total S3 API requests (method, path, status)
pranor_vault_request_duration_seconds	Histogram	Request latency distribution
pranor_vault_inflight_requests	Gauge	Currently processing requests
pranor_vault_objects_total	Gauge	Total stored objects
pranor_vault_storage_bytes	Gauge	Total storage consumed

OpenTelemetry Tracing

Every S3 operation generates an OTEL span with:

- ``http.method``, ``http.route``, ``http.status_code``
- Trace ID propagation via ``traceparent`` header
- Child spans for cluster operations, erasure coding, WASM transforms

Structured JSON Logging

All requests are logged with structured fields:

```
{
  "level": "INFO",
```

```
"msg": "Request completed",
"method": "PUT",
"path": "/uploads/file.txt",
"status": 200,
"duration": "12.3ms",
"trace_id": "abc123"
}
```

Web Console Dashboard

Access `/ui/` on the admin port for real-time monitoring:

- Bucket list with object counts
- Upload/download throughput
- Cluster node health
- Storage capacity utilization

Client Libraries & CLI

AWS CLI

```
# Configure
aws configure
# Access Key: minioadmin
# Secret Key: minioadmin
# Region: us-east-1

# Create bucket
aws s3 mb s3://my-bucket --endpoint-url http://localhost:9000

# Upload
aws s3 cp ./data.csv s3://my-bucket/data/file.csv --endpoint-url http://localhost:9000

# Download
aws s3 cp s3://my-bucket/data/file.csv ./local.csv --endpoint-url http://localhost:9000

# List
aws s3 ls s3://my-bucket/ --endpoint-url http://localhost:9000

# Delete
aws s3 rm s3://my-bucket/data/file.csv --endpoint-url http://localhost:9000

# Sync directory
aws s3 sync ./local-dir s3://my-bucket/backup/ --endpoint-url http://localhost:9000
```

MinIO Client (mc)

```
mc alias set vault http://localhost:9000 minioadmin minioadmin
mc mb vault/my-bucket
mc cp ./file.txt vault/my-bucket/
mc ls vault/my-bucket/
mc cat vault/my-bucket/file.txt
```

Python (boto3)

```
import boto3

s3 = boto3.client('s3',
endpoint_url='http://localhost:9000',
aws_access_key_id='minioadmin',
aws_secret_access_key='minioadmin'
)

# Create bucket
s3.create_bucket(Bucket='my-bucket')

# Upload
s3.put_object(Bucket='my-bucket', Key='data/file.json', Body=b'{"hello":"world"}')

# Download
response = s3.get_object(Bucket='my-bucket', Key='data/file.json')
content = response['Body'].read()

# List objects
response = s3.list_objects_v2(Bucket='my-bucket', Prefix='data/')
for obj in response.get('Contents', []):
    print(obj['Key'], obj['Size'])

# Vector search
response = s3.select_object_content(
    Bucket='my-bucket',
    Key='embeddings.jsonl',
    Expression="SELECT * FROM S3Object WHERE similarity > 0.8",
    ExpressionType='SQL',
    InputSerialization={'JSON': {'Type': 'LINES'}},
    OutputSerialization={'JSON': {}}
)
```

Go

```
import (
    "github.com/aws/aws-sdk-go-v2/config"
    "github.com/aws/aws-sdk-go-v2/service/s3"
)

cfg, _ := config.LoadDefaultConfig(context.TODO(),
    config.WithEndpointResolver(aws.EndpointResolverFunc(
        func(service, region string) (aws.Endpoint, error) {
            return aws.Endpoint{URL: "http://localhost:9000"}, nil
        },
    )),
)

client := s3.NewFromConfig(cfg)
_, err := client.PutObject(context.TODO(), &s3.PutObjectInput{
    Bucket: aws.String("my-bucket"),
    Key: aws.String("data/file.txt"),
    Body: strings.NewReader("hello world"),
})
```

Pranor CLI

```
pranor vault buckets list
pranor vault buckets create my-bucket
pranor vault upload ./file.txt my-bucket/path/file.txt
pranor vault download my-bucket/path/file.txt ./local.txt
pranor vault ls my-bucket/path/
pranor vault bench --bucket test-bucket --objects 10000
pranor vault import --source s3://existing/data --target local-bucket
pranor vault serve-static --bucket my-site --port 3000
```

cURL (Direct S3 API)

```
# List buckets (requires proper AWS Sig V4 – simplified with mc or aws-cli)
curl http://localhost:9000/ \
-H "Authorization: AWS4-HMAC-SHA256 ..."

# Health check (no auth required)
curl http://localhost:9000/healthz
```

Enterprise Edition

Feature OSS EE
----- :--- :---
Full S3 API ✓ ✓
Local storage engine ✓ ✓
Object versioning & time-travel ✓ ✓
Multipart upload ✓ ✓
Object tagging ✓ ✓
Bucket lifecycle rules ✓ ✓
S3 Select (SQL queries) ✓ ✓
WASM transform pipelines ✓ ✓
Event notifications (webhook/STOMP) ✓ ✓
Batch operations ✓ ✓
Rate limiting ✓ ✓
Prometheus metrics & OTel tracing ✓ ✓
Console Web UI ✓ ✓
CSI driver & Helm charts ✓ ✓
Vector search (HNSW) ✓ ✓
Federation routing ✓ ✓
Static site hosting ✓ ✓
Immutable object access audit trail — ✓
Active-active multi-region replication — ✓
Copy-on-Write (CoW) bucket branching — ✓
Sovereign client envelope encryption — ✓
Erasure coding cluster — ✓
Raft consensus replication — ✓
Geo-placement data residency — ✓

Operational Runbook

Object not found (404)

1. Verify bucket exists: ``aws s3 ls --endpoint-url http://localhost:9000``
2. Check if object was deleted — list versions: ``GET /bucket?versions``
3. If versioned, retrieve by version ID: ``GET /bucket/key?versionId=v1``
4. Check federation rules — object may be on a remote cluster

Upload failing (403 Access Denied)

1. Verify credentials: ``AWS_ACCESS_KEY_ID`` and ``AWS_SECRET_ACCESS_KEY``

2. Check RBAC policy allows the operation on this bucket
3. Verify AWS Signature V4 is correctly computed (clock skew can cause failures)
4. Check rate limiting — 429 means tenant budget exhausted

Cluster node offline

1. Check `/metrics`` for cluster health indicators
2. Erasure coding tolerates ``parityShards`` node failures — data remains accessible
3. The consistent hash ring automatically routes to surviving owners
4. New writes target remaining healthy nodes
5. When the node recovers, rebalancing syncs missed data

High latency on large objects

1. Use multipart upload for objects > 5MB
2. Check erasure coding overhead — encoding adds CPU time
3. Verify cold tier sweep isn't running (blocks I/O during sweep)
4. Review OTel traces for bottleneck identification

Storage capacity approaching limit

1. Review lifecycle rules — ensure expiration is configured
2. Run cold tier sweep: ``POST /bucket?cold-tier&sweep;``
3. Check for orphaned multipart uploads: list and abort incomplete uploads
4. Review object versioning — old versions consume space

Bucket deletion failing

1. Bucket must be empty before deletion
2. Use batch delete to remove all objects first
3. Check for object lock (WORM) — locked objects cannot be deleted
4. Verify no active multipart uploads on the bucket

Federation routing not working

1. Check registered federation rules: ``GET /admin/federation``
2. Verify remote cluster is reachable from this node
3. Pattern matching is prefix-based — verify bucket name matches rule
4. Check auth credentials for cross-cluster communication

Console login issues

1. Console auth is separate from S3 credentials
2. Check session cookie validity
3. Verify admin port (9001) is accessible
4. Review console session endpoint: ``GET /console/session``

Versioning & Compatibility

- S3 API follows AWS S3 specification (2006-03-01 namespace)
- Admin API is versioned at `/api/v1/`
- Storage format is forward-compatible within major versions
- Object data is portable — can be migrated via standard S3 tools
- CSI driver follows CSI spec v1.x
- Helm charts follow Helm 3 conventions